Elementary Computational Mathematics

Research Training Project

Yimin Zhong

January 2, 2024

PREFACE

This note is the first component of the *Research Training Project* (RTP). The purpose is to provide a comprehensive introduction to basic computational mathematics for undergraduate and graduate students.

Prerequisites

1. Real Analysis, Complex Analysis, Functional Analysis, Harmonic Analysis (optional)
2. Linear Algebra, Abstract Algebra, Representation Theory (optional)
3. Differential Geometry
4. Probability, Stochastic Processes
5. Ordinary Differential Equations, Partial Differential Equations
6. Linear Programming, Convex Optimization
7. Scientific Programming (MATLAB, Python, Julia, C/C++, Fortran, Rust, etc.)
8. Lean (Proof Assistant Programming Language)

CONTENTS

I Preliminaries **1**

1 Floating Point Arithmetic **3**

1.1 Representation of Real Numbers 3

1.2 Floating Point Numbers 4

1.3 Rounding 5

1.4 Arithmetic Operations 6

1.4.1 Error Accumulation: Multiplication 6

1.4.2 Error Accumulation: Addition 7

1.5 Exercises 8

1.5.1 Theoretical Part 9

1.5.2 Computational Part 9

2 Interpolation **11**

2.1 Polynomial Interpolation 11

2.1.1 Lagrange Polynomial 12

2.1.2 Interpolation Error 13

2.1.3 Runge's Phenomenon 15

2.1.4 Interpolation Remainder Theory 16

2.1.5 Chebyshev Interpolation 17

2.1.6 Stability of Polynomial Interpolation 20

2.1.7 Newton Form 22

2.1.8 Hermite Polynomial Interpolation 25

2.2 Trigonometric Interpolation 26

2.2.1 Fourier Series . . . 26
2.2.2 Fast Fourier Transform . . . 28
2.2.3 Interpolation Error of Trigonometric Polynomial . . . 29
2.3 Spline Interpolation . . . 32
2.3.1 Linear Splines . . . 32
2.3.2 Cubic Splines . . . 33
2.3.3 B-Spline Representation . . . 36
2.3.4 B-Spline Interpolation . . . 39
2.4 Reproducing Kernel Hilbert Space . . . 39
2.4.1 Interpolation in RKHS . . . 41
2.4.2 Hardy Space $\mathbb{H}^2$ . . . 42
2.5 Exercises . . . 43
2.5.1 Theoretical Part . . . 43
2.5.2 Computational Part . . . 45

3 Differentiation and Quadrature **47**
3.1 Extrapolation . . . 47
3.1.1 Richardson Extrapolation . . . 47
3.1.2 Wynn's Epsilon Method . . . 49
3.2 Differentiation with Finite Difference . . . 50
3.2.1 Finite Difference from Taylor Expansion . . . 51
3.2.2 Rounding Error Issue . . . 54
3.2.3 Improve by Extrapolation . . . 54
3.3 Quadrature Rules . . . 56
3.3.1 Interpolation Based Rules . . . 56
3.3.2 Numerical Error of Interpolation Based Rules . . . 57
3.3.3 Newton-Cotes Formula . . . 58
3.3.4 Romberg Integration . . . 61
3.3.5 Adaptive Integrations . . . 62
3.3.6 Improper Integral . . . 63
3.4 Gauss Quadrature . . . 64
3.4.1 Orthogonal Polynomials . . . 64
3.4.2 The Riemann-Hilbert Problem . . . 66
3.4.3 Gauss Quadrature on Bounded Domain . . . 66
3.4.4 Gauss Quadrature on Unbounded Domain . . . 69
3.5 Probabilistic Integration . . . 72

3.5.1 Monte Carlo Integration . . . 73
3.5.2 Quasi-Monte Carlo Integration . . . 73
3.6 Exercises . . . 75
3.6.1 Theoretical Part . . . 75
3.6.2 Computational Part . . . 76

4 Approximation **79**
4.1 General Approximation Theory . . . 79
4.1.1 Orthogonality in L^p Spaces . . . 80
4.2 Minimax Approximation . . . 81
4.2.1 Remez Algorithm . . . 84
4.2.2 Polynomial Approximation for Analytic Functions . . . 84
4.2.3 Polynomial Approximation for Differentiable Functions . . . 87
4.3 Padé Approximation . . . 89
4.4 Neural Network . . . 89
4.4.1 Universal Approximation Theorem . . . 90
4.4.2 Shallow Neural Network . . . 90
4.4.3 Gradient-Flow . . . 90
4.5 Exercises . . . 90
4.5.1 Theoretical Part . . . 91
4.5.2 Computational Part . . . 91

5 Ordinary Differential Equations **93**
5.1 Initial Value Problem . . . 93
5.1.1 One-step Methods . . . 94
5.1.2 Absolute Stability . . . 96
5.1.3 Rounding Error . . . 97
5.1.4 Extrapolation Methods . . . 98
5.1.5 Adaptive One-step Methods . . . 98
5.1.6 Multistep Methods . . . 99
5.1.7 Adams Method . . . 102
5.2 Boundary Value Problem . . . 104
5.2.1 Finite Difference Method . . . 104
5.2.2 Galerkin Method . . . 106
5.3 Sturm Livouille Theory . . . 106
5.3.1 Orthogonal Polynomials . . . 106

5.3.2 Perturbation of Orthogonal Basis . . . 106
5.4 Exercises . . . 106
5.4.1 Theoretical Part . . . 106
5.4.2 Computational Part . . . 106

6 Partial Differential Equations 109
6.1 Finite Difference Method . . . 109
6.2 Finite Volume Method . . . 109
6.2.1 Godunov's Scheme . . . 109
6.3 Pseudo Spectral Method . . . 109
6.4 IMEX Method . . . 109
6.5 Sector Operators . . . 109
6.6 PINN . . . 109
6.7 Exercises . . . 109
6.7.1 Theoretical Part . . . 109
6.7.2 Computational Part . . . 109

7 Integral Equations 111
7.1 Nyström Method . . . 111
7.1.1 Weakly Singular Kernel . . . 111
7.1.2 Singular Kernel . . . 111
7.2 Galerkin Method . . . 111
7.3 Boundary Integrals . . . 111

8 Matrix Computation 113
8.1 Decompositions . . . 113
8.1.1 LU Decomposition . . . 113
8.1.2 QR Decomposition . . . 113
8.1.3 Schur Decomposition . . . 113
8.1.4 Singular Value Decomposition . . . 113
8.2 Special Matrices . . . 113
8.2.1 Sparse Matrices . . . 113
8.2.2 Positive Matrices . . . 113
8.2.3 Totally Positive Matrices . . . 113
8.2.4 M-Matrices . . . 114
8.2.5 Circulant and Toeplitz Matrices . . . 114
8.3 Iterative Schemes . . . 114

8.3.1 Jacobi/Gauss-Seidel Iterations 114
8.3.2 Relaxation Schemes 114
8.3.3 Chebyshev iterations 114
8.4 Krylov Subspace Methods 115
8.4.1 Conjugate Gradient 115
8.4.2 GMRES 115
8.5 Preconditioning 115
8.6 Fast Matrix-Vector Multiplication 115
8.6.1 Hierarchical Semiseparable Matrices 115
8.7 Sketch Algorithms 115

9 Tensor Computation **117**
9.1 Tensor Decomposition 117

II Foundations **119**

10 Gradient Descent **121**
10.1 Fixed Step Length 121
10.2 Step Length Scheduling 121
10.3 Momentum Methods 121

III Topics **123**

11 Extended Topics **125**
11.1 Radial Basis Interpolation 126
11.2 Quadrature in multi-dimension 126
11.3 Stiff ODE 126
11.4 Signal Processing 126
11.5 Physics Models 126
11.5.1 Ising Models 126
11.6 Randomized Algorithm 126
11.7 Variational Methods 126
11.8 Sampling Algorithm 126
11.9 Parallel Computing 126
11.10 Optimal Transport 126
11.11 Point Cloud 127

11.12 Low Rank Approximation 127
11.13 Computation on Graph 127

12 Matrix Inequalities **129**
12.1 Golden-Thompson inequality 129
12.2 Klein's inequality 129
12.3 Araki–Lieb–Thirring inequality 129
12.4 Exercises 129

13 Computational Inverse Problems **131**
13.1 Regularization 131
13.2 Unique Continuation 131

IV Miscellany **133**

14 Proof Assistant Programming **135**

V Solutions to Exercises **137**

15 Solutions to Chapter 1 **139**

Part I

Preliminaries

CHAPTER 1

FLOATING POINT ARITHMETIC

In this chapter, we will introduce some basics of the real number system for modern computers and discuss the arithmetic operations of the number system.

1.1 Representation of Real Numbers

Any *nonzero* real number $x \in \mathbb{R}$ can be accurately represented with an infinite sequence of digits. This can be understood as the consequence that rational numbers are dense on any interval. Therefore, with the binary representation, we can write

$$x = \pm(0.d_1 d_2 d_3 \dots d_{t-1} d_t d_{t+1} \dots) \times 2^e,$$

where e is an integer exponent and $d_1 = 1$, the other binary digits $d_i \in \{0, 1\}$. The mantissa part

$$0.d_1 d_2 d_3 \cdots = \frac{d_1}{2} + \frac{d_2}{2^2} + \frac{d_3}{2^3} + \cdots .$$

Remark 1.1.1. *To guarantee the uniqueness of the above representation, we need a further assumption that there exists an infinite subset $S \subset \mathbb{N}$ that $d_j \neq 1$ for $j \in S$. For example, under binary representation*

$$0.111 \cdots = (0.1) \times 2^1,$$

then we will take the latter representation.

1.2 Floating Point Numbers

The floating point numbers generally refer to a set of real numbers with *finite* mantissa length. More precisely, we consider the set of real numbers $\mathbb{F} = \mathbb{F}(t, e_{\text{min}}, e_{\text{max}}) \subset \mathbb{R}$ that

$$\mathbb{F} := \{x \in \mathbb{R} \mid x = \pm(0.d_1 d_2 d_3 \dots d_{t-1} d_t) \times 2^e, d_1 = 1, e_{\text{min}} \leq e \leq e_{\text{max}}\} \cup \{0\}.$$

It can be seen that there are only finite numbers in $\mathbb{F}$ with the smallest positive element $x_{\text{min}} = 2^{e_{\text{min}}-1}$ and the largest element $x_{\text{max}} = (1 - 2^{-t}) \times 2^{e_{\text{max}}}$. Therefore

$$\mathbb{F} \subset \overline{\mathbb{F}} := \{x \in \mathbb{R} \mid x_{\text{min}} \leq |x| \leq x_{\text{max}}\} \cup \{0\}.$$

Remark 1.2.1. *The elements in $\mathbb{F}$ are called normalized. If we allow $d_1 = 0$ in the definition of $\mathbb{F}$, then the numbers in the set are called denormalized.*

Theorem 1.2.2 (Distribution of floating numbers)**.** *For any $e_{\text{min}} \leq e \leq e_{\text{max}}$, the distribution of the floating point number system $\mathbb{F}$ on interval $[2^{e-1}, 2^e]$ is equidistant with distances of length 2^{e-t}.*

Proof. For any $x \in \mathbb{F} \cap [2^{e-1}, 2^e]$ it can be represented by

$$x = (0.d_1 d_2 \dots d_t) \times 2^e$$

where $d_1 = 1$. The mantissa is equidistantly distributed with distance 2^{-t}, therefore the floating point numbers are equidistantly distributed with distances of length 2^{e-t}. □

To understand the approximation to real numbers by the floating point number system $\mathbb{F}$, it is important to consider the maximal relative distance between the numbers in $\overline{\mathbb{F}}$ and their respective closest element in $\mathbb{F}$, which is the following quantity:

$$\max_{x \in \overline{\mathbb{F}}, x \neq 0} \min_{z \in \mathbb{F}} \frac{|z - x|}{|x|}.$$

The following holds:

Theorem 1.2.3 (Machine precision)**.**

$$\max_{x \in \overline{\mathbb{F}}, x \neq 0} \min_{z \in \mathbb{F}} \frac{|z - x|}{|x|} \leq 2^{-t}.$$

The number $\mathrm{u} := 2^{-t}$ *is also called rounding unit or machine precision.*

Proof. Without loss of generality, we only need to consider the positive numbers in $\overline{\mathbb{F}}$, then one can represent any nonzero $x \in [x_{\text{min}}, x_{\text{max}}]$ by

$$x = (0.d_1 d_2 \dots d_t \dots) \times 2^e \in [2^{e-1}, 2^e].$$

Since the floating point numbers are equidistantly distributed on $[2^{e-1}, 2^e]$ from Theorem 1.2.2, one can find $z^* \in \mathbb{F}$ such that

$$|z^* - x| \leq \frac{1}{2} 2^{e-t},$$

therefore

$$\frac{|z^* - x|}{|x|} \leq \frac{1}{2} 2^{e-t} \frac{1}{2^{e-1}} = 2^{-t}.$$

□

Remark 1.2.4. *On modern computers, the following two floating point number systems*

$$\mathbb{F}_{32} := \mathbb{F}(24, -125, 128), \quad \mathbb{F}_{64} := \mathbb{F}(53, -1021, 1024)$$

are supported, they are often called single precision and double precision, respectively.

1.3 Rounding

The rounding operation FL is to map any real numbers of $\overline{\mathbb{F}}$ into the floating point number system $\mathbb{F}$ with the smallest error. Such rounding operation can be written out explicitly, let $= \pm(0.d_1 d_2 \dots d_t d_{t+1} \dots) \times 2^e$, then

$$\text{FL}(x) = \begin{cases} \pm(0.d_1 d_2 \dots d_t) \times 2^e & \text{if } d_{t+1} = 0, \\ \pm(0.d_1 d_2 \dots d_t + 2^{-t}) \times 2^e & \text{if } d_{t+1} = 1. \end{cases}$$

It is clear that rounding FL is monotone and idempotent, which means

- $x \leq y \Rightarrow \text{FL}(x) \leq \text{FL}(y)$.
- $\text{FL}(z) = z$ if $z \in \mathbb{F}$.

Theorem 1.3.1 (Rounding error)**.** *For any $x \in \overline{\mathbb{F}}$, $|\text{FL}(x) - x| = \min_{z \in \mathbb{F}} |z - x|$. If $x \neq 0$, then*

$$\frac{|\text{FL}(x) - x|}{|x|} \leq \mathrm{u} = 2^{-t}.$$

Proof. The special case that $x = 0$ is trivial, we only consider $x \in [x_{\min}, x_{\max}]$, it can be seen that

$$|\text{FL}(x) - x| = |(0.d_1 d_2 \dots \tilde{d}_t) - (0.d_1 d_2 \dots d_t d_{t+1} \dots)| \times 2^e \leq 2^{-(t+1)} \times 2^e,$$

where $\tilde{d}_t$ is the rounding bit, therefore

$$\frac{|\text{FL}(x) - x|}{|x|} \leq \frac{2^{e-(t+1)}}{2^{e-1}} = 2^{-t}.$$

□

Corollary 1.3.2. *For any $x \in \overline{\mathbb{F}}$, $\text{FL}(x) = x(1 + \delta)$ with $|\delta| \leq \mathrm{u}$.*

1.4 Arithmetic Operations

Let $\mathbb{F} = \mathbb{F}(t, e_{\text{min}}, e_{\text{max}})$ be a given floating point number system and we consider the basic binary operation $\circ \in \{+, -, *, /\}$ on $\mathbb{F}$, to represent the outcome in $\mathbb{F}$, a straightforward realization is to define the binary operation $\boxed{\circ}$ as the following (for the case of division, we assume $y \neq 0$):

$$x \boxed{\circ} y := \text{FL}(x \circ y),$$

then for any $x, y \in \mathbb{F}$, if $x \circ y \in \overline{\mathbb{F}}$, then $x \boxed{\circ} y = (x \circ y)(1 + \delta)$ with $|\delta| \leq \mathrm{u}$ from the Corollary 1.3.2.

Remark 1.4.1 (Cancellation error). *If $x, y \in \mathbb{R}$, then the relative error from the following binary operation $\text{FL}(x) \boxed{+} \text{FL}(y)$ can be estimated by*

$$\begin{aligned}\frac{|\text{FL}(x) \boxed{+} \text{FL}(y) - (x + y)|}{|x + y|} &\leq \frac{|\text{FL}(x) \boxed{+} \text{FL}(y) - (\text{FL}(x) + \text{FL}(y))|}{|x + y|} + \frac{|(\text{FL}(x) + \text{FL}(y)) - (x + y)|}{|x + y|} \\ &\leq \mathrm{u} + (\mathrm{u} + \mathrm{u}^2)\frac{|x| + |y|}{|x + y|}.\end{aligned}$$

When x and y are close in magnitude but with opposite signs, the cancellation error will be significant.

1.4.1 Error Accumulation: Multiplication

For complicated computations on modern computers, the errors from arithmetic operations will accumulate towards the final result (we do not consider techniques such as fused multiply-add (FMA) here). To quantify the accumulation effect, we will need the following lemma.

Lemma 1.4.2. *For real numbers $a_1, a_2, \ldots, a_n$ with $|a_k| \leq \delta$ for $k = 1, \ldots, n$, then for $n\delta < 1$, the following holds*

$$\prod_{k=1}^{n}(1 + a_k) = 1 + b_n,$$

where $|b_n| \leq \frac{n\delta}{1-n\delta}$.

Proof. The proof is quite easy with induction. When $n = 1$, $|b_1| = |a_1| \leq \delta \leq \frac{\delta}{1-\delta}$. Suppose the claim holds for $n = m$, then for $n = m + 1$, we could see that

$$\prod_{k=1}^{m+1}(1 + a_k) = (1 + b_m)(1 + a_{m+1}) = 1 + b_{m+1},$$

which implies that $b_{m+1} = b_m + a_{m+1} + a_{m+1}b_m$, with the given bounds on a_{m+1} and b_m, we can estimate

$$|b_{m+1}| = |b_m + a_{m+1} + a_{m+1}b_m| \leq \frac{(m + 1)\delta}{1 - m\delta} \leq \frac{(m + 1)\delta}{1 - (m + 1)\delta}.$$

□

Next, we consider the naive floating point product P_n of n real numbers $\{x_j\}_{j=1}^n \subset \mathbb{R}$ with assumption that $(2n-1)\mathrm{u} < 1$ by the following iteration

$$\begin{cases} P_k = \mathrm{FL}(x_1) & k = 1, \\ P_k = P_{k-1} \boxed{*} \mathrm{FL}(x_k) & k \geq 2. \end{cases}$$

Let $\mathrm{FL}(x_k) = x_k(1+\tau_k)$, then $|\tau_k| \leq \mathrm{u}$. From the n-th iteration step

$$P_n = P_{n-1} \boxed{*} \mathrm{FL}(x_k) = \mathrm{FL}(\mathrm{FL}(P_{n-1})\mathrm{FL}(x_n)) = \mathrm{FL}(P_{n-1})\mathrm{FL}(x_n)(1+\delta_n)$$

such that $|\delta_n| \leq \mathrm{u}$, since $P_{n-1} \in \mathbb{F}$, $\mathrm{FL}(P_{n-1}) = P_{n-1}$, then

$$\begin{aligned} P_n &= P_{n-1}\mathrm{FL}(x_n)(1+\delta_n) = P_{n-2}\mathrm{FL}(x_{n-1})(1+\delta_{n-1})(1+\delta_k) = \cdots \\ &= \mathrm{FL}(x_1)\mathrm{FL}(x_2)\cdots\mathrm{FL}(x_n)\prod_{j=2}^{n}(1+\delta_j) = \prod_{j=1}^{n} x_j(1+\tau_j)\prod_{j=2}^{n}(1+\delta_j) \\ &\leq (1+\eta_n)\prod_{j=1}^{n} x_j, \end{aligned}$$

where $|\eta_n| \leq \frac{(2n-1)\mathrm{u}}{1-(2n-1)\mathrm{u}}$ by Lemma 1.4.2.

1.4.2 Error Accumulation: Addition

For naive floating point summation S_n of n real numbers $\{x_j\}_{j=1}^n$ by the iteration

$$\begin{cases} S_k = 0 & k = 0, \\ S_k = S_{k-1} \boxed{+} \mathrm{FL}(x_k) & k \geq 1, \end{cases}$$

we can carry out a similar analysis. Let $S_j^* = \sum_{k=1}^{j} x_k$ and $\mathrm{FL}(x_k) = x_k(1+\tau_k)$ for $|\tau_k| \leq \mathrm{u}$, denote $\Delta S_j = S_j^* - S_j$, then

$$\begin{aligned} \Delta S_j = S_j^* - S_j &= S_j^* - (S_{j-1} \boxed{+} \mathrm{FL}(x_j)) \\ &= \Delta S_{j-1}(1+\delta_j) - \delta_j S_j^* - x_j\tau_j(1+\delta_j), \end{aligned}$$

where $|\tau_j|, |\delta_j| \leq \mathrm{u}$. Therefore

$$
\begin{aligned}
|\Delta S_n| &\leq |\Delta S_{n-1}|(1+\mathrm{u}) + \mathrm{u}\sum_{k=1}^{n}|x_k| + |x_n|\mathrm{u}(1+\mathrm{u}) \\
&\leq |\Delta S_{n-2}|(1+\mathrm{u})^2 + \mathrm{u}(1+\mathrm{u})\sum_{k=1}^{j}|x_k| + |x_j|\mathrm{u}(1+\mathrm{u})^2 \\
&\quad + \mathrm{u}\sum_{k=1}^{j-1}|x_k| + |x_{j-1}|\mathrm{u}(1+\mathrm{u}) \\
&\leq \cdots \\
&\leq \sum_{l=1}^{n}\left(\mathrm{u}(1+\mathrm{u})^{l-1}\sum_{k=1}^{l}|x_k| + |x_l|\mathrm{u}(1+\mathrm{u})^{l-1}\right) \\
&\leq \sum_{l=1}^{n}\left(\mathrm{u}(1+\mathrm{u})^{l-1}\sum_{k=1}^{n}|x_k|\right) + \left(\sum_{l=1}^{n}|x_l|\right)\mathrm{u}(1+\mathrm{u})^{n-1} \\
&= \left(\sum_{l=1}^{n}|x_l|\right)((1+\mathrm{u})^n - 1).
\end{aligned}
$$

Using the previous Lemma 1.4.2 will provide an estimate of $((1+\mathrm{u})^n - 1)$ as long as $n\mathrm{u} < 1$.

1.5 Exercises

Assume $n \in \mathbb{N}$ and $n\mathrm{u} < 1$, let $\{x_j\}_{j=1}^n$ be a sequence of real numbers, $\mathrm{FL} : \overline{\mathbb{F}} \mapsto \mathbb{F}$ is the rounding operation, u is the machine epsilon. In the following, we briefly discuss the rounding error for floating point summation (sum reduction).

Definition 1.5.1. *A reduction Π of the floating point summation*

$$\mathit{FL}(x_1)\boxed{+}\mathit{FL}(x_2)\boxed{+}\cdots\boxed{+}\mathit{FL}(x_n)$$

is an evaluation order for the $\boxed{+}$ operations. For example, $n = 4$, then the reduction $\Pi = (3, 1, 2)$ is corresponding to the following calculation

$$\mathit{FL}(x_1)\boxed{+}\mathit{FL}(x_2) + \underbrace{(\mathit{FL}(x_3)\boxed{+}\mathit{FL}(x_4))}_{=y_1} \to \underbrace{(\mathit{FL}(x_1)\boxed{+}\mathit{FL}(x_2))}_{=y_2}\boxed{+}y_1 \to \underbrace{(y_2\boxed{+}y_1)}_{=y_3},$$

where y_i denotes the result of i-th $\boxed{+}$ operation in the reduction. The final summation will be y_{n-1}. We denote $T_n(\Pi)$ as the result of floating point summation with reduction order Π.

1.5.1 Theoretical Part

Problem 1.5.2. *Prove that the native summation has the following error estimate*

$$|T_n(\Pi) - S_n| \leq \left(\frac{\mathrm{u}n}{1-\mathrm{u}n}\right) \sum_{j=1}^{n} |x_j|, \tag{1.1}$$

where $S_n = \sum_{j=1}^{n} x_j$ *and* $\Pi = (1, 2, \ldots, (n-1))$.

Problem 1.5.3. *Prove that*

$$\min_{\Pi} |T_n(\Pi) - S_n| \leq \left(\frac{\mathrm{u}H}{1-\mathrm{u}H}\right) \sum_{j=1}^{n} |x_j|, \tag{1.2}$$

where $H = \lceil \log_2 n \rceil$ *and* $T_n(\Pi)$ *is the floating point summation with reduction order* Π.

Problem 1.5.4 (Horner's scheme)**.** *The evaluation of polynomial*

$$p(x) = a_0 + a_1 x + \cdots + a_n x^n$$

is mostly using Horner's scheme, which writes the polynomial in 'nested' form:

$$p(x) = a_0 + x(a_1 + x(a_2 + \ldots x(a_{n-1} + x(a_n)))). \tag{1.3}$$

Please find an upper bound of the rounding error for this scheme.

1.5.2 Computational Part

Problem 1.5.5 (Pairwise summation)**.** *Based on the theoretical part, implement an algorithm for the summation* $\sum_{j=1}^{n} x_j$ *which has* $\mathcal{O}(\mathrm{u} \log_2 n)$ *rounding error.*

Problem 1.5.6 (Kahan compensated summation)**.** *Suppose* $a, b \in \mathbb{R}$*, the rounding error for the sum* $s = \mathit{FL}(\mathit{FL}(a) + \mathit{FL}(b))$ *(*$a \geq b$*) can be computed using*

$$\mathit{FL}(\mathit{FL}(s - \mathit{FL}(a)) - \mathit{FL}(b))$$

Based on this property, one can keep track of the rounding error. Implement the Algorithm 1 described below and compare the accuracy with naive summation and pairwise summation with test cases.

Problem 1.5.7. *Suppose the inputs* $\{x_j\}_{j=1}^{n} \subset \mathbb{R}$ *are randomly distributed (say* $x_j \sim U(0,1)$ *i.i.d), what is the estimated growth of the expected rounding error in terms of the total number of inputs* n *for the naive summation and pairwise summation? Please explain your result. You can use Algorithm 1 as the accurate result approximately.*

Algorithm 1: Kahan compensated summation

Data: $\{x_j\}_{j=1}^n \subset \mathbb{R}$
Result: $s_n = \sum_{j=1}^n x_j$
$j \leftarrow 1$, $e_j \leftarrow 0$, $s_j \leftarrow x_j$ // initialization;
while $j < n$ **do**
  $j \leftarrow j + 1$
  $y_j = x_j - e_{j-1}$ //remove compensated error;
  $s_j = s_{j-1} + y_j$ //perform summation;
  $e_j = (s_j - s_{j-1}) - y_j$ //restore the rounding error;
end

Extended Reading

Higham, N. J. (1993). The accuracy of floating point summation. *SIAM Journal on Scientific Computing*, 14(4):783–799.

Higham, N. J. and Mary, T. (2019). A new approach to probabilistic rounding error analysis. *SIAM Journal on Scientific Computing*, 41(5):A2815–A2835.

Muller, J.-M. (2006). *Elementary functions*. Springer.

CHAPTER 2

INTERPOLATION

The interpolation solves problems of the following type:

> Given a set of predefined functions $\mathcal{K}$, find an element $f : \mathbb{I} \mapsto \mathbb{R}$ in $\mathcal{K}$ such that $y_j = f(x_j)$ for all $j = 0, \ldots, n$.

Here, $\mathbb{I}$ denotes a finite or infinite interval such that $x_1, \ldots x_n \in \mathbb{I}$. One of the important applications for interpolation is computer-assisted design (CAD), which is used extensively in the manufacturing industry. Generally speaking, the interpolation provides a closed form of the function to determine the value of y where the parameter x is not accessible.

2.1 Polynomial Interpolation

The polynomial interpolation considers the set $\mathcal{K} = \Pi_m$, where the set Π_m represents the polynomials of degree $\leq m$. We will seek for a polynomial $f(x)$ with the constraints that

$$\begin{cases} f \in \mathcal{K} = \Pi_n, \\ f(x_k) = y_k \qquad \text{for } k = 0, 1, \ldots, n. \end{cases}$$

The points x_k are called *interpolation nodes*, if $m > n$ (resp. $m < n$), the problem is underdetermined (resp. overdetermined). For the case that $m = n$, we have

Theorem 2.1.1. *There exists a unique polynomial function $f \in \Pi_n$ such that $f(x_j) = y_j$ for $j = 0, \ldots, n$.*

Proof. Existence: In order to construct the polynomial f, it is straightforward to consider the general form of polynomial $f(x) = \sum_{j=0}^{n} a_j x^j$, then we can formulate a linear system for the

coefficients a_j, $j = 0, \ldots, n$, which is

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

The matrix

$$V = \begin{pmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{pmatrix}$$

is called *Vandermonde matrix*. To determine the coefficients a_j, the matrix V must be invertible. Its determinant can be computed (try to prove it) as

$$\det(V) = \prod_{0 \le i \le j \le n} (x_j - x_i). \tag{2.1}$$

When $\{x_j\}_{j=0}^n$ are distinct, the determinant is nonzero.

Uniqueness: Suppose there are two distinct polynomials $f, g \in \Pi_n$ satisfying the condition that $f(x_j) = g(x_j) = y_j$, then $f - g$ has $(n+1)$ roots x_j, $j = 0, \ldots, n$. If $f \neq g$, it is clear that $f - g \in \Pi_n$ has at most n roots. Contradiction. □

In the above proof, the interpolation polynomial can be uniquely determined by solving the linear system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

However, it is generally easier to compute the polynomial f with the *Lagrange polynomial interpolation* (which is somewhat equivalent to computing the inverse of V).

2.1.1 Lagrange Polynomial

Definition 2.1.2. *For the given distinct x_j, $j = 0, 1, \ldots, n$, the Lagrange polynomials $L_j \in \Pi_n$, $j = 0, 1, \ldots, n$ are defined by*

$$L_j(x) = \prod_{s=0, s \neq j}^{n} \frac{x - x_s}{x_j - x_s}, \quad j = 0, 1, \ldots, n. \tag{2.2}$$

It is clear that these polynomials satisfy the conditions that

$$L_j(x_k) = \delta_{jk} := \begin{cases} 1 & \text{for } k = j, \\ 0 & \text{for } k \neq j. \end{cases} \tag{2.3}$$

Therefore these polynomials are linearly independent, which form a basis of the $(n+1)$-dimensional space Π_n.

Theorem 2.1.3. *The unique interpolating polynomial f satisfying $f(x_j) = y_j$, $j = 0, 1, \ldots, n$ can be represented by*

$$f(x) = \sum_{j=0}^{n} y_j L_j(x). \tag{2.4}$$

Proof. It is straightforward to check the interpolation conditions are satisfied. □

Remark 2.1.4. *We introduce a preliminary procedure to compute the value of the interpolating polynomial f at a point x. Let the constants k_j and $q(x)$ be defined as*

$$k_j = \prod_{s=0, s\neq j} \frac{1}{x_j - x_s}, \quad q(x) = \prod_{j=0}^{n} (x - x_j), \tag{2.5}$$

then

$$f(x) = \sum_{j=0}^{n} y_j L_j(x) = q(x) \sum_{j=0}^{n} k_j y_j \frac{1}{x - x_j}. \tag{2.6}$$

One can first compute k_j with $\mathcal{O}(n^2)$ flops, then $f(x)$ can be computed with $\mathcal{O}(n)$ flops. The advantage of the above scheme is the constants k_j are independent of y_j, therefore evaluating another instance of the interpolating polynomial will not need to re-compute them. The disadvantage is that if we add a new node, the constants k_j must be updated with an additional cost of $\mathcal{O}(n)$ flops. Later we will see if Newton's form can overcome this issue.

2.1.2 Interpolation Error

When the data pairs (x_j, y_j), $j = 0, 1, \ldots, n$ are generated by a sufficiently smooth function $h(x)$, it is possible to quantify the error between the interpolating polynomial $f(x)$ and $h(x)$.

Theorem 2.1.5. *Let $h : [a, b] \mapsto \mathbb{R}$ be a $(n+1)$-times differentiable function. If $f(x) \in \Pi_n$ is the interpolating polynomial that*

$$f(x_j) = h(x_j),$$

for $j = 0, 1, \ldots, n$. Then, for each $\overline{x} \in [a, b]$, the error can be represented by

$$h(\overline{x}) - f(\overline{x}) = \frac{\omega(\overline{x})}{(n+1)!} h^{(n+1)}(\xi), \tag{2.7}$$

where $\xi = \xi(\overline{x}) \in [a, b]$ and $\omega(x) = \prod_{j=0}^{n} (x - x_j)$.

Proof. The proof is based on Rolle's Theorem. Select any $\overline{x} \in [a, b]$ such that $\omega(\overline{x}) \neq 0$, then let

$$\psi(x) = h(x) - f(x) - k\omega(x),$$

the constant k is chosen such that $\psi(\overline{x}) = 0$. Then $\psi(x) = 0$ at $(n + 2)$ points,

$$x_0, x_1, \ldots, x_n, \overline{x} \in [a, b].$$

By Rolle's Theorem, $\psi^{(n+1)}$ has at least one zero ξ in $[a, b]$. Therefore,

$$\psi^{(n+1)}(\xi) = h^{(n+1)}(\xi) - 0 - k(n + 1)! = 0. \tag{2.8}$$

□

Corollary 2.1.6. *If $h(x) \in C^\infty([a, b])$ satisfies that $\max_{x\in[a,b]} |h^{(n)}(x)| \leq M < \infty$ for all $n \geq 0$, then the interpolating polynomial approximates h uniformly as the number of nodes $n \to \infty$.*

Proof. Since $|x - x_j| \leq b - a$, the error is bounded by $\frac{(b-a)^{n+1}}{(n+1)!}M$, which converges to zero. □

It is interesting to think about the converse: under what condition does the interpolation error not vanish as the number of nodes tends to infinity? From Theorem 2.1.5, the error depends on the sizes of three terms.

1. The bound of the $(n + 1)$-th derivative, $\max_{x\in[a,b]} |h^{(n+1)}(x)|$. This could proliferate. For example, $h(x) = x^{-1/2}$ on $[1/2, 3/2]$,

$$h^{(n+1)}(x) = \frac{(-1)^{n+1}}{2^{n+1}}(2n + 1)!!x^{-(2n+3)/2}. \tag{2.9}$$

2. The function $\omega(x) = \prod_{j=0}^{n}(x - x_j)$, such a product, could be large if x and the nodes x_j are not so close.

3. The term $\frac{1}{(n+1)!}$, which decays fast.

We can see that for the function $h(x) = x^{-1/2}$ on $[1/2, 3/2]$, it is not trivial to show the interpolating polynomial could converge to h anymore (although it is still true for certain choices of x_j, see Exercise 2.5.3).

Next, we try to provide a better estimate of ω for the special choice: equally spaced nodes. Let the nodes $x_j = a + j\Delta$, where $\Delta = \frac{b-a}{n}$. It is not difficult (prove it) to see $\omega(x)$ will be the worst if x is located on the end sub-intervals, $[x_0, x_1]$ and $[x_{n-1}, x_n]$. Without loss of generality, we assume x is located on $[x_0, x_1]$, then

$$|x - x_j| \leq j\Delta$$

for $j = 2, \ldots, n$, which implies

$$|\omega(x)| \leq \prod_{j=0}^{n} |x - x_j| \leq n!\Delta^{n-1} \sup_{x\in[x_0,x_1]} |(x - x_0)(x - x_1)| = \frac{n!}{4}\frac{(b - a)^{n+1}}{n^{n+1}}, \tag{2.10}$$

Thus interpolation error is bounded by

$$\|h - f\|_\infty \leq \frac{\sup_{x\in[a,b]} |h^{(n+1)}(x)|}{4(n+1)} \frac{(b-a)^{n+1}}{n^{n+1}}.$$

Such an estimate is useful to derive uniform convergence.

Example 2.1.7. *Consider* $h(x) = x^{-1}$ *on* $[1/2, 3/2]$. *Then* $h^{(n+1)}(x) = \frac{(n+1)!(-1)^{n+1}}{x^{n+2}}$, *hence*

$$\frac{|h^{(n+1)}(x)|}{4(n+1)} \frac{(b-a)^{n+1}}{n^{n+1}} \leq \frac{1}{4n^{n+1}} \max_{x\in[1/2,3/2]} \left| \frac{1}{x^{n+2}} \right| = \frac{1}{2} \left(\frac{2}{n} \right)^{n+1}, \tag{2.11}$$

therefore, the interpolation error converges to zero exponentially. It is important to notice that the above method only works for intervals away from the origin.

2.1.3 Runge's Phenomenon

From the above discussion, we can see there is a possibility that $\max_{x\in[a,b]} |h^{n+1}(x)|$ grows much faster than n^{n+2}, which would lead to divergence. Hence, increasing the number of interpolation nodes (at least for equally spaced nodes) is not guaranteed for better approximation. The most famous example is the one made by Carl Runge.

$$h(x) = \frac{1}{1+x^2}, \quad x \in [-5, 5]. \tag{2.12}$$

It can be shown that the interpolation will diverge at around 3.63 as $n \to \infty$ and the maximum

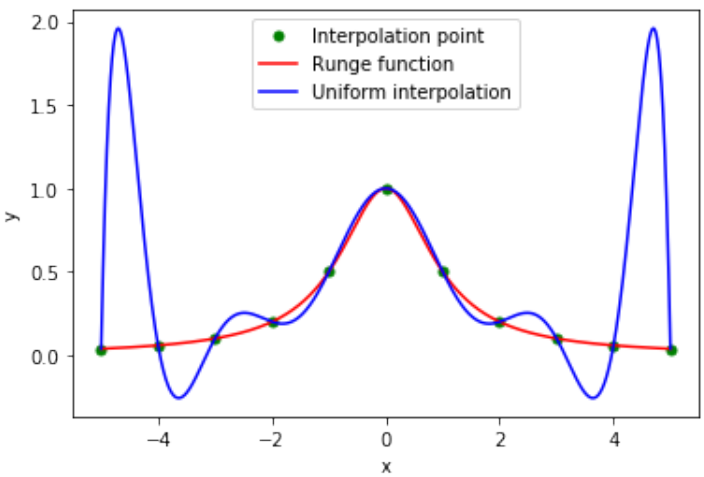

Figure 2.1: Runge's phenomenon. Interpolation with 11 equally spaced nodes.

error $\max_{x\in[-5,5]} |f_n(x) - h(x)|$ grows exponentially, where f_n is the interpolating polynomial with $n+1$ equally spaced nodes. In the next section, we outline the idea of briefly explaining the phenomenon.

2.1.4 Interpolation Remainder Theory

Let f_n be the degree-n polynomial interpolates h at nodes $\{x_j\}_{j=0}^n$. If h is analytic in a domain T (possibly contains holes), then the interpolation (Lagrange interpolant) can be written as

$$f_n(z) = \sum_{j=0}^{n} \frac{\omega(z)h(x_j)}{(z-x_j)\omega'(x_j)} \tag{2.13}$$

Let $\psi(\xi; z) = \frac{(\omega(\xi)-\omega(z))h(\xi)}{(\xi-z)\omega(\xi)}$, then by the Residue theorem for simple poles, if $z \neq x_j$,

$$\frac{1}{2\pi i}\int_{\partial T} \psi(\xi; z)d\xi = \sum_{j=0}^{n} \text{Res}(\psi, x_j) = \sum_{j=0}^{n} \frac{(\omega(x_j)-\omega(z))h(x_j)}{(x_j-z)\omega'(x_j)} = f_n(z), \tag{2.14}$$

which implies that

$$h(z) - f_n(z) = \frac{1}{2\pi i}\int_{\partial T} \frac{\omega(z)h(\xi)}{(\xi-z)\omega(\xi)}d\xi. \tag{2.15}$$

The error analysis focuses on studying the behavior of $|\omega(z)|$ as $n \to \infty$.

Lemma 2.1.8. *If $\{x_j\}_{j=0}^n$ are equispaced nodes over $[a, b]$, then*

$$\lim_{n\to\infty} |\omega(z)|^{\frac{1}{n+1}} = \exp\left(\frac{1}{b-a}\int_a^b \log|z-\xi|d\xi\right). \tag{2.16}$$

Proof. Taking log on $|\omega|^{\frac{1}{n+1}}$, then

$$\log|\omega|^{\frac{1}{n+1}} = \frac{1}{n+1}\sum_{j=0}^{n} \log|z-x_j| \to \frac{1}{b-a}\int_a^b \log|z-\xi|d\xi. \tag{2.17}$$

□

Let $\sigma_n(z) := |\omega(z)|^{\frac{1}{n+1}}$ and define the contour $C_\rho = \{z \in \mathbb{C} \mid \sigma_n(z) = \rho\}$. These level sets are concentric closed curves about the midpoint of $[a, b]$.

Lemma 2.1.9. *Suppose the interpolation nodes $\{x_j\}_{j=0}^n$ are enclosed by C_ρ and h is analytic inside C_ρ. Let $z \in C_{\rho'}$ be such that $\rho' < \rho$, then $f_n \to h$ uniformly as $n \to \infty$.*

Proof. Using the maximum modulus principle, the analytic function $h - f_n$ must attain its maximum modulus at the boundary C_ρ, thus

$$|h(z) - f_n(z)| = \frac{1}{2\pi}\sup_{z\in C_{\rho'}}\left|\int_{C_\rho} \frac{\omega(z)}{\omega(\xi)}\frac{h(\xi)}{(\xi-z)}d\xi\right| \leq C(\rho, \rho')\sup_{\xi\in C_\rho}\frac{|\omega(z)|}{|\omega(\xi)|}, \tag{2.18}$$

where $C(\rho, \rho')$ is a positive constant independent of n. For n sufficiently large, we can find $0 < \delta < \frac{1}{3}(\rho - \rho')$ sufficiently small such that

$$\sup_{\xi\in C_\rho}\frac{|\omega(z)|}{|\omega(\xi)|} \leq \left|\frac{\rho'+\delta}{\rho-\delta}\right|^{n+1} \to 0 \text{ as } n \to \infty. \tag{2.19}$$

□

When h is not analytic inside C_ρ, let us consider a generic situation in which there exist isolated simple poles $z_k \in C_{\rho_k}$, $k \in [m]$ with $\rho_k < \rho$, then we select a contour $C_{\rho'}$ that $\rho_k < \rho' < \rho$ for all k. For $z \in C_{\rho'}$, we have

$$\begin{aligned} h(z) - f_n(z) &= \frac{1}{2\pi i} \int_{C_\rho - \bigcup_{k=1}^m \Gamma_k} \frac{\omega(z)h(\xi)}{(\xi - z)\omega(\xi)} d\xi \\ &= \frac{1}{2\pi i} \int_{C_\rho} \frac{\omega(z)h(\xi)}{(\xi - z)\omega(\xi)} d\xi - \frac{1}{2\pi i} \sum_{k=1}^m \int_{\Gamma_k} \frac{\omega(z)h(\xi)}{(\xi - z)\omega(\xi)} d\xi, \end{aligned} \tag{2.20}$$

where Γ_k is a small path surrounding z_k. The first term can be estimated using the lemma 2.1.9 whose limit goes to zero as $n \to \infty$. The second term is the summation

$$\sum_{k=1}^m \frac{\omega(z)}{\omega(z_k)} \frac{\text{Res}(h, z_k)}{z_k - z}. \tag{2.21}$$

Since for sufficiently large n, we can find $0 < \delta < \min_{k\in[m]} \frac{1}{3}(\rho' - \rho_k)$,

$$\left| \frac{\omega(z)}{\omega(z_k)} \right|^{\frac{1}{n+1}} = \frac{\sigma_n(z)}{\sigma_n(z_k)} > \min_{k\in[m]} \frac{\rho' - \delta}{\rho_k + \delta} > 1. \tag{2.22}$$

Define the unique set $\mathcal{U} = \{u_1, u_2, \cdots, u_l\}$ that $u_1 < u_2 < \cdots < u_l$ which consists of all distinct values from $\{\rho_1, \rho_2, \cdots, \rho_m\}$, then the summation can be decomposed into l groups:

$$\sum_{k=1}^m \frac{\omega(z)}{\omega(z_k)} \frac{\text{Res}(h, z_k)}{z_k - z} = \sum_{s=1}^l \sum_{\substack{k\in[m] \\ \rho_k = u_s}} \frac{\omega(z)}{\omega(z_k)} \frac{\text{Res}(h, z_k)}{z_k - z}. \tag{2.23}$$

As $n \to \infty$, if any of the groups does not vanish, then the whole summation must blow up as $n \to \infty$ (why?). Otherwise, the limiting summation should vanish, which violates the maximum modulus principle.

In Runge's example (2.12), the simple poles $\pm i$ are on the contour C_ρ which intersects the real line at $x_c \approx 3.6334$. Therefore, for $|x| < x_c$, the interpolation f_n uniformly converges to h and diverges once $|x| > x_c$. See Figure 2.2. There exist better choices of interpolation nodes to prevent such a phenomenon. We will discuss this topic in Section 2.1.5.

2.1.5 Chebyshev Interpolation

The Chebyshev interpolation aims to minimize the bound of the interpolation error. The bound of $\omega(x)$ depends only on the choice of the nodes, so a natural question is: What kind of interpolation nodes will *minimize* $\max_{x\in[a,b]} \prod_{j=0}^n |x - x_j|$. We first restrict our analysis to the interval $[a, b] = [-1, 1]$ for simplicity, the general case will be discussed later.

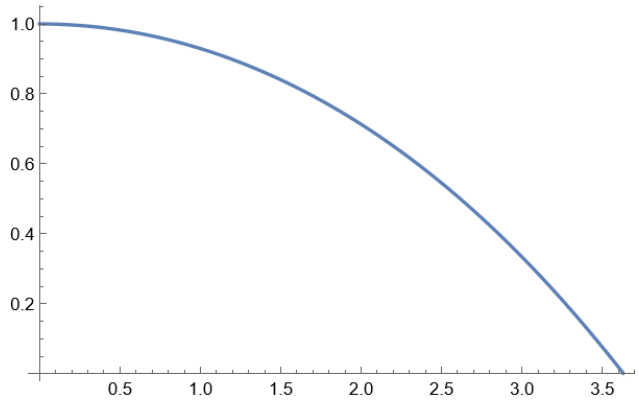

Figure 2.2: Contour C_ρ with $\rho \approx 2.46879$, which passes through the simple pole i.

Example 2.1.10. *When $n = 1, \omega(x) = (x - x_0)(x - x_1)$, this function changes sign over the subintervals $[-1, x_0)$, (x_0, x_1), $(x_1, 1]$, then we can compute the maximum of $|\omega(x)|$ on these subintervals. Therefore, we need to solve*

$$\min_{x_0, x_1 \in [-1,1]} \max((1 + x_0)(1 + x_1), \frac{(x_1 - x_0)^2}{4}, (1 - x_0)(1 - x_1)), \tag{2.24}$$

while we can observe that

$$\frac{1}{2}(1 + x_0)(1 + x_1) + \frac{(x_1 - x_0)^2}{4} + \frac{1}{2}(1 - x_0)(1 - x_1) = 1 + \frac{(x_0 + x_1)^2}{4} \geq 1 \tag{2.25}$$

holds for any choice of x_0, x_1, which means the maximum is at least $1/2$, it occurs when all terms are equal and $x_0 + x_1 = 0$. Hence $x_0 = -\sqrt{2}/2, x_1 = \sqrt{2}/2$.

Definition 2.1.11. *The Chebyshev polynomials of the first kind are defined by:*

$$T_k(x) = \cos(k \arccos x), \quad x \in [-1, 1].$$

Theorem 2.1.12. *The Chebyshev polynomial satisfies the following:*

1. $T_k(\cos\theta) = \cos k\theta, \quad \theta \in [0, \pi]$.
2. $T_0 \equiv 1, T_1(x) = x$ *and*
$$T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x), \quad k \geq 1.$$
3. $\max_{x \in [-1,1]} |T_k(x)| = 1$.
4. *The leading coefficient of $T_k(x)$ is 2^{k-1}.*
5. *T_k has a total of $(k + 1)$ extrema $s_j = \cos(\frac{j\pi}{k}), j = 0, 1, \ldots, n$ in the interval $[-1, 1]$ such that $T_k(s_j) = (-1)^j$.*

Proof. The first three statements are straightforward after replacing the variable $x = \cos\theta$. The fourth statement is an immediate result, with induction through the recursion formula $T_{k+1}(x) = 2xT_k(x) - T_{k-1}$. The last statement is trivial. □

More importantly, the Chebyshev polynomial has the following optimality property.

Theorem 2.1.13. *The optimal choice of interpolation nodes $\{x_j\}_{j=0}^n$ that minimize* $\max|\omega(x)|$ *is the extrema of Chebyshev polynomial T_{n+1}.*

$$\min_{x_j\in[-1,1]} \max_{x\in[-1,1]} |\omega(x)| = \max_{x\in[-1,1]} \frac{1}{2^n}|T_{n+1}(x)| = \frac{1}{2^n} \tag{2.26}$$

Proof. Let the roots of $T_{n+1}(x)$ be $z_0, z_1, \ldots, z_n \in [-1,1]$, then we can write

$$T_{n+1} = 2^n(x-z_0)(x-z_1)\ldots(x-z_n),$$

therefore $\frac{1}{2^n}T_{n+1}(x)$ is a polynomial with leading coefficient as 1. Since $\max_{x\in[-1,1]}|T_{n+1}(x)| = 1$, it is clear that $\max_{x\in[-1,1]}\frac{1}{2^n}|T_{n+1}(x)| = \frac{1}{2^n}$, which is the second equality in (2.26). For the first equality, we try to prove it by contradiction. Let $x_0, x_1, \ldots, x_n \in [-1,1]$, such that

$$\max_{x\in[-1,1]} |\omega(x)| < \frac{1}{2^n},$$

then we define the polynomial $\psi(x) = \frac{1}{2^n}T_{n+1}(x) - \omega(x)$, its degree is at most n due to cancellation, therefore at most have n zeros. On the other side, because $\frac{1}{2^n}T_{n+1}(s_j) = \frac{1}{2^n}(-1)^j$ achieves the extrema at $s_j = \cos(\frac{j\pi}{n+1})$, $j = 0, \ldots, (n+1)$, the polynomial $\psi(s_j)$ must share the same sign of $\frac{1}{2^n}T_{n+1}(s_j)$. This means $\psi(x)$ changes sign $(n+1)$ times, hence $(n+1)$ zeros. It is a contradiction. □

Definition 2.1.14. *The interpolation nodes $z_j = \cos(\frac{(2j+1)\pi}{2(n+1)})$, $j = 0, 1, \ldots, n$ are called "Chebyshev nodes". These nodes are the zeros of Chebyshev polynomial T_{n+1}.*

We can generalize the above theorem to any interval $[a,b]$. One can define the affine transformation ϕ mapping $[-1,1]$ to $[a,b]$ by $\phi(x) = \frac{1}{2}(a+b+(b-a)x)$. It is not difficult to prove the following.

Corollary 2.1.15. *The optimal choice of interpolation nodes that minimize* $\max|\omega(x)|$ *on $[a,b]$ are, $\phi(z_j)$ and*

$$\min_{x_j\in[a,b]} \max_{x\in[a,b]} |\omega(x)| = \frac{(b-a)^{n+1}}{2\cdot 4^n}.$$

This bound is much smaller than the bound for equally spaced nodes.

2.1.6 Stability of Polynomial Interpolation

Suppose there is some perturbation of the data $\tilde{y}_j = y_j + \varepsilon_j$ at the interpolation node x_j. Let $\tilde{f}_n(x)$ and $f_n(x)$ be the interpolating polynomials on perturbed data and original data. Then, with Lagrange polynomials,

$$\begin{aligned} |f_n(x) - \tilde{f}_n(x)| &= |\sum_{j=0}^{n}(y_j - \tilde{y}_j)L_j(x)| \\ &\leq \left(\max_j |\varepsilon_j|\right) \sum_{j=0}^{n} |L_j(x)|. \end{aligned} \tag{2.27}$$

Here $\lambda_n(x) := \sum_{j=0}^{n} |L_j(x)|$ is the *Lebesgue function*. It is a piecewise polynomial. Its maximum Λ_n is the *Lebesgue constant* and only depends on the choice of interpolation nodes. For the equally spaced nodes, this Lebesgue constant grows exponentially. In fact, Turetskii (1940) proved the following sharp result.

Lemma 2.1.16. *Let $\{x_j\}_{j=0}^{n}$ be equispaced nodes on $[0, 1]$, then the Lebesgue constant*

$$\Lambda_n = \frac{2^{n+1}}{en \log n}\left(1 + o(1)\right), \quad n \to \infty. \tag{2.28}$$

Proof. Assume $n \geq 3$, we prove the lower bound by construction. Let $f(x) = e^{in\pi x}$ and define $p_n(x) = \sum_{j=0}^{n} f(x_j)L_j(x)$ as the interpolation polynomial at nodes $x_j = j\Delta$, $\Delta = 1/n$, then

$$|p_n(x)| = \left|\sum_{j=0}^{n}(-1)^j L_j(x)\right| \leq \Lambda_n.$$

Let δ_+ be the forward difference operator

$$\delta_+ p_n(x) := p_n(x + 1/n) - p_n(x), \tag{2.29}$$

then $p_n(x_j) = (1 + \delta_+)^j p_n(0)$ for $j = 0, 1, \cdots, n$, which means the interpolation polynomial is

$$p_n(x) = \sum_{k=0}^{n} \binom{nx}{k} \delta_+^k p_n(0) = \sum_{k=0}^{n} \binom{nx}{k} \delta_+^k f(0), \tag{2.30}$$

which is exactly the Newton form at the equispaced nodes. See Section 2.1.7. Because $\delta_+ f(x) = (e^{i\pi} - 1)f(x) = -2f(x)$ acts as a multiplicative operator, therefore

$$p_n(x) = \sum_{k=0}^{n} \binom{nx}{k} (-2)^k. \tag{2.31}$$

Let $x_* = \frac{1}{n \log n}$ and set $\mu = nx_* \in (0, 1/\log(3))$. Then, for $k \geq 1$, we have

$$\frac{\binom{\mu}{k}(-2)^k}{\binom{\mu}{k+1}(-2)^{k+1}} = \frac{k+1}{2(k-\mu)} > \frac{1}{2}. \tag{2.32}$$

Therefore,

$$\begin{aligned}\Lambda_n \geq |p_n(x_*)| &\geq \binom{\mu}{n}(-2)^n\left(2-\frac{1}{2^n}\right)-1\\ &= \exp\left(\log\mu - \log n + \sum_{k=1}^{n-1}\log\left(1-\frac{\mu}{k}\right)\right)(2^{n+1}-1)-1.\end{aligned} \tag{2.33}$$

Denote the positive constant $C = 2(1 - {}^1\!/\!\log(3))$ and notice $\log(1-x) \geq -x - \frac{x^2}{C}$ for all $x \in (0, {}^1\!/\!\log(3))$, then

$$\begin{aligned}\Lambda_n &\geq \frac{\mu}{n}\exp\left(-\mu\sum_{k=1}^{n-1}\frac{1}{k}\right)\exp\left(-\frac{\mu^2}{C}\sum_{k=1}^{n-1}\frac{1}{k^2}\right)(2^{n+1}-1)-1\\ &\geq \frac{\mu}{n}\exp(-\mu(\log n+\gamma))\exp\left(-\frac{\pi^2\mu^2}{6C}\right)(2^{n+1}-1)-1\\ &= \frac{2^{n+1}}{en\log n}(1+o(1)).\end{aligned} \tag{2.34}$$

We notice that if $\|f\|_\infty \leq 1$, then $\|\delta_+ f\|_\infty \leq 2$, using (2.30), the upper bound of Λ_n can be estimated by

$$\Lambda_n \leq \sup_{x\in(0,1)}\sum_{k=0}^{n}\left|\binom{nx}{k}\right|2^k. \tag{2.35}$$

By symmetry, it is sufficient to consider $x \leq {}^1\!/\!2$, otherwise, a similar bound can be derived with the backward difference operator δ_-. Hence,

$$\begin{aligned}\Lambda_n &\leq \sum_{k=0}^{\lceil n/2\rceil}\binom{\lceil n/2\rceil}{k}2^k + \sup_{z\in(0,n/2)}\sum_{k=\lceil n/2\rceil+1}^{n}\binom{z}{k}2^k\\ &= 3^{\lceil n/2\rceil} + \sum_{k=\lceil n/2\rceil+1}^{n}\frac{2^k}{ek\log k}(1+o(1)),\end{aligned} \tag{2.36}$$

where the following inequalities are applied:

$$\left|\binom{z}{k}\right| \leq \binom{\lceil n/2\rceil}{k}, \quad \forall z,k \leq \lceil n/2\rceil \tag{2.37}$$

and $\forall k \geq \lceil n/2\rceil + 1$,

$$\begin{aligned}\sup_{z\in(0,n/2)}\left|\binom{z}{k}\right| &= \sup_{z\in(0,n/2)}\frac{\sin(\pi(z-l))}{\pi k!}\Gamma(z+1)\Gamma(k-z)\\ &= \sup_{z\in(0,1)}\frac{\sin(\pi(z-l))}{\pi k!}\Gamma(z+1)\Gamma(k-z) \quad \text{(by comparison)}\\ &\leq \sup_{z\in(0,1)}\frac{z}{k}\left(\frac{z+1}{k-z}\right)^z \quad \text{(by Gautschi's inequality)}\\ &= \frac{1}{ek\log k}(1+o(1)).\end{aligned} \tag{2.38}$$

Therefore,

$$\Lambda_n \leq 3^{\lceil n/2 \rceil} + \frac{1}{e \log n} \left(\sum_{k=\lceil n/2 \rceil + 1}^{n} \frac{2^k}{k} \right) (1 + o(1)) = \frac{2^{n+1}}{en \log n}(1 + o(1)). \tag{2.39}$$

□

For the general case, it has been proved by Paul Erdös (1964) that for any set of interpolation nodes,

$$\Lambda_n > \frac{2}{\pi} \log(n+1) + \frac{1}{2}, \quad n \geq 0. \tag{2.40}$$

As the number of nodes $n \to \infty$, $\Lambda_n \to \infty$. This leads to the result of Faber that, for any choice of nodes, there exists a continuous function not able to be approximated by the interpolating polynomial. The Chebyshev nodes are almost optimal, in the sense that

$$\Lambda_{n,\text{Chebyshev}} < \frac{2}{\pi} \log(n+1) + 1. \tag{2.41}$$

The set of nodes that minimizes Λ_n is difficult to compute. A slightly better set of nodes than Chebyshev nodes are the *extended Chebyshev nodes*:

$$\tilde{x}_j = \frac{\cos\left(\frac{2j+1}{2(n+1)\pi}\right)}{\cos\left(\frac{\pi}{2(n+1)}\right)}. \tag{2.42}$$

2.1.7 Newton Form

The Newton form is useful when we dynamically add interpolation nodes. Consider the following scenario: we have already found an interpolation polynomial f_k through the data (x_0, y_0), (x_1, y_1),..., (x_k, y_k), then if an addition pair (x_{k+1}, y_{k+1}) is provided, how can we effectively transform f_k to f_{k+1}? If we write,

$$f_{k+1}(x) = f_k(x) + c_{k+1}(x - x_0)(x - x_1) \dots (x - x_k),$$

then $f_{k+1}(x_j) = f_k(x_j)$, $j = 0, 1, \dots, k$. Therefore, we only need to take care of the equality $f_{k+1}(x_{k+1}) = y_{k+1}$, which means that

$$c_{k+1} = \frac{y_{k+1} - f_k(x_{k+1})}{\prod_{j=0}^{k}(x_{k+1} - x_j)}. \tag{2.43}$$

Such an inductive procedure produces the Newton form:

$$f_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_n(x - x_0) \dots (x - x_{n-1}). \tag{2.44}$$

where the constant c_j depends on $x_0, x_1, \ldots, x_j$ only. Polynomials $\prod_{j=0}^{k}(x-x_j)$ are called *Newton polynomials*. When the coefficients c_k are known, the Newton form (2.44) can be evaluated by the famous "Horner's scheme" (see Exercise 1.5.4), which is

$$f_n(x) = c_0 + (x-x_0)(c_1 + (x-x_1)(c_2 + (x-x_2)(c_3 + \ldots))), \tag{2.45}$$

the evaluation order starts from the innermost part $c_n(x-x_{n-1})$. This formulation has a complexity of $3n$ flops.

Remark 2.1.17. *The computation of c_k is not cheap from (2.43). A naive algorithm with Horner's scheme roughly takes $5n^2/2+\mathcal{O}(n)$ flops to compute all coefficients. The "divided differences" is a better way to compute c_k.*

Definition 2.1.18. *Let the interpolation nodes be $\{x_0, x_1, \ldots, x_n\}$, the "divided differences" are defined recursively as follows (the square bracket is used to distinguish from the usual bracket):*

$$\begin{aligned} f[x_j] &:= f(x_j), \\ f[x_j, \ldots, x_{j+k}] &:= \frac{f[x_{j+1}, \ldots, x_{j+k}] - f[x_j, \ldots, x_{j+k-1}]}{x_{j+k} - x_j}, \end{aligned} \tag{2.46}$$

where $0 \le j, k \le n$ and $j + k \le n$.

The following example graph helps to understand the relationships among the divided differences.

$$\begin{array}{lllll}
f[x_0] & & & & \\
\quad\searrow & & & & \\
f[x_1] \to & f[x_0, x_1] & & & \\
\quad\searrow & \quad\searrow & & & \\
f[x_2] \to & f[x_1, x_2] & \to f[x_0, x_1, x_2] & & \\
\quad\searrow & \quad\searrow & \quad\searrow & & \\
f[x_3] \to & f[x_2, x_3] & \to f[x_1, x_2, x_3] & \to f[x_0, x_1, x_2, x_3] & \\
\quad\searrow & \quad\searrow & \quad\searrow & \quad\searrow & \\
f[x_4] \to & f[x_3, x_4] & \to f[x_2, x_3, x_4] & \to f[x_1, x_2, x_3, x_4] & \to f[x_0, x_1, x_2, x_3, x_4]
\end{array} \tag{2.47}$$

Calculating all the divided differences requires $3n^2/2 + \mathcal{O}(n)$ flops. The following theorem is the main statement for the Newton form.

Theorem 2.1.19. *The interpolation polynomial f_n in Newton form is given by,*

$$f_n(x) = f[x_0] + f[x_0, x_1](x-x_0) + \cdots + f[x_0, \ldots, x_n](x-x_0)(x-x_1)\ldots(x-x_{n-1}). \tag{2.48}$$

In other words, $c_k = f[x_0, \ldots, x_k]$.

Proof. We prove this by induction. Assume that the statement is true for n and the interpolation node and the corresponding values $(x_i, f[x_i])$, $i = 0, 1, \ldots, n$. For a new node and value $(x_{n+1}, f[x_{n+1}])$, it is known from (2.43) that c_{n+1} is the coefficient of the leading power. Let g_n be the interpolation polynomial in Newton form through nodes $(x_i, f[x_i])$, $i = 1, 2, \ldots, n+1$, then

$$\psi(x) := g_n(x)(x - x_0) - f_n(x)(x - x_{n+1})$$

satisfies that $\psi(x_j) = f[x_j](x_{n+1} - x_0)$ for $0 \le j \le n+1$. Therefore,

$$f_{n+1}(x) = \frac{g_n(x)(x - x_0) - f_n(x)(x - x_{n+1})}{x_{n+1} - x_0}. \tag{2.49}$$

The leading power's coefficient is then

$$\frac{f[x_1, \ldots, x_{n+1}] - f[x_0, \ldots, x_n]}{x_{n+1} - x_0} = f[x_0, x_1, \ldots, x_{n+1}]. \tag{2.50}$$

□

Remark 2.1.20. *The divided difference $f[x_j, \ldots, x_{j+k}]$ is the coefficient of leading power of the interpolating polynomial through $(x_j, f[x_j]), \ldots, (x_{j+k}, f[x_{j+k}])$. It can be shown that*

$$f[x_j, \ldots, x_{j+k}] = \frac{1}{k!} f^{(k)}(\xi)$$

for some $\xi \in [a, b]$. See Exercise 2.5.7.

Remark 2.1.21. *The error estimate can be derived by*

$$f(x) - f_n(x) = f[x_0, x_1, \ldots, x_n, x](x - x_0) \ldots (x - x_n). \tag{2.51}$$

Remark 2.1.22. *The Newton form (2.44) does not require distinct nodes. The divided difference can be defined as a limit for repeated nodes:*

$$f[x_0, x_0] = \lim_{x_1 \to x_0} \frac{f[x_1] - f[x_0]}{x_1 - x_0} = f'(x_0). \tag{2.52}$$

Moreover, using the Taylor expansion, $f[\underbrace{x_0, \ldots, x_0}_{(k+1)\text{ times}}] = \frac{1}{k!} f^{(k)}(x_0)$. However, the divided differences cannot be computed in such cases if the derivative values are not provided. We will discuss this scenario later in the Hermite interpolation polynomial; see Section 2.1.8.

Remark 2.1.23. *The algorithm to compute the divided difference can be more efficient with a single column to store diagonal elements. $\rightsquigarrow$ represents the number that is not changing.*

$$
\begin{array}{llllll}
f[x_0] & \rightsquigarrow f[x_0] & \rightsquigarrow f[x_0] & \rightsquigarrow f[x_0] & \rightsquigarrow f[x_0] & \\
 & \searrow & & & & \\
f[x_1] & \rightarrow f[x_0,x_1] & \rightsquigarrow f[x_0,x_1] & \rightsquigarrow f[x_0,x_1] & \rightsquigarrow f[x_0,x_1] & \\
 & \searrow & \searrow & & & \\
f[x_2] & \rightarrow f[x_1,x_2] & \rightarrow f[x_0,x_1,x_2] & \rightsquigarrow f[x_0,x_1,x_2] & \rightsquigarrow f[x_0,x_1,x_2] & \\
 & \searrow & \searrow & \searrow & & \\
f[x_3] & \rightarrow f[x_2,x_3] & \rightarrow f[x_1,x_2,x_3] & \rightarrow f[x_0,x_1,x_2,x_3] & \rightsquigarrow f[x_0,x_1,x_2,x_3] & \\
 & \searrow & \searrow & \searrow & \searrow & \\
f[x_4] & \rightarrow f[x_3,x_4] & \rightarrow f[x_2,x_3,x_4] & \rightarrow f[x_1,x_2,x_3,x_4] & \rightarrow f[x_0,x_1,x_2,x_3,x_4] &
\end{array}
\tag{2.53}
$$

2.1.8 Hermite Polynomial Interpolation

The Lagrange polynomial interpolation only requires the values of the data function h at each node. It can be generalized when the derivative values of h are also available.

Let the tuple $(h(x_j), h^{(1)}(x_j), \ldots, h^{(m_j)}(x_j))$ be the provided derivative values at the interpolation node x_j, $j = 0, \ldots, n$ and $m_j \geq 0$. $N = \sum_{j=0}^{n}(m_j + 1)$ is the total number of constraints. It can be shown that there exists a unique polynomial $H_{N-1} \in \Pi_{N-1}$ satisfies

$$H_{N-1}^{(k)}(x_j) = y_j^k := h^{(k)}(x_j), \quad j = 0, \ldots, n, \quad 0 \leq k \leq m_j.$$

This polynomial is called *Hermite interpolation polynomial*. The idea to construct the Hermite interpolation polynomial borrows from the Lagrange polynomials, which is to find a basis L_{jk} such that

$$\frac{d^p}{dx^p} L_{jk}(x_l) = \begin{cases} 1, & l = j, k = p \\ 0, & \text{otherwise.} \end{cases} \tag{2.54}$$

Once these polynomials are obtained, the Hermite interpolation is straightforward:

$$H_{N-1}(x) = \sum_{j=0}^{n} \sum_{k=0}^{m_j} y_j^k L_{jk}(x).$$

Its uniqueness can be concluded from the linear independence of the basis L_{jk}. However, the construction method in (2.54) is not the simplest. It is known that the Newton form (2.44) works for repeated nodes as long as the diagram's diagonal (2.47) can be filled. Therefore, we can arrange the nodes

$$\underbrace{x_0, \ldots, x_0}_{(m_0+1)\text{ times}}, \quad \underbrace{x_1, \ldots, x_1}_{(m_1+1)\text{ times}}, \quad \ldots, \quad \underbrace{x_n, \ldots, x_n}_{(m_n+1)\text{ times}}$$

In this way, all the necessary divided differences can be computed.

Remark 2.1.24. *The error estimate for Hermite polynomial interpolation will be the same as the Newton form, see Remark 2.1.21.*

2.2 Trigonometric Interpolation

Periodic functions occur in many applications, that is, $f(x+T) = f(x)$, $x \in \mathbb{R}$ for some $T > 0$. For example, a closed planar curve can be parameterized as a periodic function naturally. The polynomial interpolation does not suit periodic functions, this is because polynomials will eventually go to infinity as $x \to \infty$. The most used interpolation for the periodic function is the *trigonometric polynomial interpolation*. In the following, we assume the period $T = 2\pi$ without loss of generality.

2.2.1 Fourier Series

Definition 2.2.1. *For $n \geq 0$, we defined F_n the space of trigonometric polynomials*

$$F_n := \{f(x) \mid f(x) = \frac{a_0}{2} + \sum_{k=1}^{n} a_k \cos kx + \sum_{k=1}^{n} b_k \sin kx,\ a_k, b_k \in \mathbb{R}\}. \tag{2.55}$$

The coefficients $a_0, \ldots, a_n$, $b_1, \ldots, b_n$ can be also chosen as complex numbers. $f \in F_n$ is said to be of degree n if $|a_n| + |b_n| > 0$.

The concept of "degree" here can be validated by the addition theorem of trigonometric functions. For instance, if $f_1 \in F_k$, $f_2 \in F_l$, then $f_1 f_2 \in F_{k+l}$. In the next, we discuss the uniqueness of the interpolation with the trigonometric polynomial.

Lemma 2.2.2. *A trigonometric polynomial $f \in F_n$ that has more than $2n$ zeros in $[0, 2\pi)$ must vanish identically.*

Proof. Rewrite the trigonometric function in the form of

$$f_n(x) = \sum_{k=-n}^{n} \gamma_k e^{ikx}. \tag{2.56}$$

where $\gamma_0 = \frac{1}{2}a_0$ and $\gamma_k = \frac{1}{2}(a_k - ib_k)$ and $\gamma_{-k} = \frac{1}{2}(a_k + ib_k)$, $k = 1, \ldots, n$. Then substitute $z = e^{ix}$ and set

$$p(z) = \sum_{k=-n}^{n} \gamma_k z^{n+k}, \tag{2.57}$$

one can rewrite $f_n(x) = z^{-n}p(z)$. If $f_n(x)$ has more than $2n$ zeros, then $p(z)$ has more than $2n$ zeros, which is a contradiction since $p(z)$ is a polynomial of degree $2n$. □

Remark 2.2.3. *Since $\sin nx \in F_n$ has $2n$ zeros $\frac{\pi j}{n}$, $j = 0, \ldots, 2n-1$, it means to uniquely determine a trigonometric polynomial in F_n, exactly $2n+1$ values are needed. This is also known as the "Nyquist-Shanon sampling theorem".*

A direct corollary is the linear independence of the functions 1, $\cos kx$ and $\sin kx$, $k = 1, \dots n$, these $(2n+1)$ functions form a natural basis for the trigonometric polynomial space F_n.

Corollary 2.2.4. *The functions* $1, \cos kx, \sin kx$, $k = 1, \dots, n$ *are linearly independent on* $C[0, 2\pi]$, *hence* F_n *is a* $(2n+1)$ *dimensional space.*

To determine the coefficients a_k, b_k from $(2n+1)$ data pairs (x_j, y_j), $j = 0, \dots, 2n$. We follow the idea of Lagrange polynomials by creating the basis polynomial $l_k(x)$ such that

$$l_k(x_j) = \begin{cases} 1, & j = k, \\ 0, & \text{otherwise}. \end{cases} \tag{2.58}$$

Remark 2.2.5. *A natural idea is replacing* $x - x_j$ *in the Lagrange basis by* $\sin(x - x_j)$ *and produce something like*

$$\prod_{j=0, j\neq k}^{2n} \frac{\sin(x - x_j)}{\sin(x_k - x_j)}$$

but $\sin(x - x_j)$ *has two roots on* $[0, 2\pi)$, *therefore we need to rescale it to* $[0, \pi)$.

Theorem 2.2.6. *Let the basis trigonometric polynomial,*

$$l_k(x) = \prod_{j=0, j\neq k}^{2n} \frac{\sin(\frac{x - x_j}{2})}{\sin(\frac{x_k - x_j}{2})},$$

then the interpolation trigonometric polynomial is

$$f_n(x) = \sum_{k=0}^{2n} y_k l_k(x). \tag{2.59}$$

Proof. It remains to show $l_k \in F_n$. This can be seen by splitting l_k into n pairs, each pair takes the form of

$$\sin(\frac{x - x_0}{2}) \sin(\frac{x - x_1}{2}) = \frac{1}{2} \cos\left(\frac{x_0 - x_1}{2}\right) - \frac{1}{2} \cos\left(\frac{2x - x_0 - x_1}{2}\right) \in F_1. \tag{2.60}$$

□

Computationally, we can reuse the previously known barycentric form, but there exist better methods. For simplicity, we consider the equal space nodes in the following (non-uniform nodes could achieve the same complexity though).

$$x_j = \frac{2\pi j}{2n+1}, \quad j = 0, \dots, 2n. \tag{2.61}$$

We will try to locate the coefficients γ_k in the complex form (see (2.56)) from the interpolation conditions.

$$f_n(x_j) = y_j = \sum_{k=-n}^{n} \gamma_k e^{ikx_j}. \tag{2.62}$$

Use the property of the functions e^{ikx_j} that

$$\sum_{k=0}^{2n} e^{ikx_j} = \begin{cases} 2n+1, & k=0 \\ 0, & \text{otherwise}. \end{cases} \tag{2.63}$$

It is not difficult to derive

$$\sum_{j=0}^{2n} y_j e^{-imx_j} = \sum_{k=-n}^{n} \gamma_k \sum_{j=0}^{2n} e^{i(k-m)x_j} = \gamma_m(2n+1). \tag{2.64}$$

Therefore, we can compute

$$\gamma_m = \frac{1}{2n+1} \sum_{j=0}^{2n} y_j e^{-imx_j}. \tag{2.65}$$

When the coefficients γ_m are known, Horner's scheme can be employed to evaluate the trigonometric polynomial in $\mathcal{O}(n)$ time complexity. However, naive computing of all the coefficients γ_k will cost $\mathcal{O}(n^2)$ flops. The fast Fourier transform can reduce the time complexity to $\mathcal{O}(n \log n)$.

Remark 2.2.7. *For an even number of equally spaced nodes $x_j = \frac{j\pi}{n}$, $0 \le j \le 2n-1$, the basis $\sin(nx)$ equals to zero constantly. There are $2n$ coefficients to be determined only. We can derive a similar formula as (2.65) by replacing $(2n+1)$ with $2n$.*

2.2.2 Fast Fourier Transform

The discrete Fourier transform `DFT` of a vector $\mathbf{a} = (a_0, \ldots, a_{n-1})$ is to evaluate the following vector:

$$\mathtt{DFT}(\mathbf{a})_k := \frac{1}{n} \sum_{j=0}^{n-1} a_j e^{-2\pi ijk/n}, \quad k = 0, \ldots, n-1.$$

This is the exact formula to compute the coefficients for the trigonometric interpolation polynomial. Such transform is most efficiently calculated through the fast Fourier transform (`fft`). The fast Fourier transform exploits the symmetry in $e^{2\pi ij/n}$ when n is the power of two using divide-and-conquer. Let $\omega = e^{-2\pi i/n}$ and define c_k as

$$c_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{jk}, \quad k = 0, \ldots, n-1. \tag{2.66}$$

Let $m = {}^{n}/_{2} \in \mathbb{N}$, then $\omega^n = 1$, $\omega^m = -1$. We can separate c_k into two parts with even j and odd j.

$$c_k = \frac{1}{2} A_k + \frac{1}{2} B_k \omega^k, \quad c_{k+m} = \frac{1}{2} A_k - \frac{1}{2} B_k \omega^k \tag{2.67}$$

where

$$A_k = \frac{1}{m}\sum_{j=0}^{m-1} y_{2j}(\omega^2)^{jk}, \quad B_k = \frac{1}{m}\sum_{j=0}^{m-1} y_{2j+1}(\omega^2)^{jk}, \tag{2.68}$$

both A_k and B_k are in the same form and similar to c_k, but with only half of the terms in summation. This implies a recursive algorithm. Suppose A_k and B_k, $0 \le k \le m-1$ can be computed with $f(m)$ operations each, then

$$f(n) = f(2m) = 2f(m) + 4m \tag{2.69}$$

The second term includes $2m$ multiplications and $2m$ additions in (2.67). Therefore,

$$\begin{aligned} f(n) &= 2f(\frac{n}{2}) + 2n \\ &= 4f(\frac{n}{4}) + 2n + 2n \\ &= \dots \\ &= nf(1) + \underbrace{2n + \cdots + 2n}_{\log_2 n \text{ times}} = 2n\log_2 n. \end{aligned} \tag{2.70}$$

since $f(1) = 0$, no computation is needed in this case. The `fft` is usually a standard routine in modern scientific computing software.

2.2.3 Interpolation Error of Trigonometric Polynomial

The L^2 error estimate will be discussed at a later point. This part only focuses on the L^∞ error estimate. We will need the following lemma, the proof is left as an exercise (see Exercise 2.5.9).

Lemma 2.2.8. *Let $g(x)$ be the truncated Fourier series of $f(x)$*

$$g(x) = \sum_{s=-n}^{n} \gamma_s e^{isx}, \quad \gamma_s = \frac{1}{2\pi}\int_0^{2\pi} f(x)e^{-isx}dx$$

then

$$g(x) = \frac{1}{2\pi}\int_0^{2\pi} D_n(x-y)f(y)dy, \tag{2.71}$$

where $D_n(x)$ is the Dirichlet kernel

$$D_n(x) := \sum_{s=-n}^{n} e^{isx} = \frac{\sin((2n+1)x/2)}{\sin(x/2)}. \tag{2.72}$$

Theorem 2.2.9 (Uniform convergence for Hölder continuous functions)**.** *If $f \in C^{0,\alpha}(\mathbb{R})$ is a 2π-period function, then the trigonometric interpolation polynomial with $2n+1$ equally spaced nodes converges uniformly as the number of nodes tends to infinity.*

Proof. Since $f \in C^{0,\alpha}(\mathbb{R})$, there exists a constant $K > 0$ that $|f(x) - f(y)| \leq K|x-y|^\alpha$. Let $g(x)$ be the truncated Fourier series

$$g(x) = \sum_{s=-n}^{n} \gamma_s e^{isx}, \quad \gamma_s = \frac{1}{2\pi}\int_0^{2\pi} f(x)e^{-isx}dx$$

and denote $h(x) = f(x) - g(x) = \sum_{|s|>n} \gamma_s e^{isx}$ the reminder. The interpolation polynomial satisfies

$$\begin{aligned} f_n(x) &= \sum_{m=-n}^{n}\left(\frac{1}{2n+1}\sum_{j=0}^{2n} y_j e^{-imx_j}\right)e^{imx} \\ &= \sum_{m=-n}^{n}\left(\frac{1}{2n+1}\sum_{j=0}^{2n}(g(x_j)+h(x_j))e^{-imx_j}\right)e^{imx} \\ &= \sum_{m=-n}^{n}\left(\frac{1}{2n+1}\sum_{j=0}^{2n}(\sum_{s=-n}^{n}\gamma_s e^{isx_j}+h(x_j))e^{-imx_j}\right)e^{imx} \\ &= g(x) + \sum_{m=-n}^{n}\left(\frac{1}{2n+1}\sum_{j=0}^{2n} h(x_j)e^{-imx_j}\right)e^{imx} \\ &= g(x) + \frac{1}{2n+1}\sum_{j=0}^{2n} h(x_j)\frac{\sin((2n+1)(x-x_j)/2)}{\sin((x-x_j)/2)} \end{aligned} \tag{2.73}$$

Therefore,

$$|f - f_n| \leq \|h\|_\infty \left(1 + \frac{1}{2n+1}\sum_{j=0}^{2n}\left|\frac{\sin((2n+1)(x-x_j)/2)}{\sin((x-x_j)/2)}\right|\right). \tag{2.74}$$

We first estimate

$$\sum_{j=0}^{2n}\left|\frac{\sin((2n+1)(x-x_j)/2)}{\sin((x-x_j)/2)}\right|$$

Separate the nodes into two groups: The first group with $|x - x_j| < \frac{2\pi}{2n+1}$, the absolute value is bounded by $(2n+1)$, there are at most 3 nodes lying in this region, thus the contribution of the first group is at most $\mathcal{O}(n)$. The second group is $\pi \geq |x - x_j| \geq \frac{2\pi}{2n+1}$, and then one can estimate

$$\left|\frac{\sin((2n+1)(x-x_j)/2)}{\sin((x-x_j)/2)}\right| \leq \frac{\pi}{|x-x_j|}, \tag{2.75}$$

since $\pi \sin x \geq 2x$ for $0 \leq x \leq \pi/2$. This implies the contribution from the second group is $\mathcal{O}(n\log n)$, and the total contribution is bounded by $\mathcal{O}(n\log n)$. Then (2.74) can be bounded by

$$|f - f_n| \leq C\|h\|_\infty \log n, \tag{2.76}$$

where C is an absolute constant. Next, we estimate $\|h\|_\infty$. Using Lemma 2.2.8, we can represent

$$\begin{aligned} h(x) &= \frac{1}{2\pi}\int_{-\pi}^{\pi} D_n(y)(f(x)-f(x-y))dy \\ &= \frac{1}{2\pi}\sum_{k=-n}^{n}\int_{\frac{(2k-1)\pi}{2n+1}}^{\frac{(2k+1)\pi}{2n+1}} \frac{\sin((2n+1)y/2)}{\sin(y/2)}(f(x)-f(x-y))dy. \end{aligned} \tag{2.77}$$

Notice the cancellation property

$$\int_{\frac{(2k-1)\pi}{2n+1}}^{\frac{(2k+1)\pi}{2n+1}} \sin\left(\frac{(2n+1)y}{2}\right)dy = 0, \tag{2.78}$$

then for any $y' \in \mathcal{I}_k := \left(\frac{(2k-1)\pi}{2n+1}, \frac{(2k+1)\pi}{2n+1}\right)$, we have

$$\begin{aligned} &\left|\int_{\frac{(2k-1)\pi}{2n+1}}^{\frac{(2k+1)\pi}{2n+1}} \frac{\sin((2n+1)y/2)}{\sin(y/2)}(f(x)-f(x-y))dy\right| \\ &= \left|\int_{\frac{(2k-1)\pi}{2n+1}}^{\frac{(2k+1)\pi}{2n+1}} \sin((2n+1)y/2)\left(\frac{(f(x)-f(x-y))}{\sin(y/2)} - \frac{(f(x)-f(x-y'))}{\sin(y'/2)}\right)dy\right| \\ &\le \frac{4}{2n+1}\sup_{y,y'\in\mathcal{I}_k}\left|\frac{(f(x)-f(x-y))}{\sin(y/2)} - \frac{(f(x)-f(x-y'))}{\sin(y'/2)}\right| \\ &\le \frac{4K}{2n+1}\left(\frac{\frac{\pi}{2n+1}\left|\frac{(2|k|+1)\pi}{2n+1}\right|^\alpha + \frac{(2|k|+1)\pi}{4n+2}\left|\frac{2\pi}{2n+1}\right|^\alpha}{\left|\frac{(2|k|-1)\pi}{4n+2}\right|^2}\right) \\ &= \frac{8K\pi^{\alpha-1}}{(2n+1)^\alpha}\left(\frac{2(2|k|+1)^\alpha}{(2|k|-1)^2} + 2^\alpha\frac{2|k|+1}{(2|k|-1)^2}\right), \end{aligned} \tag{2.79}$$

Because $\left|\frac{2|k|-1}{2|k|+1}\right| \ge \frac{1}{3}$ for all $k \in \mathbb{N}$, we obtain the bound for h:

$$\begin{aligned} |h(x)| &\le \frac{4K\pi^{\alpha-2}}{(2n+1)^\alpha}\sum_{k=-n}^{n}\left(\frac{2(2|k|+1)^\alpha}{(2|k|-1)^2} + 2^\alpha\frac{2|k|+1}{(2|k|-1)^2}\right) \\ &\le \frac{36K\pi^{\alpha-2}}{(2n+1)^\alpha}\sum_{k=-n}^{n}\left(\frac{2}{(2|k|+1)^{2-\alpha}} + \frac{2^\alpha}{(2|k|+1)}\right) \\ &= \mathcal{O}(n^{-\alpha}\log n). \end{aligned} \tag{2.80}$$

Therefore $|f - f_n| = \mathcal{O}(n^{-\alpha}|\log n|^2)$. □

Remark 2.2.10. *The uniform convergence of trigonometric polynomial interpolation be further extended to general continuous functions with* modulus of continuity $\omega(f;\tau)$ *satisfying*

$$\omega(f;\frac{1}{n})|\log n|^2 \to 0$$

as $n \to \infty$. The estimate of $\|h\|_\infty$ matches the work of Dunham Jackson (1913). More related topics are discussed in Chapter 4.

2.3 Spline Interpolation

It has been seen that increasing the number of interpolation nodes will not always help improve the approximation. The spline interpolation is to conquer this issue by using the piecewise low-degree polynomials.

Definition 2.3.1. *Let $x_0, \ldots, x_n$ be the distinct nodes on $[a, b]$ such that $a = x_0 < \cdots < x_n = b$. The piecewise defined function $s_k(x)$ on the interval $[a, b]$ is a spline of degree k to the nodes if*

$$s_k|_{[x_j, x_{j+1}]} \in \Pi_k, \quad s_k \in C^{k-1}([a, b]). \tag{2.81}$$

The spline function s_k is $(k-1)$-times continuously differentiable and piecewise polynomial of degree k.

Then the space of splines s_k will be $(n + k)$ dimension: each interval has $(k + 1)$ dimensions, each interface imposes k constraints, therefore $n(k + 1) - (n - 1)k = n + k$ dimensions. This shows that to determine a spline on the nodes uniquely, we will require $n + 1$ interpolation values and $k - 1$ additional constraints. Usual choices are

1. periodic splines. $s_k^{(m)}(a) = s_k^{(m)}(b)$ for $m = 0, 1, \ldots, k - 1$.
2. natural splines. $s_k^{(l+j)}(a) = s_k^{(l+j)}(b) = 0$, $j = 0, 1, \ldots, l - 2$ and $k = 2l - 1$ with $l \geq 2$.

In the following, we discuss some useful examples of spline.

2.3.1 Linear Splines

The linear splines are a special case of splines. It uses piecewise linear polynomials on each subinterval and does not impose any derivative continuity. Let y_j be the interpolation values at nodes x_j, respectively. The interpolation has an explicit form:

$$s_1(x) = y_{j-1} + \frac{x - x_{j-1}}{x_j - x_{j-1}} y_j \tag{2.82}$$

on the interval $[x_{j-1}, x_j]$. It can be represented as a linear combination of the "hat" basis function $\theta_j(x)$, defined by

$$\theta_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & x \in [x_{j-1}, x_j], \quad 1 \leq j \leq n \\ \frac{x - x_{j+1}}{x_j - x_{j+1}}, & x \in [x_j, x_{j+1}], \quad 0 \leq j \leq n - 1 \\ 0, & \text{otherwise} \end{cases} \tag{2.83}$$

then $s_1(x)$ can be written as

$$s_1(x) = \sum_{j=0}^{n} \theta_j(x) y_j.$$

The interpolation error can be derived directly from the previous theory for two interpolation nodes. Let $f \in C^2([a,b])$, then on $[x_{j-1}, x_j]$, the interpolation error is

$$\frac{1}{2!} f''(\xi)(x - x_{j-1})(x - x_j) \leq \frac{1}{8} \|f''\|_\infty |x_j - x_{j-1}|^2. \tag{2.84}$$

Therefore, the interpolation error on $[a,b]$ is $\frac{1}{8}\|f''\|_\infty h^2$, where $h = \max |x_j - x_{j-1}|$. Once f'' is not uniformly bounded or even f' is not well-defined somewhere, e.g., $f \in C^{0,\alpha}[a,b]$, the interpolation error will be replaced by the modulus of continuity. On $x \in [x_j, x_{j+1}]$, we have

$$\begin{aligned} |s_1(x) - f(x)| &= \left| \frac{(x_{j+1} - x) y_j + (x - x_j) y_{j+1}}{x_{j+1} - x_j} - f(x) \right| \\ &= \left| \frac{(x_{j+1} - x)(y_j - f(x)) + (x - x_j)(y_{j+1} - f(x))}{x_{j+1} - x_j} \right| \\ &\leq \omega(f; |x_{j+1} - x_j|), \end{aligned} \tag{2.85}$$

where $\omega(f;\tau)$ is the modulus of continuity of f.

2.3.2 Cubic Splines

The cubic splines are particularly important in practice. Let $a = x_0 < x_1 < \cdots < x_n = b$, and the corresponding values are y_j, $j = 0, \ldots, n$. The constraints for cubic splines are: piecewise polynomial of degree 3 and continuous second derivative.a Denote the interpolation spline as s_3, then $s_3{}''$ is a piecewise linear function. On the sub-interval $[x_{j-1}, x_j]$, it can be represented by

$$s_3{}''(x) = M_{j-1} \frac{x_j - x}{h_j} + M_j \frac{x - x_{j-1}}{h_j}, \quad j = 1, \ldots, n, \tag{2.86}$$

where $h_j = x_j - x_{j-1}$, $M_j = s_3{}''(x_j)$. Integrating the above formula twice,

$$s_3(x) = M_{j-1} \frac{(x_j - x)^3}{6h_j} + M_j \frac{(x - x_{j-1})^3}{6h_j} + A_j(x - x_{j-1}) + B_j \tag{2.87}$$

The additional constants A_j, B_j can be determined by imposing $f(x_{j-1}) = y_{j-1}$ and $f(x_j) = y_j$. That is

$$A_j = \frac{y_j - y_{j-1}}{h_j} - \frac{h_j}{6}(M_j - M_{j-1}), \quad B_j = y_{j-1} - M_{j-1} \frac{h_j^2}{6}. \tag{2.88}$$

Now we will determine the constants M_j using the first derivative's continuity.

$$s_3'(x_j^-) = s_3'(x_j^+), \quad j = 1, \ldots, n-1. \tag{2.89}$$

That is equivalent to $j = 1, \ldots, n-1$,

$$\begin{aligned} s_3'(x_j^-) &= M_j \frac{h_j}{3} + M_{j-1}\frac{h_j}{6} + \frac{y_j - y_{j-1}}{h_j} \\ &= -M_j \frac{h_{j+1}}{3} - M_{j+1}\frac{h_{j+1}}{6} + \frac{y_{j+1} - y_j}{h_{j+1}} = s_3'(x_j^+). \end{aligned} \tag{2.90}$$

We can write the corresponding equations into a tridiagonal linear system

$$\begin{pmatrix} \frac{h_1}{6} & \frac{h_1+h_2}{3} & \frac{h_2}{6} & & & \\ & \frac{h_2}{6} & \frac{h_2+h_3}{3} & \frac{h_3}{6} & & \\ & & \ddots & \ddots & \ddots & \\ & & & \frac{h_{n-1}}{6} & \frac{h_{n-1}+h_n}{3} & \frac{h_n}{6} \end{pmatrix} \begin{pmatrix} M_0 \\ M_1 \\ \vdots \\ M_n \end{pmatrix} = \begin{pmatrix} \frac{y_2-y_1}{h_2} - \frac{y_1-y_0}{h_1} \\ \frac{y_3-y_2}{h_3} - \frac{y_2-y_1}{h_2} \\ \vdots \\ \frac{y_n-y_{n-1}}{h_n} - \frac{y_{n-1}-y_{n-2}}{h_{n-1}} \end{pmatrix} \tag{2.91}$$

In practice, the system will be rescaled for numerical stability.

$$\begin{pmatrix} \frac{h_1}{2(h_1+h_2)} & 1 & \frac{h_2}{2(h_1+h_2)} & & & \\ & \frac{h_2}{2(h_2+h_3)} & 1 & \frac{h_3}{2(h_2+h_3)} & & \\ & & \ddots & \ddots & \ddots & \\ & & & \frac{h_{n-1}}{2(h_{n-1}+h_n)} & 1 & \frac{h_n}{2(h_{n-1}+h_n)} \end{pmatrix} \begin{pmatrix} M_0 \\ M_1 \\ \vdots \\ M_n \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \end{pmatrix}$$

where $d_j = \frac{3}{h_{j-1}+h_j}\left[\frac{y_j-y_{j-1}}{h_j} - \frac{y_{j-1}-y_{j-2}}{h_{j-1}}\right]$. The above system still lacks 2 more constraints, since the matrix is of size $(n-1) \times (n+1)$. Then we can apply the periodic spline or natural spline conditions. For example, if the natural constraint is applied: $s_3''(a) = s_3''(b) = 0$. We should have two more equations:

$$M_0 = M_n = 0. \tag{2.92}$$

Then we can simply ignore the first and last columns of the matrix (also M_0 and M_n). If the periodic constraint is imposed, then we can add two more constraints: $M_0 = M_n$ and

$$-M_0\frac{h_1}{3} - M_1\frac{h_1}{6} + \frac{y_1 - y_0}{h_1} = M_n\frac{h_n}{3} + M_{n-1}\frac{h_n}{6} + \frac{y_n - y_{n-1}}{h_n}.$$

In both cases, the resulting linear system is still tridiagonal and the solution takes $\mathcal{O}(n)$ time complexity with the Thomas algorithm.

Another popular choice to complete the matrix is to impose the constraints in the similar form on x_0 and x_n:

$$2M_0 + \frac{h_1}{h_0 + h_1}M_1 = d_0, \quad \frac{h_n}{h_n + h_{n+1}}M_{n-1} + 2M_n = d_n, \tag{2.93}$$

where $h_0 = h_{n+1} = 0$ and $d_0 = d_1$, $d_n = d_{n-1}$ are assumed.

Remark 2.3.2. *The Jacobi and Gauss-Seidel iterations are suitable for solving the system. The iteration will converge to system accuracy within $\mathcal{O}(\log_2 \mathrm{u})$ iterations. Related topics are discussed in Chapter 8.*

The error estimate for the cubic spline can be derived in a way similar to the Lagrange polynomial interpolation. The following result is attributed to Charles Hall (1968).

Theorem 2.3.3. *Let $f \in C^4([a,b])$ and $a = x_0 < \cdots < x_n = b$ be a set of nodes. Then the natural cubic spline s_3 interpolating f satisfies*

$$\|f - s_3\|_\infty \leq \frac{5}{384}\|f^{(4)}\|_\infty h^4, \tag{2.94}$$

where $h = \max_j |x_j - x_{j-1}|$.

Proof. Here we only state the rough idea to prove the error bound. Let $u(x)$ be the piecewise Hermite interpolation polynomial that

$$u(x_j) = f(x_j), \quad u'(x_j) = f'(x_j), \tag{2.95}$$

then one can estimate

$$\max_{x\in[x_j,x_{j+1}]} |u - f| \leq \frac{1}{24}\|f^{(4)}\|_\infty (x - x_j)^2 (x - x_{j+1})^2 \leq \frac{1}{384}\|f^{(4)}\|_\infty h^4. \tag{2.96}$$

On the subinterval $[x_i, x_{i+1}]$, s_3 and u are both cubic polynomial interpolations, thus

$$u(x) - s_3(x) = \frac{(x - x_i)(x_{i+1} - x)}{(x_{i+1} - x_i)} \left(e'(x_i)\frac{x_{i+1} - x}{x_{i+1} - x_i} - e'(x_{i+1})\frac{x - x_i}{x_{i+1} - x_i} \right), \tag{2.97}$$

where $e(x) = f(x) - s_3(x)$. Therefore,

$$\begin{aligned} \|f - s_3\|_\infty &\leq \|u - s_3\|_\infty + \|f - u\|_\infty \\ &\leq \frac{h}{4}\max_{0\leq i\leq n} |e'(x_i)| + \frac{1}{384}\|f^{(4)}\|_\infty h^4. \end{aligned} \tag{2.98}$$

Using (2.90), we find that

$$\frac{1}{h_j}\left[2s_3'(x_j) + s_3'(x_{j-1})\right] + \frac{1}{h_{j+1}}\left[2s_3'(x_j) + s_3'(x_{j+1})\right] = \frac{3(y_j - y_{j-1})}{h_j^2} + \frac{3(y_{j+1} - y_j)}{h_{j+1}^2}, \tag{2.99}$$

Using Taylor expansion locally at x_j, there exist $\zeta \in (x_{j-1}, x_j)$ and $\xi \in (x_j, x_{j+1})$ that

$$\frac{2e'(x_j) + e'(x_{j-1})}{h_j} + \frac{2e'(x_j) + e'(x_{j+1})}{h_{j+1}} = \frac{1}{24}\left[-h_j^2 f^{(4)}(\zeta) + h_{j+1}^2 f^{(4)}(\xi)\right]. \tag{2.100}$$

Suppose $\max_{0\leq i\leq n} |e'(x_i)|$ attains its maximum at node x_k, then

$$\left| \frac{2e'(x_k) + e'(x_{k-1})}{h_j} + \frac{2e'(x_k) + e'(x_{k+1})}{h_{k+1}} \right| \geq \frac{h_j + h_{j+1}}{h_j h_{j+1}} |e'(x_k)|. \tag{2.101}$$

Therefore, by AM-GM inequality,

$$\max_{0\leq i\leq n} |e'(x_i)| \leq \frac{h_j h_{j+1}}{24(h_j + h_{j+1})}(h_j^2 + h_{j+1}^2)\|f^{(4)}\|_\infty \leq \frac{1}{24}h^3\|f^{(4)}\|_\infty. \tag{2.102}$$

Finally, combined with the estimate (2.98) will arrive at the desired bound. □

2.3.3 B-Spline Representation

The definition of B-spline (short for basis spline) was initially proposed by Curry and Schoenberg (1947). Let $\mathbf{t} := \{t_j\}_{j\in\mathcal{J}}$ be a nondecreasing sequence; the index set $\mathcal{J}$ consists of consecutive integers which may be finite or infinite. The sequence is called "knots". The B-splines are defined in the following recurrence relation.

Definition 2.3.4 (Cox–de Boor). *The B-splines of order one $B_{j,1}$ consist of the characteristic functions of the knot intervals:*

$$B_{j,1}(x) = \begin{cases} 1, & x \in [t_j, t_{j+1}) \\ 0, & \textit{otherwise} \end{cases}$$

and for $k > 1$, the B-spline of order k satisfies

$$B_{j,k}(x) = \frac{x - t_j}{t_{j+k-1} - t_j} B_{j,k-1}(x) + \frac{t_{j+k} - x}{t_{j+k} - t_{j+1}} B_{j+1,k-1}(x).$$

Using induction, we can show the following property of the B-spline (see Exercise 2.5.13).

Lemma 2.3.5 (Property I). *Each spline $B_{j,k}$ is a non-negative piecewise polynomial of degree $k-1$ supported on (t_j, t_{j+k}).*

Lemma 2.3.6 (Property II). *For any order $k \geq 1$, the B-splines of order k form a partition of unity:*

$$\sum_{j=1}^{n} B_{j,k}(x) = 1, \quad x \in (t_k, t_{n+1}).$$

Proof. Let us prove this by induction. For $k = 1$, this is exactly the definition. Assume the conclusion for $k = l$, then for $x \in (t_{l+1}, t_{n+1}) \subset (t_l, t_{n+1})$, since $(t_{l+1}, t_{n+1}) \bigcap \operatorname{supp} B_{1,l} = \emptyset$ and $(t_{l+1}, t_{n+1}) \bigcap \operatorname{supp} B_{n+1,l} = \emptyset$, we have

$$\begin{aligned} \sum_{j=1}^{n} B_{j,l+1}(x) &= \sum_{j=1}^{n} \frac{x - t_j}{t_{j+l} - t_j} B_{j,l}(x) + \sum_{j=1}^{n} \frac{t_{j+l+1} - x}{t_{j+l+1} - t_{j+1}} B_{j+1,l}(x) \\ &= \sum_{j=1}^{n} \frac{x - t_j}{t_{j+l} - t_j} B_{j,l}(x) + \sum_{j=2}^{n+1} \frac{t_{j+l} - x}{t_{j+l} - t_j} B_{j,l}(x) \\ &= \sum_{j=1}^{n} B_{j,l}(x) - \frac{t_{l+1} - x}{t_{l+1} - t_1} B_{1,l}(x) + \frac{t_{n+l+1} - x}{t_{n+l+1} - t_{n+1}} B_{n+1,l}(x) \\ &= 1. \end{aligned} \tag{2.103}$$

As an extension of Lemma 2.3.6, we actually can show B-spline can represent any polynomials.

Lemma 2.3.7 (Marsden). *For $k \geq 1$, the B-splines of order k can represent the polynomial*

$$(x - \mu)^{k-1} = \sum_{j=1}^{n} \phi_{jk}(\mu) B_{jk}(x), \quad x \in (t_k, t_{+1}), \tag{2.104}$$

where $\phi_{jk}(\mu) = \prod_{l=j+1}^{j+k-1}(t_l - \mu)$ if $k > 1$, otherwise $\phi_{jk}(\mu) = 1$.

Proof. The equality can be proved by induction as well. For the induction step, by the recursive definition, we have

$$\begin{aligned}
\sum_{j=1}^{n} \phi_{j,l+1}(\mu) B_{j,l+1}(x) &= \sum_{j=1}^{n} \frac{x-t_j}{t_{j+l}-t_j} \phi_{j,l+1}(\mu) B_{j,l}(x) + \sum_{j=1}^{n} \frac{t_{j+l+1}-x}{t_{j+l+1}-t_{j+1}} \phi_{j,l+1}(\mu) B_{j+1,l}(x) \\
&= \sum_{j=1}^{n} \frac{x-t_j}{t_{j+l}-t_j} (t_{j+l}-\mu) \phi_{j,l}(\mu) B_{j,l}(x) \\
&\quad + \sum_{j=2}^{n+1} \frac{t_{j+l}-x}{t_{j+l}-t_j} (t_{j-1+l}-\mu) \phi_{j-1,l}(\mu) B_{j,l}(x) \\
&= \sum_{j=1}^{n} \frac{x-t_j}{t_{j+l}-t_j} (t_{j+l}-\mu) \phi_{j,l}(\mu) B_{j,l}(x) \\
&\quad + \sum_{j=2}^{n+1} \frac{t_{j+l}-x}{t_{j+l}-t_j} (t_{j}-\mu) \phi_{j,l}(\mu) B_{j,l}(x) \\
&= (x-\mu) \sum_{j=1}^{n} \phi_{j,l}(\mu) B_{j,l}(x), \quad x \in (t_{l+1}, t_{n+1}) \subset (t_l, t_{s+1}).
\end{aligned}$$

□

The B-splines of order k naturally define a linear subspace $\mathcal{S}_{k,\mathbf{t}}$:

$$\mathcal{S}_{k,\mathbf{t}} = \text{span}\left\{ B_{jk}(x) \mid j = 1, \cdots, n \right\}. \tag{2.105}$$

Then $\mathcal{S}$ contains all polynomials of degree $\leq k-1$. Indeed, differentiating (2.104) in μ, one gets

$$\frac{(x-\mu)^{k-\nu}}{(k-\nu)!} = \sum_{j=1}^{n} \frac{(-1)^{\nu} \partial_{\mu}^{\nu} \phi_{jk}(\mu) B_{jk}(x)}{(k-1)!}, \quad x \in (t_k, t_{n+1}). \tag{2.106}$$

If a knot t_i has a multiplicity of $m_i > 1$, that is,

$$t_{i-1} < \underbrace{t_i = t_{i+1} = \cdots = t_{i+m_i-1}}_{\text{multiplicity } = m_i} < t_{i+m_i},$$

then let $\mu = t_i$, for any $1 \leq \nu \leq m_i$,

$$\frac{(x-t_i)^{k-\nu}}{(k-\nu)!} = \sum_{j=i-(k-\nu)}^{s} \frac{(-1)^{\nu} \partial_{\mu}^{\nu} \phi_{jk}(t_i) B_{jk}(x)}{(k-1)!}, \quad x \in (t_{i+\nu-1}, t_{n+1}) = (t_i, t_{n+1}). \tag{2.107}$$

However, $\partial_{\mu}^{\nu} \phi_{jk}(t_i) = 0$ for $i-1 \geq j \geq i-(k-\nu)$, thus

$$\frac{(x-t_i)^{k-\nu}}{(k-\nu)!} = \sum_{j=i}^{n} \frac{(-1)^{\nu} \partial_{\mu}^{\nu} \phi_{jk}(t_i) B_{jk}(x)}{(k-1)!}, \quad x \in (t_i, t_{n+1}). \tag{2.108}$$

We also notice $B_{jk}(x) = 0$ if $x \le t_i$ for all $j \ge i$, therefore,

$$\frac{(x-t_i)_+^{k-\nu}}{(k-\nu)!} = \sum_{j=i}^{n} \frac{(-1)^\nu \partial_\mu^\nu \phi_{jk}(t_i) B_{jk}(x)}{(k-1)!}, \quad x \in (t_k, t_{n+1}). \tag{2.109}$$

□

Definition 2.3.8. *Suppose $\boldsymbol{\zeta} = \{\zeta_i\}_{i=1}^{q+1}$ is an increasing sequence and denote by $\mathbf{m} = \{m_i\}_{i=2}^{q}$ an integer nonnegative sequence that $m_i \le k$. Define $\Pi_{k,\mathbf{t},\mathbf{m}}$ as the space of piecewise polynomials of degree at most $k-1$ with knots $\{\zeta_i\}_{i=1}^{q+1}$ and satisfying the additional jump condition $\forall f \in \Pi_{k,\boldsymbol{\zeta},\mathbf{m}}$*

$$\partial^{\nu-1} f(\zeta_i^+) = \partial^{\nu-1} f(\zeta_i^-), \quad 1 \le \nu \le m_i. \tag{2.110}$$

Theorem 2.3.9 (Curry-Schoenberg)**.** *Let the non-decreasing sequence* $\mathbf{t}$ *be*

$$\underbrace{\zeta_1, \cdots, \zeta_1}_{\text{multiplicity}=k}, \underbrace{\zeta_2, \cdots, \zeta_2}_{\text{multiplicity}=k-m_2}, \cdots, \underbrace{\zeta_q, \cdots, \zeta_q}_{\text{multiplicity}=k-m_n}, \underbrace{\zeta_{q+1}, \cdots, \zeta_{q+1}}_{\text{multiplicity}=k} \tag{2.111}$$

then on the interval $[t_k, t_{n+1}]$, $n = \text{card}(\mathbf{t}) - k$,

$$\Pi_{k,\boldsymbol{\zeta},\mathbf{m}} = \mathcal{S}_{k,\mathbf{t}}.$$

Proof. $\Pi_{k,\boldsymbol{\zeta},\mathbf{m}}$ is spanned by the basis functions

$$\begin{aligned} &(x-\mu)^{k-\nu}, \quad \nu = 1, \cdots, k; \\ &(x-t_i)_+^{k-\nu}, \quad i = 2, \cdots, q, \quad 1 \le \nu \le k - m_i. \end{aligned}$$

Therefore, $\Pi_{k,\boldsymbol{\zeta},\mathbf{m}} \subset \mathcal{S}_{k,\mathbf{t}}$. The dimension of $\mathcal{S}_{k,\mathbf{t}}$ is the number of B-splines B_{jk}, which is $\text{card}(\mathbf{t}) - k = qk - \sum_{i=2}^{n} m_i$ which corresponds to the total number of bases of $\Pi_{k,\boldsymbol{\zeta},\mathbf{m}}$. □

From now on, we assume that the knot sequence $\mathbf{t}$ is defined as Theorem 2.3.9, where the first and last knots are repeated k times, then the basic interval $[t_k, t_{n+1}]$ coincides with the range of knots. By the partition of unity (see Lemma 2.3.6), the spline function in $\mathcal{S}_{k,\mathbf{t}}$ is a strictly convex combination of the coefficients. In other words, the spline function is bounded from below and above by the coefficients nearby.

Theorem 2.3.10 (Convex Hull)**.** *Suppose $x \in (t_i, t_{i+1})$, then*

$$\min_{i+1-k \le j \le i} c_j \le \sum_{j=i-k+1}^{i} c_j B_{jk}(x) \le \max_{i+1-k \le j \le i} c_j.$$

2.3.4 B-Spline Interpolation

Let $n = \dim(\mathcal{S}_{k,\mathbf{t}})$ for the knots $\mathbf{t}$. For the observation data $\{(x_i, y_i)\}_{i=1}^n$, the B-spline interpolation finds a spline $f(x) = \sum_{j=1}^n c_j B_{jk}$ that

$$\sum_{j=1}^{n} c_j B_{jk}(x_i) = y_i, \quad i = 1, \cdots, n. \tag{2.112}$$

We first consider the criteria for the existence and uniqueness of the interpolation.

Theorem 2.3.11 (Existence and Uniqueness). *Let $\{x_j\}_{j=1}^n$ be strictly increasing. The matrix $A := (B_{jk}(x_i))_{ij}$ is non-singular if and only if $B_{ik}(x_i) \neq 0$ or equivalently $t_i < x_i < t_{i+k}$.*

Proof. The "only if" part is trivial. □

The following theorem shows that the B-spline representation is relatively stable for lower degrees.

Theorem 2.3.12 (Good Condition). *Given any knot sequence, let $f = \sum_{j=1}^n c_j B_{jk}$ on $[t_k, t_{n+1}]$, then*

$$\frac{1}{D_{k,\infty}} \max_{1\le j\le n} |c_j| \le \|f\|_\infty \le \max_{1\le j\le n} |c_j|, \tag{2.113}$$

where the condition number $D_{k,\infty} \le k2^{k-1}$.

Proof. □

A precision characterization of the constant is conjectured as $D_{k,\infty} < c2^k$, this is known as the "De Boor's 2^k conjecture" De Boor (1990). Currently, the best-known result is $D_{k,\infty} \le k2^{k-1}$ by Scherer and Shadrin (1999).

2.4 Reproducing Kernel Hilbert Space

Let H be a Hilbert space of functions defined on $\mathcal{X}$, with inner product $\langle \cdot, \cdot \rangle$ such that the *point evaluation map* $x : H \mapsto \mathbb{C}$ is bounded $\forall x \in \mathcal{X}$. Then H is called *reproducing kernel Hilbert space* or RKHS for short. By Riesz's representation Theorem, there exists $\psi_x \in H$ that

$$f(x) = \langle f, \psi_x \rangle, \quad \forall f \in H. \tag{2.114}$$

The function $\mathcal{K}(x, y) := \psi_x(y)$ is the *reproducing kernel* and $\psi_x(\cdot)$ is the *reproducing element* for point $x \in \mathcal{X}$.

Example 2.4.1. *$L^2[0, 1]$ with the usual inner product is not an RKHS. However, the Sobolev space $W^{1,2}[0, 1]$ with its usual norm is a RKHS by the embedding theorems. One can prove that any Sobolev spaces with embedding $C^0(\Omega) \subset\subset W^{k,p}(\Omega)$ will be an RKHS on Ω.*

The kernel $\mathcal{K}(x,y) : \mathcal{X} \times \mathcal{X} \to \mathbb{C}$ satisfies

$$\mathcal{K}(x,y) = \langle \psi_x, \psi_y \rangle, \tag{2.115}$$

and the kernel is Hermitian and positive definite. Its converse is also true.

Example 2.4.2. *The positive definite $\mathcal{K}(x,x) \geq 0$ is strict, that is, if $\mathcal{K}(x,x) = 0$, then $\|\psi_x\|_H = 0$ which forces $\psi_x = 0$. Furthermore, the Cauchy-Schwarz inequality implies that*

$$|\mathcal{K}(x,y)| \leq \sqrt{\mathcal{K}(x,x)\mathcal{K}(y,y)}. \tag{2.116}$$

Theorem 2.4.3 (Moore-Aronszajn)**.** *If $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{C}$ is a Hermitian, positive definite kernel, then there is a unique RKHS with $\mathcal{K}$ as its reproducing kernel.*

Proof. We prove the RKHS is $H = \overline{\text{span}\{\mathcal{K}(x,\cdot) \mid x \in \mathcal{X}\}}$ equipped with the induced inner product $\langle \mathcal{K}(x,\cdot), \mathcal{K}(y,\cdot) \rangle_H = \mathcal{K}(x,y)$. The verification is straightforward. For uniqueness, assume that $H' \supset H$ is another RKHS with kernel $\mathcal{K}$ and inner product $\langle \cdot, \cdot \rangle_{H'}$. Then

$$\mathcal{K}(x,y) = \langle \mathcal{K}(x,\cdot), \mathcal{K}(y,\cdot) \rangle_H = \langle \mathcal{K}(x,\cdot), \mathcal{K}(y,\cdot) \rangle_{H'}, \tag{2.117}$$

which implies the inner products are identical on H. Finally, $\forall f \in H'$, write $f = f_H + f_{H^\perp}$, then

$$f(x) = \langle f, \mathcal{K}(x,\cdot) \rangle_{H'} = \langle f_H, \mathcal{K}(x,\cdot) \rangle_{H'} + \langle f_{H^\perp}, \mathcal{K}(x,\cdot) \rangle_{H'} = f_H(x). \tag{2.118}$$

Therefore, $H' = H$. □

Theorem 2.4.4. *Let $\mathcal{K}_H : \mathcal{X} \times \mathcal{X} \to \mathbb{C}$ be the reproducing kernel for the Hilbert space H and $\mathcal{K}_{H'} : \mathcal{X} \times \mathcal{X} \to \mathbb{C}$ be the reproducing kernel for a closed subspace $H' \subset H$. Then $\forall f \in H$,*

$$f'(x) := \langle f, \mathcal{K}_{H'}(x,\cdot) \rangle \in H', \tag{2.119}$$

and f' is the projection of f onto H'.

Proof. For any $f \in H$, let $f = f_{H'} + f_{H'^\perp}$, where $f_{H'}$ is the projection of f onto H', then note $\mathcal{K}_{H'}(x,\cdot) \in H'$,

$$f'(x) = \langle f_{H'} + (f - f_{H'}), \mathcal{K}_{H'}(x,\cdot) \rangle = \langle f_{H'}, \mathcal{K}_{H'}(x,\cdot) \rangle = f_{H'}(x). \tag{2.120}$$

Therefore $f' = f_{H'}$. Moreover, by the definition,

$$\mathcal{K}'_H(z,x) = \langle \mathcal{K}_H(z,\cdot), \mathcal{K}_{H'}(x,\cdot) \rangle = \overline{\langle \mathcal{K}_{H'}(x,\cdot), \mathcal{K}_H(z,\cdot) \rangle} = \overline{\mathcal{K}_{H'}(x,z)} = \mathcal{K}_{H'}(z,x) \tag{2.121}$$

which implies $\mathcal{K}'_H = \mathcal{K}_{H'}$ is the projection of $\mathcal{K}_H$ onto H'. □

Example 2.4.5. *If $H = L^2(\mathcal{X}, \mu)$ be the standard Lebesgue space with probability measure μ, then for any closed subspace $H' \subset H$, the projection of $f \in H$ onto H' is*

$$f'(x) = \langle f, \mathcal{K}_{H'}(x,\cdot) \rangle = \int_{\mathcal{X}} f(z)\mathcal{K}_{H'}(x,z)d\mu(z). \tag{2.122}$$

Note that the Hilbert space H itself may not be an RKHS; it only needs H' to be an RKHS.

Example 2.4.6. *If $\mathcal{L} \in H^*$ is a bounded linear functional on the Hilbert space H, then by Rietz representation theorem, there exists $\ell \in H$ such that*

$$\mathcal{L}\mathcal{K}(x,\cdot) = \langle \mathcal{K}(x,\cdot), \ell\rangle = \overline{\langle \ell, \mathcal{K}(x,\cdot)\rangle} = \overline{\ell(x)}. \tag{2.123}$$

Similarly, for any bounded linear operator $\mathcal{A} : H \to H$ can be represented by the reproducing kernel in the sense that

$$(\mathcal{A}f)(x) = \langle \mathcal{A}f, \mathcal{K}(x,\cdot)\rangle = \langle f, \mathcal{A}^*\mathcal{K}(x,\cdot)\rangle. \tag{2.124}$$

Theorem 2.4.7. *If H is a separable RKHS with an orthonormal basis $\{\phi_i\}_{i\geq 1}$, the reproducing kernel allows an absolutely convergent expansion.*

$$\mathcal{K}(x,y) = \sum_{i=1}^{\infty} \phi_i(x)\overline{\phi_i(y)}, \tag{2.125}$$

which is independent of the choice of basis. (Note that separable Hilbert spaces have countably many orthonormal bases.)

Proof. Let the subspace $H_n = \text{span}\{\phi_i\}_{i=1}^n$, then the reproducing kernel of H_n is

$$\mathcal{K}_n(x,y) = \sum_{i=1}^{n} \overline{\phi_i(x)}\phi_n(y),$$

since for every $f \in H_n$, $f(x) = \sum_{i=1}^n \langle f, \phi_i\rangle \phi_i(x)$, thus

$$\langle f, \mathcal{K}_n(x,\cdot)\rangle = \sum_{i=1}^{n} \langle f, \phi_i\rangle \phi_i(x) = f(x),$$

which implies $\mathcal{K}_n$ is the reproducing kernel on H_n. Then let $n \to \infty$, using the fact (see Exercise 2.5.14) that

$$\sum_{i=1}^{n} |\phi_i(x)|^2 = \mathcal{K}_n(x,x) \leq \mathcal{K}(x,x). \tag{2.126}$$

Then, the expansion converges absolutely by the Cauchy-Schwarz inequality. □

2.4.1 Interpolation in RKHS

Let H be an RKHS on the non-empty set $\mathcal{X}$ with the reproducing kernel $\mathcal{K}(x,y)$. Suppose that the data $\{(x_j, y_j)\}_{j=1}^m$ are generated by $y_j = f(x_j) = \langle f, \mathcal{K}(x_j,\cdot)\rangle$ for an unknown function $f \in H$. The interpolation finds a function f^* such that

$$\langle f^*, \mathcal{K}(x_j,\cdot)\rangle = y_j, \quad j = 1, \cdots, m. \tag{2.127}$$

The choice for f^* may not be unique. In practice, the minimal norm is preferred, that is, $f^* \in \text{span}\{\mathcal{K}(x_j,\cdot)\}_{j\in[m]}$.

Theorem 2.4.8. *Suppose $\{\mathcal{K}(x_j,\cdot)\}_{j\in[m]}$ are linearly independent. If there exists an element $h \in$ span$\{\mathcal{K}(x_j,\cdot)\}_{j\in[m]}$ such that $h(x_j) = y_j$, then $\|h\|$ has the minimal norm.*

Proof. □

If the number of data $m \to \infty$, then the problem becomes related to the *universal interpolation problem*.

Definition 2.4.9. *Let H be an RKHS with a reproducing kernel $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{C}$. The sequence of points $\{x_i\}_{i\geq 1} \subset \mathcal{X}$ is a universal interpolation sequence (UIS) if the interpolation problem of finding $f \in H$*

$$f(x_i) = y_i, \quad i \in \mathbb{N} \tag{2.128}$$

is solvable whenever $\sum_{i\geq 1} \frac{|y_i|^2}{\mathcal{K}(x_i,x_i)} < \infty$.

Lemma 2.4.10. *The sequence $\{x_i\}_{i\in\mathbb{N}}$ is a UIS if and only if the infinite Hermitian matrix A with entries*

$$A_{ij} = \frac{\mathcal{K}(x_i,x_j)}{\sqrt{\mathcal{K}(x_i,x_i)\mathcal{K}(x_j,x_j)}} \tag{2.129}$$

has all the eigenvalues bounded from zero.

Proof. □

Corollary 2.4.11. *Suppose H is an RKHS on $\mathbb{R}^d$ and let the set $G \subset \mathbb{R}^d$ be a group. If the reproducing kernel satisfies $\mathcal{K}(x,y) = \mathcal{K}(x-y)$ and*

$$\sum_{g\in G, g\neq 0} |\mathcal{K}(g)| < \mathcal{K}(0),$$

then G is a UIS.

Proof. The matrix entry $A_{ij} = \mathcal{K}(x_i - x_j)$. For $\forall x \in G$, $x - G$ traverses the group, therefore, for a fixed $j \in \mathbb{N}$, the summation

$$\sum_{i\neq j} |A_{ij}| = \sum_{g\in G, g\neq 0} |\mathcal{K}(g)| < \mathcal{K}(0) = A_{ii}. \tag{2.130}$$

Therefore, the matrix A is an M matrix (see Chapter 8). The eigenvalues are bounded away from zero by the Gershgorin circle theorem. The conclusion is directly from Lemma 2.4.10. □

Example 2.4.12 (Dirichlet kernel)**.** *TODO: the subgroup is the circle group on $\{k \in \mathbb{N} \mid |k| \leq n\}$.*

2.4.2 Hardy Space $\mathbb{H}^2$

The Hardy space $\mathbb{H}^2$ refers to the holomorphic functions in the unit disk that

$$\sup_{0\leq r<1} \int_0^{2\pi} |f(re^{i\theta})|^2 d\theta < \infty. \tag{2.131}$$

With the usual expansion $f(z) = \sum_{n=0}^{\infty} c_n z^n$, this is equivalent to $\sum_{n=0}^{\infty} |c_n|^2 < \infty$.

2.5 Exercises

The coding part and test cases are available on GitHub.

2.5.1 Theoretical Part

Problem 2.5.1. *Prove Corollary 2.1.15.*

Problem 2.5.2. *Prove there exists a constant $C > 0$ that the following bound holds. Compare this bound with (2.10).*

$$|\omega(x)| \leq \prod_{j=0}^{n} |x - x_j| \leq \frac{n!}{C + \log n} \left| \frac{b-a}{n} \right|^{n+1}. \tag{2.132}$$

Problem 2.5.3. *Estimate the interpolation error for e^x and $x^{-1/2}$ over the interval $[\frac{1}{2}, \frac{3}{2}]$ with equally spaced nodes and Chebyshev nodes, respectively.*

Problem 2.5.4. *Let $-1 \leq x_0 < x_1 < \cdots < x_n \leq 1$ be a set of interpolation nodes and $L_j(x)$ denotes the Lagrange basis polynomial at x_j. Prove if $x \in (x_i, x_{i+1})$, then*

$$L_i(x) + L_{i+1}(x) \geq 1.$$

Hint: Use Rolle's Theorem.

Problem 2.5.5. *Prove the Berstein inequality for trigonometric polynomials $t_n(\theta) = \sum_{k=-n}^{n} c_k e^{ik\theta}$.*

$$\|t_n'(\theta)\|_\infty \leq n\|t_n(\theta)\|_\infty, \quad \theta \in [0, 2\pi).$$

and use it to prove the Markov inequality for any $p_n \in \Pi_n$ and $x \in [-1, 1]$,

$$\|p_n'(x)\|_\infty \leq n^2 \|p_n'(x)\|_\infty.$$

Problem 2.5.6. *Prove the contour C_ρ (see Section 2.1.4) for Chebyshev nodes on $[-1, 1]$ is an ellipse with foci at ± 1.*

Problem 2.5.7. *Let the nodes $x_0, \ldots, x_n \in [a, b]$ and $f \in C^{n+1}([a, b])$. For any given $x \in [a, b]$, prove there exists $\xi \in [a, b]$ such that the divided difference*

$$f[x_0, x_1, \ldots, x_n, x] = \frac{1}{(n+1)!} f^{(n+1)}(\xi).$$

Hint: Construct the interpolation polynomial on nodes $x_0, \ldots, x_n$ with Newton form first, then regard x as the additional node by (2.44), finally recall the Theorem 2.1.5.

Problem 2.5.8. *Show that the divided difference*

$$f[x_0, x_1, \ldots, x_n] = f[x_{\pi(0)}, x_{\pi(1)}, \ldots, x_{\pi(n)}],$$

where $\pi \in S_{n+1}$ is any permutation.

Problem 2.5.9. *Prove Lemma 2.2.8.*

Problem 2.5.10 (discrete circular convolution)**.** *Let two vectors* $\mathbf{a} = (a_0, \ldots, a_{N-1})$ *and* $\mathbf{b} = (b_0, \ldots, b_{N-1})$ *and we assume the convention that* $a_{N+j} = a_j$ *and* $b_{N+j} = b_j$ *to extend the vector to infinite size. The discrete circular convolution* $\mathbf{c} = (c_0, \ldots, c_{N-1}) = \mathbf{a} * \mathbf{b}$ *is defined by*

$$c_j = \sum_{l=0}^{N-1} a_l b_{j-l}.$$

Prove that

$$\mathtt{DFT}(\mathbf{c}) = \mathtt{DFT}(\mathbf{a}) \circ \mathtt{DFT}(\mathbf{b}),$$

where $\circ$ *is the Hadamard product or element-wise product.*

Problem 2.5.11 (Minimimum norm property)**.** *Let* $f \in C^2([a,b])$ *and* s_3 *be the* <u>*natural*</u> *cubic spline with interpolating* f*. Prove*

$$\int_a^b |{s_3}''(x)|^2 dx \le \int_a^b |f''(x)|^2 dx \tag{2.133}$$

The equality holds only when $f(x) = s_3(x)$ *everywhere. Hint: One needs to prove*

$$\int_a^b (f''(x) - {s_3}''(x)){s_3}''(x) dx = 0, \tag{2.134}$$

then simply use the AM-GM inequality. The above equality can be proved using integration-by-parts twice (remember the interpolation conditions).

Problem 2.5.12. *Let the following tridiagonal matrix (corresponding to natural cubic spline)*

$$A = \begin{pmatrix} 1 & \frac{h_2}{2(h_1+h_2)} & & & \\ \frac{h_2}{2(h_2+h_3)} & 1 & \frac{h_3}{2(h_2+h_3)} & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & \frac{h_{n-2}}{2(h_{n-2}+h_{n-1})} \\ & & & \frac{h_{n-1}}{2(h_{n-1}+h_n)} & 1 \end{pmatrix}$$

and denote $B = A - I$*. Prove* $\|B\|_\infty \le \frac{1}{2}$ *and* $\|A^{-1}\|_\infty \le 2$*.*

Problem 2.5.13. *Prove the Lemma 2.3.5.*

Problem 2.5.14. *Let* H' *be a closed subspace of the Hilbert space* H*. Show that the reproducing kernels* $\mathcal{K}_{H'}(x,x) \le \mathcal{K}_H(x,x)$*.*

Problem 2.5.15. *Find the reproducing kernel for* $W^{1,2}(\mathbb{R})$ *with the induced inner product*

$$\langle f, g \rangle = \int_{\mathbb{R}} (f(x)g(x) + f'(x)g'(x))\, dx.$$

2.5.2 Computational Part

Problem 2.5.16. *Implement the divided difference to compute the coefficients of the Newton form. You should call the Horner's scheme from the previous coding project. State the consequence when the length of* $[a, b]$ *is very small.*

Problem 2.5.17. *Implement the Lagrange interpolation schemes on equispaced and (extended) Chebyshev nodes.*

Problem 2.5.18. *Implement a routine to compute Hermite interpolation with* $m_j = 1$ *for each* $j = 0, \ldots, n$*. Estimate your routine's time complexity and space complexity.*

Problem 2.5.19. *Implement the cubic spline interpolation with periodic and natural spline conditions.*

Problem 2.5.20. *Implement discrete Fourier transform using a recursive approach (assume the total number of nodes is a power of two).*

Problem 2.5.21. *Implement the B-splines* B_{jk} *for the knots mentioned in Theorem 2.3.9.*

Extended Reading

Curry, H. B. and Schoenberg, I. J. (1947). On spline distributions and their limits-the pólya distribution functions. In *Bulletin of the American Mathematical Society*, volume 53, pages 1114–1114. AMER MATHEMATICAL SOC 201 CHARLES ST, PROVIDENCE, RI 02940-2213.

De Boor, C. (1978). *A practical guide to splines*, volume 27. springer-verlag New York.

De Boor, C. (1990). The exact condition of the b-spline basis may be hard to determine. *Journal of Approximation theory*, 60(3):344–359.

Erdös, P. (1958). Problems and results on the theory of interpolation. i. *Acta Mathematica Academiae Scientiarum Hungarica*, 9(3-4):381–388.

Erdős, P. (1964). Problems and results on the theory of interpolation. ii. *Acta Mathematica Academiae Scientiarum Hungarica*, 12(1-2):235–244.

Jackson, D. (1912). On approximation by trigonometric sums and polynomials. *Transactions of the American Mathematical society*, 13(4):491–515.

Jackson, D. (1913). On the accuracy of trigonometric interpolation. *Transactions of the American Mathematical Society*, 14(4):453–461.

Prenter, P. M. et al. (2008). *Splines and variational methods*. Courier Corporation.

Scherer, K. and Shadrin, A. Y. (1999). New upper bound for the b-spline basis condition number: Ii. a proof of de boor's 2k-conjecture. *Journal of Approximation theory*, 99(2):217–229.

Trefethen, L. N. and Weideman, J. (1991). Two results on polynomial interpolation in equally spaced points. *Journal of Approximation Theory*, 65(3):247–260.

Turetskii, A. H. (1940). The bounding of polynomials prescribed at equally distributed points. In *Proc. Pedag. Inst. Vitebsk*, volume 3, pages 117–127.

CHAPTER 3

DIFFERENTIATION AND QUADRATURE

A vast number of applications such as the calculation of tangent vectors or areas lead to the problem of computing

$$\mathcal{D}(f) := \frac{d}{dx}f(x), \quad \mathcal{I}(f) := \int_a^b f(x)dx, \tag{3.1}$$

for certain function $f(x) \in C^k([a,b])$. Accurate evaluations would sometimes be difficult if an analytic expression is absent. Especially when the function values of f are only accessible at a finite number of nodes. Therefore, it is important to find simple yet effective methods to approximate the derivatives and integrals.

3.1 Extrapolation

From the previous discussion, we already know that interpolation provides an estimate *within* the original observation range. The extrapolation is similar but aims to produce estimates outside the observation range. However, extrapolation may be subject to a greater uncertainty (Fig 3.1), one should use it only when an overestimate is hardly occurring.

3.1.1 Richardson Extrapolation

Suppose there is a sequence of estimates $A(h)$ depending on the parameter h smoothly, the limit $A^* = \lim_{h\to 0^+} A(h)$ is the quantity to be computed. In practice, we only have access to $A(h)$ for a few values of h. Using these values to estimate A^* is a typical problem in extrapolation.

The basic idea behind *Richardson extrapolation* is to use polynomial interpolation with a sequence of nodes $h_j \to 0$. Suppose that the function $A(h)$ admits the following asymptotic

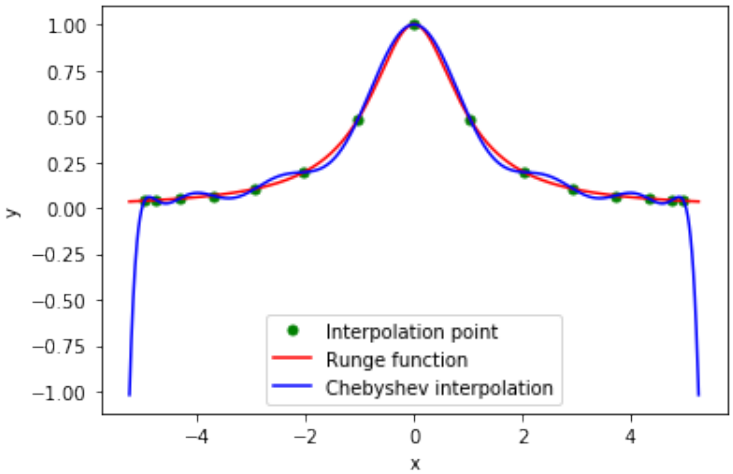

Figure 3.1: Extrapolation behavior for Chebyshev interpolation with 15 nodes.

expansion:

$$A(h) = a_0 + a_1 h^\gamma + a_2 h^{2\gamma} + \cdots + a_k h^{k\gamma} + \mathcal{O}(h^{(k+1)\gamma}) \tag{3.2}$$

for any $h > 0$ and $k \geq 0$. Then $A^* = a_0$ and $A(h) = A^* + \mathcal{O}(h^\gamma)$. Suppose we have access to the values $A(h_0), \ldots, A(h_n)$, then this uniquely determines a polynomial $f_n \in \Pi_n$ and $f_n(h_j^\gamma) = A(h_j)$. We will approximate $A(0) \approx f_n(0)$. The computation of f_n follows the construction of Newton form.

Lemma 3.1.1. *Suppose h_j can be represented as*

$$h_j = \frac{\hbar}{t_j}$$

for some adjustable parameter $\hbar$ and scaling constants $1 < t_0 < t_1 < \cdots < t_{n-1}$. Then

$$f_n(0) = A^* + (-1)^n \frac{a_{n+1}}{\prod_{j=0}^n t_j^\gamma} \hbar^{(n+1)\gamma} + \mathcal{O}(\hbar^{(n+2)\gamma}), \quad \text{as } \hbar \to 0.$$

Proof. We view $A(h)$ as a polynomial with respect to h^γ of degree $(n+1)$ with an addition perturbation $\mathcal{O}(h^{(n+2)\gamma})$. Then we have the following.

$$A(h) = p_{n+1}(h^\gamma) + \mathcal{O}(h^{(n+2)\gamma}). \tag{3.3}$$

Let $\tilde{f}_n$ be the interpolation polynomial of degree n to p_{n+1},

$$p_{n+1}(x) \equiv \tilde{f}_n(x) + p[x, x_0, x_1, \ldots, x_n] \prod_{j=0} (x - h_j^\gamma). \tag{3.4}$$

where $p[x, x_0, x_1, \ldots, x_n]$ is the coefficient of the leading power in p_{n+1}, a_{n+1}. Thus,

$$A^* = p_{n+1}(0) = \tilde{f}_n(0) + a_{n+1} \prod_{j=0} (0 - h_j^\gamma). \tag{3.5}$$

Use the result we have discussed in the stability of polynomial interpolation, see Chapter 2.1.6. Therefore,

$$|\tilde{f}_n(0) - f_n(0)| \leq \lambda_n(0) \cdot \mathcal{O}(\hbar^{(n+2)\gamma}) \tag{3.6}$$

Here the Lebesgue function at zero $\lambda_n(0)$ is

$$\lambda_n(0) = \sum_{j=0}^{n} \prod_{k=0,k\neq j}^{n} \left| \frac{h_k^\gamma}{h_k^\gamma - h_j^\gamma} \right| = \sum_{j=0}^{n} \prod_{k=0,k\neq j}^{n} \left| \frac{1}{1-(\frac{t_k}{t_j})^\gamma} \right|, \tag{3.7}$$

which is independent of $\hbar$. □

The Richardson extrapolation considers the special choice of $t_j = t^j$ for some $t > 1$. The error estimate then is

$$f_n(0) = A^* + \left(\frac{(-1)^n}{t^{n(n+1)\gamma/2}} a_{n+1} \right) \hbar^{(n+1)\gamma} + \mathcal{O}(\hbar^{(n+2)\gamma}). \tag{3.8}$$

There are easier ways to calculate the Richardson extrapolation using the following expansion.

$$A(h) - t^\gamma A\left(\frac{h}{t}\right) = (1-t^\gamma)A^* + \cancel{a_1 \left(h^\gamma - t^\gamma \left(\frac{h}{t}\right)^\gamma \right)} + a_2 \left(h^{2\gamma} - t^\gamma \left(\frac{h}{t}\right)^{2\gamma} \right) + \dots \tag{3.9}$$

Let $A_1(h) = \frac{A(h) - t^\gamma A(\frac{h}{t})}{1-t^\gamma}$, we obtain the first iteration result as

$$A^* \approx A_1(h) + \mathcal{O}(h^{2\gamma}), \tag{3.10}$$

then follow the same idea, we cancel the $\mathcal{O}(h^{2\gamma})$ term by

$$A_1(h) - t^{2\gamma} A_1\left(\frac{h}{t^2}\right) = (1-t^{2\gamma})A^* + \mathcal{O}(h^{3\gamma}). \tag{3.11}$$

Therefore by taking $A_2(h) = \frac{A_1(h) - t^{2\gamma} A_1(\frac{h}{t^{2\gamma}})}{1-t^{2\gamma}}$, the second iteration satisfies

$$A^* \approx A_2(h) + \mathcal{O}(h^{3\gamma}). \tag{3.12}$$

However, such a process can constantly refine the approximation due to the potentially fast-growing constant in the $\mathcal{O}$ notation.

3.1.2 Wynn's Epsilon Method

Wynn's ε method is another kind of extrapolation algorithm that is recommended as the best all-purpose acceleration method. It has a strong connection with Padé approximation and continued fractions. We will not cover the detailed derivation of the theory in this section. However, Wynn's ε method still has its limitations if the sequence converges to the desired value too slowly.

The algorithm is stated as follows. Let $s_0, s_1, \dots, s_n, \dots$ be a sequence converging to the desired quantity.

1. Initialization. For $j = 0, 1, 2, \ldots$, set
$$\varepsilon_{-1}^{(j)} = 0 \text{ (guarding elements)}, \quad \varepsilon_0^{(j)} = s_j.$$

2. Iteration. For $j, k = 0, 1, 2, \ldots$,
$$\varepsilon_{k+1}^{(j)} = \varepsilon_{k-1}^{(j+1)} + [\varepsilon_k^{(j+1)} - \varepsilon_k^{j}]^{-1}.$$

The extrapolated results are stored at the columns $\varepsilon_{2l}^{(j)}$, $j, l = 0, 1, \ldots$.

Example 3.1.2. *It is known that $\pi/4$ can be calculated by the asymptotic expansion:*
$$\arctan z = z - \frac{z^3}{3} + \frac{z^5}{5} - \ldots \tag{3.13}$$
at $z = 1$. Define the function $A(h)$ such that
$$A(h) = \sum_{j=0}^{1/h} \frac{(-1)^j}{2j+1} = \frac{\pi}{4} + a_1 h + a_3 h^3 + \ldots \tag{3.14}$$
Then the approximation error is $\mathcal{O}(h)$, which is very slow. Taking $h = 10^{-3}$ has around 2×10^{-4} error. We test with two extrapolation algorithms.

- *Richardson Extrapolation. We take $1/h = 250, 500, 1000, 2000$ and calculate that the Richardson extrapolation three times would result in almost machine precision.*
- *Wynn's ε method. We take the sequence*
$$s_k = \sum_{j=0}^{k} \frac{(-1)^j}{2j+1}$$
as the truncated series at $z = 1$. With about 20 terms, we already reached machine precision.

Theorem 3.1.3. *TODO: Theory for ε method.*

3.2 Differentiation with Finite Difference

Let $f \in C^2([a, b])$, we recall the Taylor expansion with reminder term,
$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(\xi), \tag{3.15}$$
where $\xi = \xi(x) \in [a, b]$, therefore we can compute the derivative by
$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f''(\xi). \tag{3.16}$$

This approximation offers a way to evaluate the derivative $f'(x)$ with the error term $\mathcal{O}(h)$. In addition, the above formula is *exact* when f is a polynomial of degree 1. We say that an approximation has degree k accuracy if the approximation is *exact* for any polynomial of degree k.

Another important terminology is the *order*. It describes the error term of the approximation. In the above case, the error term scales like $\mathcal{O}(h)$ as $h \to 0$, then the approximation is of order 1 or first order. In general, if the error term behaves like $\mathcal{O}(h^p)$, then we can say that it is the p-th order approximation.

The *stencil* refers to a set of nodes used in the approximation. In the above example, we have used $x, x+h$. We can of course create its sibling

$$f'(x) = \frac{f(x) - f(x-h)}{h} + \frac{h}{2} f''(\zeta), \tag{3.17}$$

which uses the nodes $x-h, x$. When all nodes are $\geq x$ or $\leq x$, we say the scheme is forward or backward, respectively.

3.2.1 Finite Difference from Taylor Expansion

All results related to finite difference can be easily derived from the Taylor expansion. Suppose we would like to approximate a high-order derivative $f^{(m)}(x)$ with some nodes scattered around x in the following form.

$$\frac{1}{h^m} \sum_{j=0}^{n} c_j f(x + a_j h) = f^{(m)}(x) + E(x, h), \tag{3.18}$$

where E is the error term and $a_j \in \mathbb{Z}$ (sometimes half integers are used). Since $h \to 0$, we can expand all $f(x + a_j h)$ locally by Taylor series and truncate at least order $(m+1)$.

$$\frac{1}{h^m} \sum_{j=0}^{n} c_j \left(\sum_{p=0}^{m} \frac{1}{p!} f^{(p)}(x) a_j^p h^p + \frac{f^{(p+1)}(\xi_j)}{(p+1)!} a_j^{p+1} h^{p+1} \right). \tag{3.19}$$

We need all lower (and maybe higher than m-th) order derivatives of f canceled in the above summation, which is (using Kronecker delta),

$$\sum_{j=0}^{n} c_j a_j^p = m! \cdot \delta_{pm}, \quad 0 \leq p \leq m. \tag{3.20}$$

It is straightforward that $n \geq m$ is necessary; otherwise, the first equation system (Vandermonde matrix) must have a zero solution. Suppose that we have found a solution (c_j, a_j), $j = 0, \ldots, n$, to the above system, then in the sequel, we try to estimate the error term $E(x, h)$. Especially, when $n = m$, there are two cases. Let the constant $C = \sum_{j=0}^{n} c_j a_j^{m+1}$, then

1. If $C = 0$, then the error term can be estimated by expanding to $(m+2)$-th derivative.

$$E(x,h) = \frac{1}{h^m}\sum_{j=0}^{n} c_j \frac{f^{(m+2)}(\xi_j)}{(m+2)!} a_j^{m+2} h^{m+2} = h^2 \left(\sum_{j=0}^{n} c_j a_j^{m+2} \frac{f^{(m+2)}(\xi_j)}{(m+2)!}\right). \tag{3.21}$$

   One can expect higher-order accuracy when more terms are involved.

2. If $C \neq 0$, then the error term is

$$E(x,h) = \frac{1}{h^m}\sum_{j=0}^{n} c_j \frac{f^{(m+1)}(\xi_j)}{(m+1)!} a_j^{m+1} h^{m+1} = h \left(\sum_{j=0}^{n} c_j a_j^{m+1} \frac{f^{(m+1)}(\xi_j)}{(m+1)!}\right). \tag{3.22}$$

Remark 3.2.1. *The abscissa ξ_j, $j = 0, \ldots, n$ are in general distinct, but it is possible to choose a single ξ to simplify the representation through the intermediate value theorem.*

Lemma 3.2.2. *If $f \in C^{m+1}(\mathcal{I})$, where $\xi_j \in \mathcal{I}$, then there exists $\xi \in \mathcal{I}$ such that*

$$\sum_{j=0}^{n} c_j a_j^{m+1} \frac{f^{(m+1)}(\xi_j)}{(m+1)!} = \sum_{j=0}^{n} c_j a_j^{m+1} \frac{f^{(m+1)}(\xi)}{(m+1)!}, \tag{3.23}$$

if $c_j a_j^{m+1} \geq 0$ (or ≤ 0) for all $j = 0, \ldots, n$.

Proof. Define

$$\psi(x) = \sum_{j=0}^{n} c_j a_j^{m+1} \frac{f^{(m+1)}(x) - f^{(m+1)}(\xi_j)}{(m+1)!},$$

then $\max_j \psi(\xi_j) \geq 0$ and $\min_j \psi(\xi_j) \leq 0$. Then apply the intermediate value theorem. □

Example 3.2.3. *Let $n = 2, m = 2$ for an example. Then the equation system becomes*

$$\begin{aligned} c_0 + c_1 + c_2 &= 0, \\ c_0 a_0 + c_1 a_1 + c_2 a_2 &= 0, \\ c_0 a_0^2 + c_1 a_1^2 + c_2 a_2^2 &= 2. \end{aligned} \tag{3.24}$$

Then using Gauss elimination, one can find that

$$c_2(a_2 - a_0)(a_2 - a_1) = 2.$$

The above formula can be generalized. The constant $C = c_2(a_2 - a_0)(a_2 - a_1)(a_0 + a_1 + a_2)$. We list a few possible choices to satisfy (3.24).

1. *$(a_0, a_1, a_2) = (-1, 0, 1)$, $c_2 = 1$. Then $c_0 = 1$ and $c_1 = -2$ are derived. In this case $C = 0$. We will have the error term*

$$E(x,h) = \frac{h^2}{24}\left(f^{(4)}(\xi_0) + f^{(4)}(\xi_2)\right) \underbrace{\Longrightarrow}_{3.2.2} \frac{h^2}{12} f^{(4)}(\xi).$$

   This is called the central difference scheme.

2. $(a_0, a_1, a_2) = (0, 1, 2)$, $c_2 = 1$. *Then* $c_1 = -2$, $c_0 = 1$, $C \neq 0$. *The error is*

$$E(x,h) = \frac{h}{6}(f^{(4)}(\xi_1) + 8f^{(4)}(\xi_2)) \underbrace{\Longrightarrow}_{3.2.2} \frac{3h}{2} f^{(4)}(\xi).$$

This is the forward difference scheme.

The combination of the coefficients is not unique. The central scheme has better approximation due to its symmetry. Any combination satisfying $a_0 + a_1 + a_2 = 0$ *should have the same order of error.*

The general scheme with $a_j = j$ (or $-j$) can be derived from the following theorem.

Theorem 3.2.4. *In general, if* $n = m$, *then the forward difference scheme satisfies*

$$\begin{aligned} &\sum_{j=0}^{n} (-1)^{n-j} \binom{n}{i} j^p = 0, \quad 0 \le p < n \\ &\sum_{j=0}^{n} (-1)^{n-j} \binom{n}{i} j^n = n!. \end{aligned} \tag{3.25}$$

Proof. It is the easiest to prove by a binomial transform. It can also be proved through induction easily. Let

$$P_0(x) = (x-1)^n, \quad P_k(x) = xP'_{k-1}(x),$$

then one can show inductively that P_k, $k \ge 1$, has following form

$$P_k(x) = n(n-1)\cdots(n-k+1)(x-1)^{n-k}x^k + (x-1)^{n-k+1}F(x) \tag{3.26}$$

with $F(x)$ as a polynomial of the highest degree $k-1$. This can be easily proved since

$$\begin{aligned} P_{k+1}(x) = xP'_k(x) = n(n-1)\cdots(n-k)(x-1)^{n-k-1}x^{k+1} &+ \\ + (x-1)^{n-k}n(n-1)\cdots(n-k+1)kx^k \\ + (x-1)^{n-k}(n-k+1)F(x) \\ + (x-1)^{n-k}(x-1)F'(x). \end{aligned} \tag{3.27}$$

The last three terms can merge into the form (3.26). Therefore, $P_k(1) = 0$ unless $k = n$ and $P_n(1) = n!$ are immediately obtained.

Now if we expand the polynomial P_0 as a monomial,

$$P_0(x) = \sum_{j=0}^{n} \binom{n}{j} x^j (-1)^{n-j}, \tag{3.28}$$

then it is not difficult to show that

$$P_k(x) = \sum_{j=0}^{n} \binom{n}{j} j^k x^j (-1)^{n-j}$$

through induction as well, which is exactly our conclusion by setting $x = 1$. □

Theorem 3.2.5 (forward difference)**.**

$$f^{(n)}(x) = \frac{1}{h^n}\sum_{j=0}^{n}(-1)^{n-j}\binom{n}{j}f(x+jh) + \mathcal{O}(h).$$

The corresponding schemes of backward and central differences can be derived similarly (see Exercise 3.6.1).

3.2.2 Rounding Error Issue

The finite difference formula provides a simple and effective way to evaluate the derivatives, however, its formulation would be sensitive to rounding errors. Take the central difference scheme for $f''(x)$ as an example, one can derive a similar estimate for higher-order derivatives.

Example 3.2.6.

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \frac{h^2}{12}f^{(4)}(\xi) \tag{3.29}$$

The error comes from two sources. The truncation error term from $\frac{h^2}{12}f^{(4)}(\xi)$ and the rounding error from the evaluation of the first term by the basic arithmetic operations. Suppose the addition/subtraction is implemented by Kahan sum (see Exercise 1.5.6) which almost does not introduce errors in the arithmetic operations. Then the rounding error of $f(x+h) - 2f(x) + f(x-h)$ is at most $4\max_{x\in\mathcal{I}}|f(x)|\mathrm{u}$. Therefore the total error

$$|E_{total}| \le \frac{4\max_{x\in\mathcal{I}}|f(x)|\mathrm{u}}{h^2} + \frac{h^2}{12}\max_{x\in\mathcal{I}}|f^{(4)}(x)| \tag{3.30}$$

Minimizing the right-hand-side of (3.30), let $M = \max_{x\in\mathcal{I}}|f(x)|\max_{x\in\mathcal{I}}|f^{(4)}(x)|$, we obtain

$$\min_{h\in\mathbb{R}}|E_{total}| \le \sqrt{\frac{4}{3}\mathrm{u}M}.$$

The optimal achieves at $h^* = \sqrt[4]{48\mathrm{u}M}$. For example, if $f(x) = \exp(x)$ and evaluate its second derivative around $x = 0$, then $\mathcal{M} \sim 1$, the error is around 1.3×10^{-8} for $h^* \sim 2.5\times10^{-4}$.

For higher-order derivatives, the rounding error would have an even greater impact on the finite difference schemes. Then it is much more important to avoid h being too small.

3.2.3 Improve by Extrapolation

Now we can combine the previously discussed extrapolation technique to acquire a higher-ordered scheme. We use a very simple example to show how this works.

Example 3.2.7. *Suppose $A(f,h)$ is the central difference scheme for $f'(x)$, which is*

$$A(f,h) = \frac{f(x+h) - f(x-h)}{2h} \tag{3.31}$$

the previous discussion has claimed that $A(f,h) = f'(x) + \mathcal{O}(h^2)$. Now we try to fit the formulation in the framework of extrapolation. Formally, we can expand $f(x \pm h)$ with Taylor series with infinite terms (might not converge though), that is,

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \cdots \\ f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \cdots \end{aligned} \tag{3.32}$$

Therefore $A(f,h) = f'(x) + \frac{h^2}{6}f''(x) + \frac{h^4}{120}f''''(x) + \cdots$. Here the coefficients are all formal since the convergence is not guaranteed. In the next, we take $A(f, \frac{h}{2})$, which uses a smaller step length to approximate $f'(x)$, then

$$A(f,h/2) = f'(x) + \frac{h^2}{24}f''(x) + \frac{h^4}{1920}f''''(x) + \cdots \tag{3.33}$$

Cancel the $\mathcal{O}(h^2)$ term by

$$\frac{1}{3}\left(4A(f,h/2) - A(f,h)\right) = f'(x) - \frac{h^4}{480}f''''(x) + \cdots .$$

In this way we have built a more accurate formula $A_1(f,h) = \frac{4A(f,h/2)-A(f,h)}{3}$ for $f'(x)$, the error is fourth order. Bring the definition of the finite difference scheme into A_1, then

$$\begin{aligned} A_1(f,h) &= \frac{4}{3}\left(\frac{f(x+h/2) - f(x-h/2)}{h}\right) - \frac{1}{3}\left(\frac{f(x+h) - f(x-h)}{2h}\right) \\ &= \frac{-f(x+h) + 8f(x+h/2) - 8f(x-h/2) + f(x-h)}{6h}. \end{aligned} \tag{3.34}$$

This central difference scheme has $\mathcal{O}(h^4)$ error.

The above example can still iterate through the extrapolation process, since $A_1(f,h) = f'(x) + \mathcal{O}(h^4)$, we can use $A_2(f,h) = \frac{16}{15}A_1(f,h/2) - \frac{1}{15}A_1(f,h)$ to cancel out the $\mathcal{O}(h^4)$ term which leads to a $\mathcal{O}(h^6)$ error. However one should also notice this process requires more nodes for computation: A_1 needs nodes $x \pm h$, $x \pm h/2$, A_2 will acquire additional nodes $x \pm h/4$ for evaluation. Such a higher precision evaluation method takes more computational time, sometimes we need to trade off the efficiency and accuracy.

Remark 3.2.8. *One of the advantages of using extrapolation is that h does not have to be too small which is sensitive to numerical rounding errors. The potential issue would be the growth of derivative with respect to order, for sufficiently smooth functions, extrapolation usually produces quite accurate evaluations. The potential limitation of the Richardson extrapolation is the requirement of known asymptotic expansion (formally only), while the Wynn ε method does not have such a limitation.*

3.3 Quadrature Rules

The numerical quadrature finds the value of an integral

$$\mathcal{I}(f) = \int_a^b f(x)dx$$

from the function values at a finite number of points. We are mostly interested in the following quadrature formula

$$\mathcal{I}_n(f) = (b-a)\sum_{j=0}^{n} w_j f(x_j) \tag{3.35}$$

where x_j are the nodes and w_j are the weights. Similar to the numerical derivatives, we also define some terminologies. The formula $\mathcal{I}_n$ is said to have degree k accuracy if $\mathcal{I}_n$ is exact for all polynomials $f \in \Pi_k$. Since the integration formula is linear, the exactness can be rephrased as

$$\begin{aligned} \mathcal{I}(x^p) &= \mathcal{I}_n(x^p), \quad 0 \le p \le k, \\ \mathcal{I}(x^{k+1}) &\neq \mathcal{I}_n(x^{k+1}). \end{aligned} \tag{3.36}$$

3.3.1 Interpolation Based Rules

The interpolation-based idea is intuitive. Let $q_n(x)$ be the interpolation polynomial on the nodes x_j with values $f(x_j)$, $j = 0, 1, \ldots, n$, respectively. We define the quadrature by interpolation formula as

$$\mathcal{I}_n(f) := \int_a^b q_n(x)dx. \tag{3.37}$$

The above quadrature formula is exact for all degree n polynomials f, therefore it has <u>at least</u> degree n accuracy.

Remark 3.3.1. *In the section of interpolation, we have seen that the L^∞ error between f and q_n could be large (e.g. Runge phenomenon) as $n \to \infty$. So the nodes would be important as well for quadratures.*

Using the Lagrange polynomials, we can represent

$$q_n(x) = \sum_{j=0}^{n} f(x_j)L_j(x), \quad L_j(x) = \prod_{k=0,k\neq j}^{n} \frac{x - x_k}{x_j - x_k}. \tag{3.38}$$

Then it is not difficult to derive

$$\begin{aligned} \mathcal{I}_n(f) &= \sum_{j=0}^{n} f(x_j) \int_a^b L_j(x)dx \\ &= (b-a)\sum_{j=0}^{n} f(x_j) \int_0^1 \prod_{k=0,k\neq j}^{n} \frac{t - t_k}{t_j - t_k} dt, \quad t_j = \frac{x_j - a}{b - a}. \end{aligned} \tag{3.39}$$

therefore the weights $w_j = \int_0^1 \prod_{k=0,k\neq j}^{n} \frac{t-t_k}{t_j-t_k} dt$.

Example 3.3.2 (rectangle rule). *The rectangle rule is the simplest one, where we choose $x_0 = \frac{a+b}{2}$ as the middle point. Then the quadrature rule writes*

$$\mathcal{I}_{0,rectangle}(f) = (b-a)f(x_0).$$

Such a rule is exact for any linear functions, therefore it has a degree one accuracy. We can see that the degree of exactness could exceed n. Later we will see the maximum degree of exactness for such a form is $2n+1$ in the next chapter.

Example 3.3.3 (trapezoid rule). *The trapezoid rule takes $x_0 = a$ and $x_1 = b$.*

$$\mathcal{I}_{1,trapezoid}(f) = (b-a)\left(\frac{1}{2}f(x_0) + \frac{1}{2}f(x_1)\right).$$

One can check that this rule is exact for $f(x) = 1, x$. It also has a degree one accuracy. It has a slightly larger constant in error estimate than the rectangle rule.

3.3.2 Numerical Error of Interpolation Based Rules

Now we try to estimate $|\mathcal{I}(f) - \mathcal{I}_n(f)|$ from above derivation.

Theorem 3.3.4. *Suppose the quadrature rule $\mathcal{I}_n$ has at least degree r accuracy that $r \geq n$ and $f \in C^{r+1}([a,b])$. Then*

$$|\mathcal{I}(f) - \mathcal{I}_n(f)| \leq \Omega_r \frac{(b-a)^{r+2}}{(r+1)!} \max_{x\in[a,b]} |f^{(r+1)}(x)|, \tag{3.40}$$

where the constant Ω_k is defined by

$$\Omega_r := \min_{t_{n+1},\ldots,t_r\in[0,1]} \int_0^1 \prod_{j=0}^{r} |t-t_j|dt, \quad t_j = \frac{x_j - a}{b-a}, j = 0,\ldots,n. \tag{3.41}$$

Proof. Let $x_{n+1},\ldots,x_r$ be additional distinct nodes on $[a,b]$ and define f_r the interpolating polynomial on the nodes $x_0,\ldots,x_r$, since the quadrature rule has degree r accuracy, then

$$\mathcal{I}_n(f) = (b-a)\sum_{j=0}^{n} w_j f(x_j) = (b-a)\sum_{j=0}^{n} w_j f_r(x_j) = \mathcal{I}_n(f_r) = \mathcal{I}(f_r).$$

Using the theories developed in Interpolation, we know that

$$f(x) - f_r(x) = \frac{\omega_r(x) f^{(r+1)}(\xi)}{(r+1)!}, \tag{3.42}$$

where $\omega_r(x) = \prod_{j=0}^{r}(x - x_j)$, therefore

$$|\mathcal{I}(f - f_r)| = \left|\int_a^b \frac{\omega_r(x) f^{(r+1)}(\xi)}{(r+1)!}dx\right| \leq \frac{(b-a)}{(r+1)!}\left(\int_a^b |\omega_r(x)|dx\right) \max_{x\in[a,b]} |f^{(r+1)}(x)|. \tag{3.43}$$

Since $x_{n+1}, \ldots, x_r$ can be chosen arbitrarily, we select the combination that minimizes

$$\left(\int_a^b |\omega_r(x)|dx\right),$$

which will lead to our conclusion using a simple scaling. □

Remark 3.3.5. *As we can see, the error of the interpolation-based quadrature has a form similar to that of the interpolation polynomial error. This implies the Runge phenomenon would occur as well. The integration of the Runge function will not converge on uniformly distributed nodes.*

One way to overcome the issue of the Runge phenomenon is to perform piecewise integration. Suppose $[a, b]$ is divided into N subintervals of equal sizes, each of which has length $H = \frac{b-a}{N}$. Then the quadrature error in each subinterval would be:

$$\Omega_r \frac{H^{r+2}}{(r+1)!} \max_{x\in[a,b]} |f^{(r+1)}(x)|$$

where Ω_r is independent of the interval length. Therefore the total quadrature error would be bounded by

$$N\Omega_r \frac{H^{r+2}}{(r+1)!} \max_{x\in[a,b]} |f^{(r+1)}(x)| = (b-a)\Omega_r \frac{H^{r+1}}{(r+1)!} \max_{x\in[a,b]} |f^{(r+1)}(x)| = \mathcal{O}((b-a)H^{r+1}).$$

Example 3.3.6 (rectangle rule)**.** *For rectangle rule, $r = 1, n = 0$, therefore*

$$\Omega_r = \min_{t_1} \int_0^1 |t - \frac{1}{2}||t - t_1|dt = \frac{1}{12}, \quad (t_1 = \frac{1}{2}). \tag{3.44}$$

This is easiest to notice by changing variable $s = t - \frac{1}{2}$ and the symmetry, then the integral is just

$$\Omega_r = \min_{z\in[-1/2,1/2]} \int_{-1/2}^{1/2} |s||s - z|ds = \min_{z\in[0,1/2]} \int_0^{1/2} s(|s - z| + |s + z|)ds \geq \int_0^{1/2} s(2s)ds.$$

Example 3.3.7 (trapezoid rule)**.** *For trapezoid rule, $r = n = 1$, therefore*

$$\Omega_r = \int_0^1 (1 - t)tdt = \frac{1}{6}. \tag{3.45}$$

then the error is bounded by $\frac{(b-a)h^2}{12} \max_{x\in[a,b]} |f^{(r+1)}(x)|$.

3.3.3 Newton-Cotes Formula

The Newton-Cotes formula is a special interpolation-based quadrature rule. The nodes are equally spaced. The rectangle and trapezoid rules are just the two simplest cases. We define

1. closed form, $x_0 = a$, $x_n = b$, $x_j = a + jh$, $h = \frac{b-a}{h}$, $n \geq 1$.
2. open form, $x_0 = a + h$, $x_n = b - h$, $h = \frac{b-a}{n+2}$, $n \geq 0$.

The difference is whether the endpoints are included or not. Using the previous result, we can compute the quadrature weights by

$$\begin{aligned} w_j &= \int_0^1 \prod_{k=0,k\neq j}^{n} \frac{t - t_k}{t_j - t_k} dt = \int_0^1 \prod_{k=0,k\neq j}^{n} \frac{nt - k}{j - k} dt \\ &= \frac{1}{n} \int_0^n \prod_{k=0,k\neq j}^{n} \frac{s - k}{j - k} ds. \end{aligned} \tag{3.46}$$

The computations of the weights can be efficient by noticing the symmetry.

Lemma 3.3.8. $w_j = w_{n-j}$.

Proof. This is because $w_j = \int_a^b L_j(x)dx = \int_a^b L_j(a + b - x)dx = \int_a^b L_{n-j}(x)dx = w_{n-j}$. □

The weights w_j are only relevant to n and j. In practice, these values are tabulated *a priori*. When $n \geq 2$, the weights include negative terms for open forms, and when $n \geq 8$, the weights include negative terms for closed forms, which could introduce numerical instability from rounding errors. Therefore one should only limit to small values of n.

Remark 3.3.9. *We can derive the error estimate for the Newton-Cotes formula using the result from the previous section.*

$$\begin{aligned} |\mathcal{I}(f) - \mathcal{I}_{n,NC}(f)| &\leq \Omega_r \frac{(b-a)^{r+2}}{(r+1)!} \\ &= M_n h^{r+2} \max_{x\in[a,b]} |f^{(r+1)}(x)| \end{aligned} \tag{3.47}$$

where $M_n = \Omega_r n^{r+2} \frac{1}{(r+1)!}$, *see the following table for a reference.*

Table 3.1: A few examples for closed form

n	w_j	r	M_n
1	$(1/2, 1/2)$	1	$1/12$
2	$(1/6, 2/3, 1/6)$	3	$1/90$
3	$(1/8, 3/8, 3/8, 1/8)$	3	$3/80$
4	$(7/90, 32/90, 12/90, 32/90, 7/90)$	5	$8/945$

From the above table, we notice that when n is even, the exactness is $r = n+1$ for closed forms. This is a general statement.

Lemma 3.3.10. *For $n \geq 2$ even, the closed forms of the Newton-Cotes formula have an accuracy of degree $r = n + 1$.*

Proof. First, we know that $r \geq n$. Consider any polynomial of degree $n+1$,

$$p(x) = \sum_{j=0}^{n+1} b_j x^j, \tag{3.48}$$

we can rewrite the polynomial by

$$p(x) = b_{n+1}(x - \frac{a+b}{2})^{n+1} + \sum_{j=0}^{n} b'_j x^j \tag{3.49}$$

with another set of coefficients b'_j. The first term will have the integral as zero on the interval $[a, b]$. Numerically, using the Newton-Cotes formula,

$$\mathcal{I}_{n,NC}((x - \frac{a+b}{2})^{n+1}) = (b-a)\sum_{j=0}^{n} w_j (x_j - \frac{a+b}{2})^{n+1} \tag{3.50}$$

while $x_{n-j} - \frac{a+b}{2} = -(x_j - \frac{a+b}{2})$ and $w_j = w_{n-j}$, we can cancel all terms.
It still remains to show that $\mathcal{I}(x^{n+2}) \neq \mathcal{I}_{n,NC}(x^{n+2})$. Because the $r = (n+1)$ degree of accuracy is achievable, we borrow the previous estimate result, let $f(x) = x^{n+2}$,

$$\begin{aligned} \mathcal{I}(f) - \mathcal{I}_n(f) &= \int_a^b \frac{\omega_{n+1}(x) f^{(n+2)}(\xi)}{(n+2)!} dx = \int_a^b \omega_{n+1}(x) dx \\ &= \int_a^b \omega_n(x)(x - x_{n+1}) dx = F(x)(x - x_{n+1})|_a^b - \int_a^b F(x) dx \end{aligned} \tag{3.51}$$

where $F(x)$ is defined by

$$F(x) := \int_a^x \omega_n(t) dt.$$

Then it is simple to derive that $F(a) = F(b) = 0$ using the symmetry. Now we only have to show that

$$\int_a^b F(x) dx \neq 0.$$

We can show a stronger claim: $F(x) > 0$ over (a, b). This is left as an exercise (see Exercise 3.6.4). □

However, the Newton-Cotes formula definitely will fail when evaluating the integral of Runge function $f(x) = \frac{1}{1+x^2}$ on the interval $[-5, 5]$. It is more practical to combine the piecewise integral technique, which is called the composite Newton-Cotes formula. In the following, we discretize the interval $[a, b]$ into m subintervals of the same size $H = \frac{b-a}{m}$, then on each subinterval, we apply the Newton-Cotes formula (say closed form) with $(n+1)$ equally spaced nodes. Then the numerical integral would have an error bounded by

$$mM_n \left(\frac{H}{n}\right)^{r+2} \max_{x\in[a,b]} |f^{(r+1)}(x)| = (b-a)\frac{M_n}{n}\left(\frac{H}{n}\right)^{r+1} \max_{x\in[a,b]} |f^{(r+1)}(x)| \tag{3.52}$$

where $r = n$ for odd n and $r = n+1$ for even n, see the previous section for a quick derivation.

Example 3.3.11 (composite trapezoid rule)**.** *The composite trapezoid rule is often used for practical integration especially when f is periodic. Let $x_j = a + jH$, $j = 0, \ldots, m$,*

$$T(f, H) = \frac{H}{2}\left(f(a) + 2\sum_{j=1}^{m-1} f(x_j) + f(b)\right).$$

Its error then can be estimated by

$$\frac{1}{12}(b-a)H^2 \max_{x\in[a,b]} |f''(x)| = \mathcal{O}((b-a)H^2). \tag{3.53}$$

In the next step, we take a more careful look at the composite trapezoid rule. Recall the asymptotic Euler-Maclaurin summation formula:

$$\sum_{j=0}^{m} g(j) \sim \int_0^m g(x)dx + \frac{g(0)+g(m)}{2} + \sum_{k=1}^{\infty} \frac{B_{2k}}{(2k)!}(g^{2k-1}(m) - g^{(2k-1)}(0)) \tag{3.54}$$

If we take $g(j) = f(a + jH)$, then we will arrive at

$$T(f, H) \sim \int_a^b f(x)dx + \sum_{k=1}^{\infty} \frac{B_{2k}}{(2k)!} H^{2k}\left(f^{(2k-1)}(b) - f^{(2k-1)}(a)\right) \tag{3.55}$$

which means there is an asymptotic expansion in the form of

$$T(f, h) = \int_a^b f(x)dx + c_2H^2 + c_4H^4 + \cdots. \tag{3.56}$$

Particularly, for a smooth periodic function, the Euler-Maclaurin summation *formally* shows the numerical error is less than any polynomial of H.

3.3.4 Romberg Integration

The composite trapezoid rule's asymptotic expansion (3.56) implies a Richardson extrapolation combination to accelerate the evaluation. The Romberg integration refers to the following scheme:

1. Compute the sequence $a_{l,0} = T(f, (b-a)/2^l)$, $l = 0, \ldots, L$, for the standard composite trapezoid rule with different sub-interval sizes.

2. Extrapolation by

$$a_{l,q+1} = \frac{4^{q+1}a_{l,q} - a_{l-1,q}}{4^{q+1} - 1}, \quad q = 0, \ldots, L-1 \text{ and } l = q+1, \ldots, L$$

3. Output $a_{L,L}$, which should have an error of $\mathcal{O}(H^{2L+2})$, $H = (b-a)/2^L$.

Remark 3.3.12. *One of the advantages of the Romberg method is the reuse of the nodes. This is extremely helpful when evaluating f is not cheap. The extrapolation process also builds a new quadrature formulation implicitly. This quadrature rule gives the error $\mathcal{O}(n^{-2})$ to $\mathcal{O}(n^{-2\log_2 n-2})$, where n is the total number of nodes. Although the computational time increases a few times, the return seems worth it when f is sufficiently smooth.*

3.3.5 Adaptive Integrations

Numerically, we can apply any composite quadrature rule to a successive partition of $[a, b]$ until the estimated error is within tolerance. Let $A(f, H)$ denote any composite quadrature rule (e.g. Newton-Cotes), $H = (b - a)/m$, then

$$A(f, H) = \int_a^b f(x)dx + \mathcal{O}(H^{r+1}) \tag{3.57}$$

where r is the degree of accuracy. Then $A(f, H/2)$ will presumably introduce an error of about $2^{-(r+1)}$ times the size of the previous case. Therefore, we obtain a rough estimate of the error by

$$\mathcal{E} \approx \left| \frac{A(f, H) - A(f, H/2)}{1 - 2^{-(r+1)}} \right|$$

One can successively halve H until the estimated error is less than tolerance. However, such a method is not efficient when the quadrature on most of the subintervals is already very accurate. In this case, the best strategy is to keep those accurate subintervals and only partition the rest. This process will produce a non-uniform distribution of sub-intervals.

There are several ways to implement this. The simplest recursive algorithm can be roughly described as follows. Let $A(f, \alpha, \beta)$ be any quadrature rule on $[\alpha, \beta]$ with degree of accuracy r and $\mathcal{E}(f, \alpha, \beta)$ be an estimate of error for $|A(f, \alpha, \beta) - \int_\alpha^\beta f(x)dx|$. Then

1. Initially, $\alpha = a$, $\beta = b$, ε is the tolerance, $\mathcal{I} = 0$.
2. If $|\mathcal{E}(f, \alpha, \beta)| \leq \varepsilon \frac{\beta-\alpha}{b-a}$ or $|\beta - \alpha|$ is too small, $\mathcal{I} = \mathcal{I} + A(f, \alpha, \beta)$, stop. Otherwise, go to Step 3.
3. Divide $[\alpha, \beta]$ into $[\alpha, \gamma]$, $[\gamma, \beta]$, $\gamma = \frac{\alpha+\beta}{2}$.
   (a) For the first half, let $\alpha = \alpha$, $\beta = \gamma$, go to step 2.
   (b) For the second half, let $\alpha = \gamma$, $\beta = \beta$, go to step 2.

Ideally, the automatic partition will generate nonuniformly distributed subintervals. The total estimated numerical error will be bounded by ε.

A slightly more economical plan is stated in section 9.7 of the book Quarteroni et al. (2010), which focuses on the left-most sub-interval until its error bound is under some tolerance, then eliminates the chosen subinterval and restarts with the rest.

3.3.6 Improper Integral

The jump discontinuity is in general simple to deal with. We will focus on the unbounded functions, for example, $\log x$, x^γ with $0 > \gamma > -1$. These singularities are integrable. Let us take the following function for an example:

$$f(x) = \phi(x)(x-a)^\gamma, \quad x \in [a,b].$$

where $\phi(x)$ is a smooth function. Using integration by parts,

$$\begin{aligned}\int_a^b \phi(x)(x-a)^\gamma dx &= \frac{(x-a)^{\gamma+1}}{\gamma+1}\phi(x)\Big|_a^b - \int_a^b \phi'(x)\frac{(x-a)^{\gamma+1}}{\gamma+1}dx \\ &= \frac{(x-a)^{\gamma+1}}{\gamma+1}\phi(x)\Big|_a^b - \frac{(x-a)^{\gamma+2}}{(\gamma+1)(\gamma+2)}\phi'(x)\Big|_a^b + \cdots \\ &+ \int_a^b \phi^{(p)}(x)\frac{(-1)^p(x-a)^{\gamma+p}}{(\gamma+1)(\gamma+2)\cdots(\gamma+p)}dx,\end{aligned} \tag{3.58}$$

where the last integral is sufficiently regular and can be evaluated by quadrature rules such as Newton-Cotes formula, the error can be estimated using the result developed in previous sections. The first p terms are explicitly known. There are other choices such as a series solution, which is

$$\int_a^b \frac{\phi(x)}{(x-a)^\gamma}dx = \int_a^b \sum_{k=0}^{p} \frac{\phi^{(k)}(a)(x-a)^{k-\gamma}}{k!}dx + \int_a^b \frac{\phi^{(p+1)}(\xi)(x-a)^{p+1-\gamma}}{(p+1)!}dx \tag{3.59}$$

The remainder determines the convergence of the series. As long as the reminder is decaying, one may use the extrapolation technique to find the integral without using too many iterations. Otherwise, the integral part has to be evaluated through quadrature rules.

Another typical method is to isolate the singularity

$$\int_a^b \frac{\phi(x)}{(x-a)^\gamma}dx = \int_a^{a+\varepsilon} \frac{\phi(x)}{(x-a)^\gamma}dx + \int_{a+\varepsilon}^b \frac{\phi(x)}{(x-a)^\gamma}dx, \tag{3.60}$$

where the integral around singularity might converge when ε is sufficiently small (less than the convergence radius of the Taylor series). The non-singular integral can be computed by the traditional methods, since the integrand grows fast when $\varepsilon \to 0$, it is more suitable to apply adaptive methods.

Another class of improper integral is with the infinite domain. In general, this problem is difficult (e.g. oscillatory integral), we only discuss the simplest case right now. Let $f(x)$ be integrable over $[a,\infty)$ and assume there exists $s > 0$ such that

$$\lim_{x\to\infty} x^{1+s} f(x) = 0. \tag{3.61}$$

The most intuitive way is to truncate the interval $[a,\infty)$ to $[a,c]$ such that the integral on $[c,\infty)$ is negligible. The point c sometimes can be inferred from *a priori* estimate, sometimes has to

be found dynamically. The other methods are more or less playing around with the change of variable to make the interval "finite" (e.g. $x \mapsto x^{-\beta}$ when $x \geq c > 0$).

Other advanced tools (e.g. complex analysis) and examples will be discussed in a later topic chapter and integral equation chapter.

3.4 Gauss Quadrature

The Gauss quadrature maximizes the exactness of quadrature rules. Let $x_0, \ldots, x_n$ the nodes on $[-1, 1]$, in the following we will discuss the numerical quadrature for the weighted integral

$$\mathcal{I}_w(f) = \int_{-1}^{1} w(x)f(x)dx \simeq \mathcal{I}_{n,w}(f) := \sum_{j=0}^{n} c_j f(x_j),$$

where the coefficients are determined later. We first review the preliminaries for the theory behind the Gauss quadrature.

3.4.1 Orthogonal Polynomials

The orthogonal polynomials can be regarded as a special case of generalized Fourier series. Let the weight function $w(x) \geq 0$ on the interval $(-1, 1)$ be an integrable function. Then we can define a sequence of polynomials $p_k \in \Pi_k$, that is, deg$(p_k) = k$ and

$$\int_{-1}^{1} w(x)p_k(x)p_j(x)dx = 0, \quad \text{if } k \neq j. \tag{3.62}$$

Here, for convenience, we define the inner product $\langle f, g\rangle_w$ by

$$\langle f, g\rangle_w = \int_{-1}^{1} w(x)f(x)g(x)dx. \tag{3.63}$$

This inner product induces a norm $\|f\|_w = \sqrt{\langle f, f\rangle_w}$, we then define the corresponding space as

$$L_w^2 = \{f : (-1, 1) \to \mathbb{R} \mid \|f\|_w < \infty\}. \tag{3.64}$$

Then for any $f \in L_w^2$, we can define the generalized Fourier series by Sf:

$$Sf = \sum_{j=0}^{\infty} a_j p_j, \quad a_j = \frac{\langle f, p_j\rangle_w}{\langle p_j, p_j\rangle_w}. \tag{3.65}$$

The series converges to f in L_w^2 sense from the Parseval's equality:

$$\|f\|_w^2 = \sum_{j=0}^{\infty} a_j^2 \|p_j\|_w^2. \tag{3.66}$$

The truncated series $f_n = \sum_{j=0}^{n} a_j p_j$ is the best degree-n polynomial approximation to f, that is,

$$\|f - f_n\|_w = \min_{q \in \Pi_n} \|f - q\|_w,$$

and the polynomial f_n is the orthogonal projection of f onto Π_n in the sense of L^2_w.

Theorem 3.4.1. *The following recursive formula generates (unnormalized) orthogonal polynomials $p_j \in \Pi_j$.*

$$p_{j+1} = (x - \alpha_j)p_j(x) - \beta_j p_{j-1}(x), \quad j \geq 0. \tag{3.67}$$

The initial conditions are $p_{-1} = 0$, $p_0 = 1$. The constants α_j and β_j are

$$\begin{aligned} \langle p_{j+1}, p_j \rangle_w = 0 \Rightarrow \alpha_j &= \frac{\langle x p_j, p_j \rangle_w}{\langle p_j, p_j \rangle_w}. \\ \langle p_{j+1}, p_{j-1} \rangle_w = 0 \Rightarrow \beta_j &= \frac{\langle p_j, p_j \rangle_w}{\langle p_{j-1}, p_{j-1} \rangle_w}. \end{aligned} \tag{3.68}$$

Proof. The proof is straightforward by induction. Notice that p_{j+1} defined in this formula will be automatically orthogonal to $\{p_k\}_{k=0}^{j-2}$, thus only have to determine the parameters α_j and β_j to fulfill the orthogonality with p_{j-1} and p_j. □

Chebyshev Polynomials

The Chebyshev polynomials are generated by using the weight $w(x) = (1 - x^2)^{-1/2}$ on $(-1, 1)$. The corresponding space is

$$L^2_w = \{f : (-1, 1) \to \mathbb{R} \mid \int_{-1}^{1} f^2(x)(1 - x^2)^{-1/2} dx < \infty\}.$$

It is clear that by setting $p_k(x) = \cos(k \arccos x)$, the integral

$$\int_{-1}^{1} p_k(x) p_j(x)(1 - x^2)^{-1/2} dx = \int_0^{\pi} \cos(k\theta)\cos(j\theta) d\theta = \begin{cases} \pi & k = j = 0 \\ \frac{\pi}{2} & k = j \neq 0 \\ 0 & \text{otherwise} \end{cases} \tag{3.69}$$

Therefore, the generalized Fourier series with Chebyshev polynomials is

$$f(x) = \sum_{j=0}^{\infty} a_j p_j(x)$$

where $p_j(x) = T_j(x)$ the j-th Chebyshev polynomial and $a_j = \frac{1}{\pi}\langle f, p_j \rangle_w$ if $j = 0$ and $a_j = \frac{2}{\pi}\langle f, p_j \rangle_w$ otherwise.

Legendre Polynomials

The Legendre polynomials are generated using the weight $w(x) = 1$. The corresponding space is normal $L^2(-1, 1)$. The recursive formula for Legendre polynomials is

$$L_{j+1}(x) = \frac{2j+1}{j+1} x L_j(x) - \frac{j}{j+1} L_{j-1}(x) \tag{3.70}$$

and $\langle L_j, L_j \rangle_w = \frac{2}{2j+1}$. Therefore, the generalized Fourier series is

$$f(x) = \sum_{j=0}^{\infty} a_j L_j(x), \quad a_j = \frac{2j+1}{2} \langle f, L_j \rangle_w. \tag{3.71}$$

The first few Legendre polynomials are

$$L_0 = 1, \quad L_1(x) = x, \quad L_2(x) = \frac{1}{2}(3x^2 - 1). \tag{3.72}$$

Remark 3.4.2. *Both of the above examples are special cases of Jacobi polynomials which are generated by weight function* $w(x) = (1-x)^\alpha (1+x)^\beta$.

3.4.2 The Riemann-Hilbert Problem

TODO

3.4.3 Gauss Quadrature on Bounded Domain

We borrow the same accuracy concept from the previous chapter ($w(x) = 1$). It is clear that with $n+1$ nodes, the highest possible accuracy is at least degree n, in the later derivation we will see that the Gauss quadrature can have an accuracy of $r = n + m$ for certain $m > 0$.

Theorem 3.4.3. *Let* $p_k \in \Pi_k$ *and* $\varpi(x) = \prod_{j=0}^{n}(x - x_j)$, *then*

$$\langle \varpi(x), p_k \rangle_w = 0, \quad 0 \le k \le m-1$$

if and only if the associated quadrature rule has an accuracy at the order of $n + m$.

Proof. The key idea is to represent any polynomial q of Π_{n+m} by

$$q(x) = \varpi(x) s(x) + t(x) \tag{3.73}$$

where $s(x) \in \Pi_{m-1}$ and $t \in \Pi_n$. Therefore the quadrature over the reminder term $t(x)$ is exact,

$$\sum_{j=0}^{n} c_j t(x_j) = \int_{-1}^{1} t(x) w(x) dx = \int_{-1}^{1} q(x) w(x) dx - \underline{\int_{-1}^{1} \varpi(x) s(x) w(x) dx}. \tag{3.74}$$

□

It is clear that the maximum of $m \leq n+1$, otherwise, we can choose $s(x) = \varpi(x)$, then $\langle \varpi(x), \varpi(x) \rangle_w > 0$ violates the above theorem. This leads to the following corollary for the Gauss quadrature.

Corollary 3.4.4. *The quadrature rule*

$$\mathcal{I}_{n,w}(f) = \sum_{j=0}^{n} c_j f(x_j)$$

has maximum accuracy of degree $2n+1$.

The next question would be whether the maximum $2n+1$ is achievable. This requires that

$$\int_{-1}^{1} \varpi(x) p_k(x) w(x) dx = 0, \quad 0 \leq k \leq n. \tag{3.75}$$

This formula indicates that $\langle \varpi(x), p_k \rangle_w = 0$ for any $p_k \in \Pi_k$, $0 \leq k \leq n$.

Theorem 3.4.5. *Let* $\{p_k\}_{k \geq 0}$ *be a sequence of orthogonal polynomials, then the only possible choice of* ϖ *satisfying* (3.75) *is* p_{n+1} *(up to scaling).*

Proof. Without loss of generality, we assume p_{n+1}'s leading power's coefficient is one (for example, the recursive formula (3.67)). Since p_{n+1} also satisfies the orthogonal relation. Therefore,

$$\int_{-1}^{1} (\varpi(x) - p_{n+1}(x)) s(x) w(x) dx = 0, \quad s \in \Pi_n \tag{3.76}$$

while $\varpi - p_{n+1} \in \Pi_n$, then we have a contradiction if we take $s(x) = \varpi - p_{n+1} \neq 0$. □

The above theorem implies that the nodes x_j are the zeros of p_{n+1} (we still need to show that they are simple roots). The corresponding quadrature rule is called the Gauss quadrature, and the accuracy is of degree $2n+1$.

Remark 3.4.6. *For* $w = 1$*, the Legendre polynomial's roots are not at the endpoints, while sometimes the endpoints* ± 1 *are useful to be included in the quadrature nodes, therefore we may want to generate a similar Gauss quadrature with the end nodes as well. Following the same idea, in order to make* ± 1 *as the roots of* $\varpi(x)$ *but also keep* ϖ *concentrated on* p_k *for large* k*, we can define*

$$\widetilde{\varpi}(x) := p_{n+1}(x) + A p_n(x) + B p_{n-1}(x), \tag{3.77}$$

the constants A, B *are used to control* $\widetilde{\varpi}(\pm 1) = 0$*. The corresponding* $\widetilde{\varpi}$ *has a similar property as* ϖ *but only provides an accuracy of degree* $2n-1$*. The nodes are roots of* $\widetilde{\varpi}$*, called Gauss-Lobatto nodes. In the following, we prove some of the properties of Gauss quadrature.*

Theorem 3.4.7. *All roots* x_j *of* p_{n+1} *are real and distinct.*

Proof. The polynomial p_{n+1} must have $n+1$ real roots. Otherwise, one can generate a polynomial $q(x)$ with degree $< (n+1)$ that $\langle q, p_{n+1}\rangle > 0$. Assume some of the roots are not simple. Pick out all roots with odd multiplicity, say $z_0 < z_1 < \cdots < z_M$, then

$$q(x) = (x - z_0)\cdots(x - z_M) \tag{3.78}$$

should satisfy $q(x)p_{n+1}(x)w(x) > 0$ except for the roots. However, if $M \neq n$, we would have an issue since

$$\int_{-1}^{1} q(x)p_{n+1}(x)w(x)dx = 0. \tag{3.79}$$

□

Theorem 3.4.8. *The weights c_j for Gauss quadrature rules*

$$\mathcal{I}_n(f) = \sum_{j=0}^{n} c_j f(x_j) \tag{3.80}$$

are all positive.

Proof. The polynomials p_k satisfy the recursive formula

$$p_{k+1}(x) = (x - \alpha_k)p_k(x) - \beta_k p_{k-1}(x)$$

then

$$\begin{pmatrix} \alpha_0 & 1 & 0 & 0 & \cdots & 0 \\ \beta_1 & \alpha_1 & 1 & 0 & \ddots & 0 \\ 0 & \beta_2 & \alpha_2 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \beta_{n-1} & \alpha_{n-1} & 1 \\ 0 & 0 & 0 & 0 & \beta_n & \alpha_n \end{pmatrix} \begin{pmatrix} p_0(x_l) \\ p_1(x_l) \\ p_2(x_l) \\ \vdots \\ p_{n-1}(x_l) \\ p_n(x_l) \end{pmatrix} = x_l \begin{pmatrix} p_0(x_l) \\ p_1(x_l) \\ p_2(x_l) \\ \vdots \\ p_{n-1}(x_l) \\ p_n(x_l) \end{pmatrix} \tag{3.81}$$

which means the matrix A on the left side has $(n+1)$ eigenpairs $\{x_l, (p_j(x_l))_{j=0}^n\}$. The tridiagonal matrix has a similar transform to a symmetric matrix, denoted by $S = D^{-1}AD$ with D as a diagonal matrix; then S has the same set of eigen pairs, which means that the transformed vectors

$$D^{-1} \begin{pmatrix} p_0(x_l) \\ p_1(x_l) \\ p_2(x_l) \\ \vdots \\ p_{n-1}(x_l) \\ p_n(x_l) \end{pmatrix} \tag{3.82}$$

are the eigenvectors of S. Since all the eigenvalues are distinct, these vectors are orthogonal, which implies

$$v_k^T D^{-2} v_j = r_j \delta_{jk} \quad r_j > 0. \tag{3.83}$$

where $v_j = (p_i(x_j))_{i=0}^n$. Take $f = p_k$ in the quadrature rule,

$$\sum_{j=0}^{n} c_j v_{jk} = \delta_{0k} \tag{3.84}$$

Combined with the two equations, we have

$$\begin{aligned}\sum_{k=0}^{n}\sum_{j=0}^{n} c_j v_{jk} v_{lk} D_{kk}^{-2} &= \sum_{k=0}^{n} v_{lk} D_{kk}^{-2} \sum_{j=0}^{n} c_j v_{jk} = D_{00}^{-2} > 0 \\ &= \sum_{j=0}^{n} c_j \sum_{k=0}^{n} v_{lk} D_{kk}^{-2} v_{jk} = r_l c_l.\end{aligned} \tag{3.85}$$

□

This result should be compared with the Newton-Cotes formula, where the weights are not all positive if n is large; hence, the Gauss quadrature has better numerical stability. Finally, we briefly state the error estimate for the Gauss quadrature using the previously proved theorem 3.3.4, where $r = 2n + 1$.

Corollary 3.4.9. *For $f \in C^{2n+2}[-1, 1]$, the error of Gauss quadrature satisfies*

$$|\mathcal{I}(f) - \mathcal{I}_n(f)| \le \Omega_{2n+1} \frac{(b-a)^{2n+3}}{(2n+2)!} \max_{x\in[-1,1]} |f^{(2n+2)}(x)|. \tag{3.86}$$

Remark 3.4.10. *For Chebyshev polynomials. the weight $w(x) = (1 - x^2)^{-1/2}$, the Gauss quadrature nodes are roots of $T_{n+1}(x) = \cos((n + 1) \arccos x)$,*

$$x_j = \cos\left(\frac{2j+1}{2(n+1)}\pi\right), \quad c_j = \frac{\pi}{n+1}.$$

which is exactly the set of Chebyshev interpolation nodes. The Gauss-Lobatto nodes are

$$\tilde{x}_j = \cos\left(\frac{j\pi}{n}\right), \quad c_j = \frac{\pi}{d_j n}$$

where $d_j = 2$ if $j = 0$ or $j = n$, otherwise $d_j = 1$. This is exactly the composite trapezoid rule.

3.4.4 Gauss Quadrature on Unbounded Domain

For the integral over an infinite interval $[0, \infty)$ or $(-\infty, \infty)$ using the Gauss quadrature, the weight needs to be decaying faster than polynomial growth. Usual choices are $w(x) = e^{-x}$ and $w(x) = e^{-x^2}$. The corresponding orthogonal polynomials are called Laguerre polynomials and Hermite polynomials (not the interpolation polynomials), respectively. The derivations will use Rodrigues' formula, closely related to the Sturm Liouville theory (see Section 5.3.1).

Theorem 3.4.11 (Rodrigues' formula). *Let $\{p_n\}_{n\geq 0}$ be the sequence of orthogonal polynomials with weight $w(x)$ on $[a,b]$. If the weight function w solves*

$$\frac{w'(x)}{w(x)} = \frac{f(x)}{g(x)} \tag{3.87}$$

such that $f \in \Pi_1$ and $g \in \Pi_2$ with limits

$$\lim_{x\to a} w(x)g(x) = 0, \quad \lim_{x\to b} w(x)g(x) = 0, \tag{3.88}$$

then $p_n(x) = \frac{c_n}{w(x)}\frac{d^n}{dx^n}\left(g(x)^n w(x)\right)$, where c_n is a normalization constant.

Proof. It is simple to derive $p_n \in \Pi_n$ by induction. We only provide the idea to prove the orthogonality. If $m < n$, then

$$\int_a^b x^m p_n(x)w(x)dx = 0. \tag{3.89}$$

Using integration by parts, assuming that $\lim_{x\to a \text{ or } b} \frac{d^r}{dx^r}\left[(g(x))^n w(x)\right] = 0$ for $r < n$ (see Exercise), then

$$\begin{aligned}
\int_a^b x^m p_n(x)w(x)dx &= c_n \int_a^b x^m \frac{d^n}{dx^n}\left[(g(x))^n w(x)\right] dx \\
&= c_n x^m \frac{d^{n-1}}{dx^{n-1}}\left[(g(x))^n w(x)\right]\Big|_a^b - c_n m \int_a^b x^{m-1}\frac{d^{n-1}}{dx^{n-1}}\left[(g(x))^n w(x)\right] dx \\
&= \cdots \\
&= (-1)^m m! \int_a^b \frac{d^{n-m}}{dx^{n-m}}\left[(g(x))^n w(x)\right] dx = 0.
\end{aligned}$$

□

With $w(x) = 1$ and $g(x) = 1 - x^2$, we obtain Legendre polynomials. Laguerre and Hermite polynomials are derived using $w(x) = e^{-x}$, $g(x) = x$ and $w(x) = e^{-x^2}$, $g(x) = 1$, respectively.

Laguerre polynomial

The domain is $[0,\infty)$ and the *normalized* Laguerre polynomials are defined by

$$L_n(x) = \frac{e^x}{n!}\frac{d^n}{dx^n}(e^{-x}x^n), \quad n \geq 0 \tag{3.90}$$

It can be shown easily using integration by parts or using Rodrigues' formula that $\langle L_n, L_m\rangle_w = 0$ if $n \neq m$. The polynomials satisfy the recursive formula

$$L_{n+1}(x) = \frac{1}{n+1}\left((2n+1-x)L_n(x) - nL_{n-1}(x)\right). \tag{3.91}$$

The initial values are $L_{-1} = 0$ and $L_0 = 1$ as usual.

Lemma 3.4.12. *The roots of the Laguerre polynomial p_n are on $(0, n^2)$.*

Proof. The closed form of the Laguerre polynomial is (by induction, see Exercise 3.6.7)

$$L_n(x) = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{k!} x^k, \tag{3.92}$$

whose coefficients are having alternative signs, thus, if $x < 0$, $|L_n(x)| > 0$. Denote the polynomial

$$Q_n(z) := L_n\left(-n^2 z\right) = \sum_{k=0}^{n} a_k z^k, \quad a_k := \binom{n}{k} \frac{1}{k!} n^{2k}, \tag{3.93}$$

has all of the coefficients positive and the coefficients are monotone increasing since

$$\frac{a_k}{a_{k+1}} = \frac{\binom{n}{k}}{k!} \frac{(k+1)!}{\binom{n}{k+1}} \frac{1}{n^2} = \frac{(k+1)^2}{n^2} \frac{1}{n-k} \leq 1, \quad 0 \leq k \leq n-1. \tag{3.94}$$

Therefore if $|z| > 1$ is a root for Q_n, it must satisfy

$$Q_n(z)(1-z) = a_0 - a_n z^{n+1} + \sum_{k=1}^{n} (a_k - a_{k-1}) z^k = 0.$$

Thus,

$$a_n |z|^{n+1} = \left| a_0 + \sum_{k=1}^{n} (a_k - a_{k-1}) z^k \right| \leq a_0 |z|^n + \sum_{k=1}^{n} (a_k - a_{k-1}) |z|^k = a_n |z|^n, \tag{3.95}$$

which is a contradiction. Thus all roots of Q_n are within the disk $\{z \in \mathbb{C} \mid |z| \leq 1\}$. This is the Eneström-Kakeya theorem.

On the other side, we show that $z = n^2$ is not a root of L_n. Since the inequality (3.94) is strict for $0 \leq k < n-1$, then by grouping the neighboring coefficients,

$$L_n(n^2) = \sum_{k=0}^{n} (-1)^k a_k \tag{3.96}$$

is negative for odd n and positive for even n. □

Remark 3.4.13. *A tighter bound for the roots is $(0, n + (n-1)\sqrt{n})$. An elementary proof of the weaker bound $(0, \sqrt{2n^3 - n^2})$ is left as an exercise (See Exercise 3.6.8).*

Hermite polynomial

The domain is $(-\infty, \infty)$ and the *unnormalized* Hermite polynomials are defined by

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2}. \tag{3.97}$$

The recursive formulation is

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x).$$

It is easy to check the following explicit formulation for Hermite polynomials.

Theorem 3.4.14. *The Hermite polynomials are*

$$H_n(x) = n! \sum_{k=0}^{\lfloor n/2 \rfloor} \frac{(-1)^k}{k!(n-2k)!} (2x)^{n-2k}. \tag{3.98}$$

Corollary 3.4.15. *The roots of H_n are on $[-\sqrt{2n-2}, \sqrt{2n-2}]$.*

Proof. Normalize the leading coefficients of the Hermite polynomials $\hat{H}_n(x) = 2^{-n} H_n(x)$, then the recursive formula becomes

$$\hat{H}_{n+1}(x) = x\hat{H}_n(x) - \frac{n}{2}\hat{H}_{n-1}(x). \tag{3.99}$$

Recall the Theorem 3.4.8, the roots of $\hat{H}_{n+1}(x)$ are the eigenvalues of the matrix

$$\mathcal{E}_{n+1} := \begin{pmatrix} 0 & 1 & 0 & 0 & \cdots & 0 \\ \frac{1}{2} & 0 & 1 & 0 & \ddots & 0 \\ 0 & \frac{2}{2} & 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \frac{(n-1)}{2} & 0 & 1 \\ 0 & 0 & 0 & 0 & \frac{n}{2} & 0 \end{pmatrix} \sim \begin{pmatrix} 0 & \sqrt{\frac{1}{2}} & 0 & 0 & \cdots & 0 \\ \sqrt{\frac{1}{2}} & 0 & \sqrt{\frac{2}{2}} & 0 & \ddots & 0 \\ 0 & \sqrt{\frac{2}{2}} & 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \sqrt{\frac{(n-1)}{2}} & 0 & \sqrt{\frac{n}{2}} \\ 0 & 0 & 0 & 0 & \sqrt{\frac{n}{2}} & 0 \end{pmatrix}, \tag{3.100}$$

where $\sim$ means similarity equivalence which does not change the spectrum. Then, by Gershgorin circle Theorem, $\rho(\mathcal{E}_{n+1}) \leq 2\sqrt{n}$. □

3.5 Probabilistic Integration

High dimensional integration often suffers from *the curse of dimensionality* for classical quadrature rules. Instead of a deterministic approach, the probabilistic integration employs a non-deterministic way to compute the integration in high dimensions.

3.5.1 Monte Carlo Integration

The Monte Carlo integration approximates the definite integral over the domain $D \subset \mathbb{R}^n$

$$\mathcal{I} := \int_D f(\mathbf{z}) d\mathbf{z} \tag{3.101}$$

by the summation over uniformly distributed sample points $\mathbf{x}_i \in D$, $i = 1, \cdots, m$,

$$\mathcal{I}_{\text{MC},m} = \frac{\text{Vol}(D)}{m} \sum_{i=1}^{m} f(\mathbf{x}_i) \approx \mathcal{I} = \text{Vol}(D) \mathbb{E}_{\mathbf{x} \sim \mathcal{U}(D)} [f] . \tag{3.102}$$

As the sample points are independent, the law of large numbers (LLN) shows that almost surely $\lim_{m\to\infty} \mathcal{I}_{\text{MC},m} = \mathcal{I}$. The error can be estimated by Hoeffding inequality,

$$\mathbb{P}\left[|\mathcal{I}_{\text{MC},m} - \mathcal{I}| \geq t\right] \leq 2 \exp\left(-2m \left|\frac{t}{\omega(f)\text{Vol}(D)}\right|^2\right) . \tag{3.103}$$

where $\omega(f) = \sup_D f - \inf_D f$. The inequality implies that the error is $\mathcal{O}_p(\omega(f)\text{Vol}(D)m^{-1/2})$. In practice, the random samples are generated by certain pseudo-random number generators which may lose their independence. However, if the covariance is relatively weak, then the weak law of large numbers could still apply.

Example 3.5.1. *If the sample point $\mathbf{x}_i \sim X_i$, where $\{X_i\}_{i\geq 1}$ are uniformly distributed dependent random variables on D and $\text{Cov}(X_i, X_j) = \theta_{ij}$ such that $\sum_{1\leq i<j\leq m} \theta_{ij} = \mathcal{O}(m)$ for some $\varepsilon > 0$, then by Chebyshev inequality,*

$$\begin{aligned}
\mathbb{P}\left[|\mathcal{I}_{MC,m} - \mathcal{I}| \geq t\right] &\leq \frac{1}{t^2} \textit{Var}(\mathcal{I}_{MC,m}) \\
&= \frac{|\textit{Vol}(D)|^2}{t^2 m^2} \left(m \textit{Var}_{\mathbf{x}\sim\mathcal{U}(D)}(f) + 2 \sum_{i<j} \textit{Cov}(X_i, X_j) \right) \\
&\leq \frac{|\textit{Vol}(D)\omega(f)|^2}{t^2 m} + 2t^{-2}m^{-2}|\textit{Vol}(D)|^2 \sum_{1\leq i<j\leq m} \theta_{ij} \\
&= \mathcal{O}(t^{-2}m^{-1}).
\end{aligned} \tag{3.104}$$

3.5.2 Quasi-Monte Carlo Integration

Ideally, the Monte Carlo integration will be more accurate when the sample points are placed "evenly". However, the Monte Carlo approach has an error of $\mathcal{O}_p(m^{-1/2})$ due to the randomness. To arrange the sample point "evenly", the quasi-Monte Carlo integration generates a sequence of points in a deterministic way and can achieve an error of $\mathcal{O}(\log^d m/m)$ for a good set of points and outperforms the Monte Carlo method.

Let us first carry out the definition of the $*$-discrepancy of a sequence $\{\mathbf{x}_i\}_{i\geq 1} \subset [0,1]^d$.

Definition 3.5.2. *The $*$-discrepancy of a sequence $\{\mathbf{x}_i\}_{i\geq 1} \subset [0,1]^d$ is*

$$D_m^* = \sup_{B=\prod_{i=1}^d [0,z_i]\subset[0,1]^d} \left| \frac{\textit{card}\left(\{\mathbf{x}_i\}_{i=1}^m \bigcap B\right)}{m} - \mu(B) \right|,$$

where μ is the usual Lebesgue measure.

The following theorem is classical, see Theorem 2.11 Niederreiter (1992), which shows the quadrature error is small as long as the $*$-discrepany is small.

Definition 3.5.3. *Define the variation of $f \in C^d([0,1]^d)$ as $V(f)$ in the sense of Hardy and Krause that*

$$V(f) = \sum_{k=1}^{d} \sum_{1\leq i_1<i_2<\cdots<i_k\leq d} V^{(k)}(f; i_1, \cdots, i_k), \tag{3.105}$$

and

$$V^{(k)}(f; i_1, \cdots, i_k) := \int_0^1 \cdots \int_0^1 \left| \frac{\partial^d f}{\partial x_{i_1} \cdots \partial x_{i_k}} \right| dx_{i_1} \cdots dx_{i_k}. \tag{3.106}$$

Theorem 3.5.4 (Koksma-Hlawka)**.** *If f has bounded variation $V(f)$ on $[0,1]^d$, then*

$$\left| \frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}_i) - \mathcal{I}(f) \right| \leq V(f) D_m^*. \tag{3.107}$$

Proof. We prove the case of $d = 1$. Let the guarding nodes $\mathbf{x}_0 = 0$ and $\mathbf{x}_{m+1} = 1$, then, use integration by parts,

$$\frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}_i) - \mathcal{I}(f) = \sum_{i=0}^{m} \int_{\mathbf{x}_i}^{\mathbf{x}_{i+1}} \left(\mathbf{x} - \frac{i}{m} \right) f'(\mathbf{x}) d\mathbf{x}, \tag{3.108}$$

Since $|\mathbf{x} - \frac{i}{m}| \leq D_m^*$ for $\mathbf{x} \in [\mathbf{x}_i, \mathbf{x}_{i+1}]$, the proof is complete. For higher dimensions, one can adapt a similar idea. Let π_k be the collection of all k subsets in from $\{1, 2, \cdots, d\}$. For a subset $\mathbf{u} = (i_1, \cdots, i_k) \in \pi_k$, we denote $z_{\mathbf{u}} = (z_{i_1}, \cdots, z_{i_k})$, $dz_{\mathbf{u}} = \prod_{j=1}^k dz_{i_j}$, and $(z_{\mathbf{u}}, \mathbf{1})$ is the point with i_jth coordinate as z_{i_j}, the rest are replaced by one. Then,

$$\frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}_i) - \mathcal{I}(f) = \sum_{k=1}^{d} \sum_{\mathbf{u}\in\pi_k} (-1)^k \int_{[0,1]^k} \text{disc}(z_{\mathbf{u}}, \mathbf{1}) \frac{\partial^k f(z_{\mathbf{u}}, \mathbf{1})}{\partial z_{\mathbf{u}}} dz_{\mathbf{u}}. \tag{3.109}$$

where $\text{disc}(z) = \frac{1}{m} \sum_{i=1}^m \chi_{\prod_{j=1}^d [0,z_j)}(\mathbf{x}_i) - \prod_{j=1}^d z_i$ is the discrepancy function. This is also known as the Hlawka-Zaremba identity. Note $\chi_{[0,z)}(x) = \chi_{(x,1]}(z)$. □

Definition 3.5.5. *If the sequence $\{\mathbf{x}_i\}_{i\geq 1}$ satisfies that $D_m^* \leq C \frac{|\ln m|^{\alpha_d}}{m}$ as $m \to \infty$ for certain $\alpha_d \geq 0$, then sequence is called a* low-discrepancy point set.

It is known that $\alpha_d = d - 1$ is achievable (e.g., Hammersley point set, (t, m, s)-nets) and there are lattice point sets with $\alpha_2 = 1$ and $\alpha_d = d$ for $d \geq 3$. See Chapter 4 and Chapter 5 of Niederreiter (1992).

Example 3.5.6 (Irrational rotation)**.**

Example 3.5.7 (van der Corput sequence)**.**

Example 3.5.8 (Sobol sequence)**.**

3.6 Exercises

The source code and test cases for the computational part are hosted on GitHub.

3.6.1 Theoretical Part

Problem 3.6.1. *Based on Theorem 3.2.4 and Theorem 3.2.5, derive the backward difference formula.*

Problem 3.6.2. *Estimate the error (with rounding error) for the following central difference scheme*

$$f'(x) \simeq \frac{-f(x+h) + 8f(x+h/2) - 8f(x-h/2) + f(x-h)}{6h}.$$

When $f(x) = \exp(x)$ on $[0, 1]$, what value would be a suitable choice for h?

Problem 3.6.3. *Let the integration $\mathcal{I}(f)$ given by*

$$\mathcal{I}(f) = \int_0^1 x^\alpha f(x)dx, \quad \alpha \in [0, 1]$$

and consider the quadrature formula $\mathcal{I}_0(f) = w_0 f(x_0)$. Is it possible to find an α that the quadrature rule has a degree $r = 2$ of accuracy?

Problem 3.6.4. *Let $n \geq 2$ be even and $\{x_j\}_{j=0}^n$ be the nodes for the Newton-Cotes formula in the closed form on $[a, b]$. Define $\omega(x) = \prod_{j=0}^n (x - x_j)$ and $F(x)$ defined as*

$$F(x) = \int_a^x \omega(t)dt.$$

Prove that $F(a) = F(b) = 0$ and $F(x) > 0$ over (a, b). Hint: Note that F is symmetric. First show that $|\omega(x)| > |\omega(x+h)|$ when $x \in (a, a+h)$, where $h = (b-a)/n$.

Problem 3.6.5. *Let $f(x) \in C(\mathbb{R})$ be a 2π-periodic function and f_n be the interpolating trigonometric polynomial on equally spaced nodes $x_j = \frac{2\pi j}{2n+1}$, $j = 0, \ldots, 2n$. Show that*

$$\mathcal{I}(f_n) = \sum_{j=0}^{2n} w_j f(x_j). \tag{3.110}$$

for positive weights $w_j > 0$, $j = 0, \ldots, 2n$. Hint: Show that the "Lagrange basis" is

$$l_j(x) = \frac{1}{2n+1} \frac{\sin((2n+1)(x - x_j)/2)}{\sin((x - x_j)/2)}.$$

Problem 3.6.6 (interlacing). *Let p_n be orthogonal polynomials with weight function $w(x) \geq 0$, and prove that the zeros of p_n and p_{n+1} are alternating.*

$$-1 < x_{1,n+1} < x_{1,n} < x_{2,n+1} < x_{2,n} < \cdots < x_{n,n+1} < x_{n,n} < x_{n+1,n+1} < 1. \tag{3.111}$$

where $x_{j,k}, 1 \leq j \leq k$ are the zeros of p_k. Hint: Use the recursive formula.

Problem 3.6.7. *Use the recursive definition to prove the Laguerre polynomials are*

$$p_n(x) = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{k!} x^k. \tag{3.112}$$

Problem 3.6.8. *Prove the roots of the Laguerre polynomial p_n are on $(0, \sqrt{2n^3 - n^2})$. Hint: Let the roots be $\{\zeta_i\}_{i=1}^n$ and use Vieta's formula to find $\sum_{i=1}^n \zeta_i^2$.*

Problem 3.6.9. *Prove the roots of Hermite polynomial H_n are on*

$$\left[-\sqrt{2n-2}\cos\left(\frac{\pi}{n+1}\right), \sqrt{2n-2}\cos\left(\frac{\pi}{n+1}\right)\right],$$

which improves the bound in Corollary 3.4.15.

Problem 3.6.10. *Prove the Christoffel–Darboux formula.*

$$\sum_{j=0}^{n} \frac{p_j(x)p_j(y)}{\gamma_j} = \frac{c_n}{c_{n+1}\gamma_{n+1}} \frac{p_{n+1}(x)p_n(y) - p_n(x)p_{n+1}(y)}{x - y},$$

where $\gamma_j = \langle p_j, p_j \rangle_w$ and c_j is the leading coefficient of p_j.

3.6.2 Computational Part

Problem 3.6.11. *Let the infinite matrix A such that*

$$A_{jk} = \frac{1}{(j+k-1)(j+k)/2 - (k-1)}.$$

Compute the operator norm $\|A\|$ with ten digits accuracy. Hint: use extrapolation.

Problem 3.6.12. *Implement the adaptive integration with composite Newton-Cotes closed formula for $n \leq 10$.*

Problem 3.6.13. *Implement the Gauss-Legendre quadrature* $n = 4$ *and compare the accuracy with Newton-Cotes closed formula* $n = 4$ *for* $f(x) = e^x$ *on* $[0, 1]$.

Problem 3.6.14. *Implement the Gauss-Laguerre quadrature for*

$$\mathcal{I}_{Laguerre} = \int_0^\infty e^{-x} \cos(x) dx \tag{3.113}$$

and compare with the quadrature error by Gauss-Legendre quadrature on a truncated domain

$$\mathcal{I}_{Legendre} = \int_0^{50} e^{-x} \cos(x) dx. \tag{3.114}$$

Explain your results.

Problem 3.6.15. *Implement the quasi-Monte Carlo method with irrational rotation sequence* $\sqrt{2}$ *and compare the quadrature error with the Monte Carlo integration for*

$$\mathcal{I} = \int_{[0,1]^2} e^{-(x^2+y^2)} dxdy = \frac{\pi}{4} |erf(1)|^2.$$

Extended Reading

Niederreiter, H. (1992). *Random number generation and quasi-Monte Carlo methods*. SIAM.

Quarteroni, A., Sacco, R., and Saleri, F. (2010). *Numerical mathematics*, volume 37. Springer Science & Business Media.

CHAPTER 4

APPROXIMATION

The approximation solves the problem

$$\min_{p \in P} \|f - p\|_{\mathcal{X}}$$

which aims to select the function $p \in P$ in a specific set with the minimum distance under a certain metric $\|\cdot\|_{\mathcal{X}}$ from the target function f.

4.1 General Approximation Theory

The most famous example in approximation theory is the least square problem

$$\min_{x \in S} \|Ax - b\|_2$$

where $A \in \mathbb{R}^{N \times k}$ and a given vector $b \in \mathbb{R}^N$. Seeking for solution $x \in S = \mathbb{R}^k$ is the simplest case. The problem can be efficiently solved if S is a convex set.

Definition 4.1.1. *Let $\mathcal{M} \subset \mathcal{V}$ of a normed space $(\mathcal{V}, \|\cdot\|)$ and given $v \in \mathcal{V}$, the best approximation in $\|\cdot\|$ is*

$$u^* \in \mathcal{M}, \quad \|u^* - v\| = \inf_{u \in \mathcal{M}} \|u - v\|$$

Definition 4.1.2. *The sequence u_k, $k \in \mathbb{N}$ is an minimizing sequence if*

$$u_k \in \mathcal{M}, \quad \|u_k - v\| \to \inf_{u \in \mathcal{M}} \|u - v\|, \quad k \to \infty. \tag{4.1}$$

Theorem 4.1.3 (existence of best approximation)**.** *If u_k is a minimizing sequence and has an accumulation point u^* in $\mathcal{M}$, then u^* is a best approximation to v.*

Proof. Just take the limit (subsequence) on both sides to

$$\|u^* - v\| \leq \|u^* - u_k\| + \|u_k - v\|. \tag{4.2}$$

□

Theorem 4.1.4. *If $\mathcal{M}$ is a compact subset of $\mathcal{V}$, then the best approximation always exists.*

Proof. □

One special case is that $\mathcal{M}$ is a finite dimension linear subspace of $\mathcal{V}$, then one can take a bounded closed set truncating the minimizing sequence, then such set must be compact.

Lemma 4.1.5 (convexity)**.** *If $\mathcal{M}$ is a convex set of normed space $\mathcal{V}$, then the set of best approximations is convex.*

Proof. □

Theorem 4.1.6 (uniqueness)**.** *If $\mathcal{M}$ is strictly convex of normed space $\mathcal{V}$, then the best approximation is unique. It is worthwhile to notice that strictly convexity is sufficient but not necessary.*

Proof. □

Definition 4.1.7. *The normed space $(\mathcal{V}, \|\cdot\|)$ is strictly normed if and only if the unit ball is strictly convex).*

Theorem 4.1.8 (uniqueness)**.** *If $\mathcal{M}$ is a strictly normed linear subspace of normed space $\mathcal{V}$, then there exists at most one best approximation for each $v \in \mathcal{V}$.*

Proof. □

Remark 4.1.9. *The above theorems are quite preliminary in general Banach spaces. Formalizing the proofs with `Lean` is recommended. See Exercises.*

4.1.1 Orthogonality in L^p Spaces

The orthogonality in L^p spaces is defined similarly to L^2 spaces. Here $p \in (1, \infty)$.

Definition 4.1.10. *Let $H = L^p(\mathcal{X}, \mu)$. If $\|f\|_p \leq \|f + g\|_p$ for all $g \in H' \subset H$ in a closed subspace, then we say $f \perp H'$.*

Since the L^p norm is strictly convex, for any $f \in H$, there exists a unique $f^* \in H' \subset H$ as the best approximation in H'. It is not difficult to observe that $f - f^* \perp H'$.
Using the Hahn-Banach theorem, we can prove the following lemma.

Lemma 4.1.11. *Let H' be a closed subspace of $H = L^p(\mathcal{X}, \mu), p \in (1, \infty)$. Then $f \perp H'$ is equivalent to*

$$\langle |f|^{p-2}\overline{f}, \overline{g}\rangle = 0, \quad \forall g \in H'. \tag{4.3}$$

Proof. ($\Leftarrow$) Assuming the equality,

$$\|f\|_p^p = \langle |f|^{p-2}\overline{f}, \overline{f}\rangle = \langle |f|^{p-2}\overline{f}, \overline{f+g}\rangle \le \||f|^{p-2}\overline{f}\|_{p'}\|f+g\|_p = \|f\|_p^{p-1}\|f+g\|_p, \tag{4.4}$$

which implies the orthogonality $\|f\| \le \|f+g\|$ for all $g \in H'$.
($\Rightarrow$) Assuming the orthogonality, then by the Hahn-Banach theorem, there exists a linear functional $\mathcal{D} \in L^{p,*} = L^{p'}$ that

$$\mathcal{D}f = 1, \quad \mathcal{D}H' = 0, \quad \|\mathcal{D}\| = \|f\|_p^{-1}. \tag{4.5}$$

By Rietz representation theorem, we have an element $h \in L^{p'}$ that $\mathcal{D}f = \langle f, h\rangle$ with $\|h\|_{p'} = \|f\|_p^{-1}$. Therefore, the Hölder inequality

$$1 = \mathcal{D}f = \langle f, h\rangle \le \|f\|_p\|h\|_{p'} = 1. \tag{4.6}$$

The condition for equality is $f(x)h(x) \ge 0$ and $|h|^{p'} = \lambda|f|^p$ for certain $\lambda > 0$. Thus, $h \propto |f|^{p-2}\overline{f}$ and our conclusion is proved by $\mathcal{D}H' = 0$. □

4.2 Minimax Approximation

Given $f \in C^0([a,b])$, find a polynomial $p_n \in \Pi_n$ such that

$$\|f - p_n\|_\infty = \min_{g\in\Pi_n} \|f-g\|_\infty$$

Such a problem is a typical minimax approximation problem. In the previous Chapter 2, we have seen a similar problem which is to minimize the maximum of $|\omega(x)|$ with $\omega(x) = \prod_{j=0}^n (x - x_j)$. The proof used there can be borrowed for the following theorem as well.

Theorem 4.2.1 (de la Vallée-Poussin). *Let $f \in C^0([a,b])$ and $n \ge 0$, let $x_0 < x_1 < \cdots < x_{n+1}$ be $n+2$ nodes in $[a,b]$. If there exists a polynomial $q_n \in \Pi_n$ such that*

$$f(x_j) - q_n(x_j) = (-1)^j e_j, \quad j = 0, \ldots, n+1.$$

where e_j are having the same sign and nonzero, then

$$\min_j |e_j| \le E_n^*(f) := \min_{g\in\Pi_n} \|f-g\|_\infty \le \max_j |e_j|. \tag{4.7}$$

Proof. Prove by contradiction. Suppose $E_n^*(f)$ is achieved by some polynomial $g \in \Pi_n$. If q_n satisfies that

$$|e_j| > E_n^*(f) = \|f-g\|_\infty \quad \forall j = 0, 1, \cdots n+1. \tag{4.8}$$

Then, $q_n - g = q_n - f - (g - f)$ implies that

$$\text{sgn}(q_n - g)(x_j) = \text{sgn}(q_n - f)(x_j) = -(-1)^j \,\text{sgn}(e_j), \tag{4.9}$$

which changes sign $n+1$ times while $q_n - g$ only has n roots. □

The above theorem implies the sufficiency of the "equioscillation" property of length $n+2$ for the minimax approximation. The next theorem shows it is also necessary.

Theorem 4.2.2 (Chebyshev equioscillation theorem)**.** *Let $f \in C^0([a,b])$ and $n \geq 0$. Then there exists a* unique *polynomial $q^* \in \Pi_n$ such that*

$$E_n^*(f) = \|f - q_n^*\|_\infty. \tag{4.10}$$

This polynomial is uniquely characterized by the property that $\exists a \leq x_0 < \cdots < x_{n+1} \leq b$ for which we can select $\sigma = \pm 1$ that

$$f(x_j) - q_n^*(x_j) = \sigma(-1)^j \|f - q_n^*\|_\infty, \quad j = 0, 1, \ldots, n+1. \tag{4.11}$$

Proof. The existence of such minimizing polynomial $q^* \in \Pi_n$ can be proved through a minimizing sequence argument. Let $q^k \in \Pi_n$ be a minimizing sequence to having $\|q^k - f\| \to E_n^*(f)$, then it is clear that the set

$$\mathcal{M} = \{q \in \Pi_n \mid \|q - f\| \leq E_n^*(f) + 1\} \tag{4.12}$$

must be non-empty and such a set is compact since $\mathcal{M}$ is of finite dimension and closed. Then the minimizing sequence will have a converging sub-sequence, so the limit sits in $\mathcal{M}$.

Then we show it is necessary (sufficiency by Theorem 4.2.1) to have (4.11) for this minimizing polynomial q_n. If it is not satisfied, then we can partition the interval $[a,b]$ into $1 \leq N \leq n+1$ parts

$$[a,b] = \cup_{j=1}^N [t_{j-1}, t_j], \quad a = t_0 < \cdots < t_N = b,$$

such that

1. $f(t_k) - q_n(t_k) = 0$, $k = 1, \ldots, N-1$.
2. for each $1 \leq k \leq N$, there exists (may not be unique) $s_k \in [t_{k-1}, t_k]$ such that

   $$|f(s_k) - q_n(s_k)| = \|f - q_n\|_\infty \neq 0$$

   and for any $x \in [t_{k-1}, t_k]$,

   $$-(f(x) - q_n(x)) \neq (f(s_k) - q_n(s_k)).$$

3. for each $1 \leq k \leq N-1$,

   $$f(s_k) - q_n(s_k) = -(f(s_{k+1}) - q_n(s_{k+1})).$$

Without loss of generality, one can assume that

$$\text{sgn}(f(s_k) - q_n(s_k)) = (-1)^{k-1}, \tag{4.13}$$

then using the second condition, there exists ε such that $\forall x \in [t_{k-1}, t_k]$,

$$\begin{aligned} -\|f - q_n\|_\infty + \varepsilon \leq f(x) - q_n(x) \qquad & k \text{ is odd} \\ f(x) - q_n(x) \leq \|f - q_n\|_\infty - \varepsilon \qquad & k \text{ is even} \end{aligned} \tag{4.14}$$

Then we construct a polynomial $g \in \Pi_{N-1}$ that

$$\text{sgn}(g(x)) = (-1)^k, \quad x \in [t_{k-1}, t_k]. \tag{4.15}$$

and $\|g\| \leq \frac{\varepsilon}{2}$ and let $k(x) = q_n(x) - g(x) \in \Pi_n$ and $f(x) - k(x) = f(x) - q_n(x) + g(x)$, which implies that

1. $f(x) - k(x) < f(x) - q_n(x)$, if $x \in (t_{k-1}, t_k)$ for k odd.
2. $f(x) - k(x) \geq -\|f - q_n\| + \frac{\varepsilon}{2}$, if $x \in (t_{k-1}, t_k)$ for k odd.
3. $f(x) - k(x) > f(x) - q_n(x)$, if $x \in (t_{k-1}, t_k)$ for k even.
4. $f(x) - k(x) \leq \|f - q_n\| - \frac{\varepsilon}{2}$, if $x \in (t_{k-1}, t_k)$ for k odd.

In this way, $\|f - k\|_\infty < \|f - q_n\|_\infty$, which is a contradiction.
In the last, we show the uniqueness. Suppose there are two polynomials $q_n, \tilde{q}_n$ being the minimax approximation. Then $\frac{1}{2}(q_n + \tilde{q}_n)$ must also be a minimax approximation, therefore there exist the alternation nodes

$$a \leq s_0 < s_1 < \cdots < s_{n+1} \leq b \tag{4.16}$$

such that

$$\frac{1}{2}(q_n - f)(s_k) + \frac{1}{2}(\tilde{q}_n - f)(s_k) = \sigma(-1)^k E_n^*(f) \tag{4.17}$$

Therefore, we must have

$$(q_n - f)(s_k) = (\tilde{q}_n - f)(s_k) \Rightarrow q_n(s_k) = \tilde{q}_n(s_k), \tag{4.18}$$

which implies $q_n = \tilde{q}_n$ by the uniqueness of interpolation. □

Remark 4.2.3. *If the approximation is under L^2 norm instead of C^0 norm, then*

$$f_n^* = \underset{f_n \in \Pi_n}{\arg\min} \|f - f_n\|_{L^2[a,b]} = \sum_{k=0}^{n} a_k p_k \tag{4.19}$$

where $\{p_k\}_{k \geq 0}$ is the set of orthogonal polynomials (Legendre polynomials) on $[a, b]$ and the coefficients $a_k = \langle f, p_k \rangle / \langle p_k, p_k \rangle$.

4.2.1 Remez Algorithm

The Chebyshev equioscillation theorem does not provide a constructive way to find the best polynomial approximation in L^∞ norm. However, the numerical method to find the minimax polynomial could adopt the idea of the above Chebyshev equioscillation theorem. Intuitively, the aiming polynomial will be oscillatory compared with f, therefore we can start with the Chebyshev polynomial approximation, which is

$$C_n(x) = \sum_{j=0}^{n} c_j T_j(x), \quad c_j = \langle f, T_j \rangle_w / \langle T_j, T_j \rangle_w \tag{4.20}$$

where $w = (1-x^2)^{-1/2}$. The convergence is uniform as $n \to \infty$, especially if $f \in C^r([a,b])$, the coefficient $c_j \sim \mathcal{O}(j^{-r})$ decays fast, then the reminder approximately $f - C_n \simeq c_{n+1} T_{n+1}$, this reminder achieves equioscillation at exactly $(n+2)$ points. The Remez algorithm (see Algorithm 2) is based on the above idea and performs an iterative construction.

Algorithm 2: Remez algorithm for polynomial approximation on $[-1,1]$

Data: Objective function $f(x)$ on $[-1,1] \subset \mathbb{R}$
Result: Polynomial $p(x) \in \Pi_n$ and estimated error E
$\{x_j\}_{j=0}^n \leftarrow$ `findExtreme`(T_{n+1})// Initialize local extrema nodes of T_{n+1};
while *True* **do**
  Solve the equation for a_j and E,

$$\sum_{j=0}^{n} a_j x_k^j + (-1)^k E = f(x_k), \quad k = 0, \cdots n+1; \tag{4.21}$$

  $p(x) \leftarrow \sum_{j=0}^n a_j x^j$; // new candidate polynomial;
  stop $\leftarrow$ `isEquioscillation`$(p-f)$ // check equioscillation condition (4.11) ;
  if *stop* **then**
    return
  else
    $\{x_j\}_{j=0}^n \leftarrow$ `findExtreme`$((p-f))$ // find local extrema nodes of $p-f$.
  end
end

4.2.2 Polynomial Approximation for Analytic Functions

Let $f(z)$ be an analytic function inside a domain $D_\rho \subset \mathbb{C}$, where a finite interval $I \subset D_\rho$. Usually, f can be well approximated by polynomials in D_ρ. Here, the parameter ρ and the definition of D_ρ will be elaborated later. In particular,

$$E_{n,\infty}(f) = \min_{f_n \in \Pi_n} \|f - f_n\|_{L^\infty(I)} \tag{4.22}$$

decays very quickly in terms of n. Let $p_n \in \Pi_n$, $n = 0, 1, \cdots$ be the orthogonal polynomials over $I \subset \mathbb{R}$ with the analytic weight w. Suppose one can represent (pointwise convergence)

$$f(x) = \sum_{n=0}^{\infty} a_n p_n(x), \quad a_n = \frac{\langle f, p_n \rangle_w}{\langle p_n, p_n \rangle_w}, \tag{4.23}$$

then, $E_{n,\infty}(f) \leq \sum_{k>n} |a_k| \sup_I |p_k|$. Therefore, we only have to estimate the coefficients $|a_n|$.

Definition 4.2.4 (weighted Cauchy transform). *Let $x \in \mathbb{R}$ and suppose $w(x)p_n(x)$ satisfies that*

$$|w(x)p_n(x)| \leq C_n \max_{1 \leq j \leq J} \frac{1}{|x - t_j|^{\alpha}}, \quad \alpha \in (0, 1) \tag{4.24}$$

for a finite number of $t_j \in \mathbb{R}$. Then the weighted Cauchy transform of p_n is defined by

$$P_n(z) = \frac{1}{2\pi i} \int_I \frac{w(x)p_n(x)}{z - x} dx, \quad z \in \mathbb{C} - \overline{I}. \tag{4.25}$$

Clearly, $P_n(z)$ is a holomorphic function in $\mathbb{C} - \overline{I}$. In addition, if $|z| \to \infty$, then $|P_n(z)| = \mathcal{O}(|z|^{-1})$.

Lemma 4.2.5 (Plemelj formula). *For any $x \in I$, then*

$$\lim_{\varepsilon \to 0^+} P_n(x - \varepsilon i) - P_n(x + \varepsilon i) = w(x)p_n(x).$$

Proof. Let $\delta_\varepsilon(x) = \frac{1}{\pi} \frac{\varepsilon}{x^2 + \varepsilon^2}$ and notice that

$$\begin{aligned} P_n(x - \varepsilon i) - P_n(x + \varepsilon i) &= \frac{1}{2\pi i} \int_I w(y)p_n(y) \left(\frac{1}{x - \varepsilon i - y} - \frac{1}{x + \varepsilon i - y} \right) \\ &= \int_I \delta_\varepsilon(x - y) * w(y)p_n(y) dy \\ &\to w(x)p_n(x). \end{aligned} \tag{4.26}$$

□

Lemma 4.2.6. *If f is analytic and bounded in the domain D_ρ and the weight w satisfies the condition in Definition 4.2.4, then the projection can be computed by the contour integral*

$$\langle f, p_n \rangle_w = \int_{\mathcal{C}_\rho} f(z) P_n(z) dz, \tag{4.27}$$

where the contour $\mathcal{C}_\rho = \partial D_\rho$.

Proof. Since the functions $f(z)$ and $p_n(z)w(z)$ are analytic in D_ρ except a finite number of points. Due to the mild growth of $w(x)p_n(x)$, the infinitesimal contours around these points will not contribute. Therefore, the contour $\mathcal{C}_\rho$ can be deformed in the lower and upper half-planes to the x axis. Then, use Lemma 4.2.5,

$$\int_{\mathcal{C}} f(z)P_n(z)dz = \lim_{\varepsilon \to 0} \int_I P_n(x - \varepsilon i) + P_n(x + \varepsilon i) f(x) dx = \int_I f(x)p_n(x)w(x)dx. \tag{4.28}$$

The Lemma 4.2.6 implies a trivial bound by the maximal modulus principle. □

The following examples estimate the approximation error for Chebyshev polynomials and Legendre polynomials. The asymptotic expressions of the Cauchy transform are from Elliott and Tuan (1974) for large n.

Example 4.2.7. *For Chebyshev polynomials on $I = [-1, 1]$ that $w(x) = (1 - x^2)^{-1/2}$, then $|p_n|$ are bounded by one,*

$$P_n(z) \sim \frac{1}{2i(z^2-1)^{1/2}[z+(z^2-1)^{1/2}]^n}, \quad z \notin [-1, 1]. \tag{4.29}$$

Therefore, if we choose $D_\rho = \{|z + (z^2 - 1)^{1/2}| \le \rho\}$ for $\rho > 1$, the domain is an ellipse with foci at ± 1, the ellipse is also called the Berstein ellipse. Then use $\langle p_n, p_n\rangle = \frac{\pi}{2}$ for $n > 1$,

$$|a_n| \sim \frac{\sup_{\mathcal{C}_\rho}|f|}{\pi\rho^n}\int_{\mathcal{C}_\rho}\frac{1}{|(z^2-1)|^{1/2}}d|z| = \frac{2}{\rho^n}\sup_{\mathcal{C}_\rho}|f|. \tag{4.30}$$

This implies the approximation error $E_{n,\infty}(f) \le \sum_{k>n}\frac{2}{\rho^k}\sup_{\mathcal{C}_\rho}|f| = \mathcal{O}(\rho^{-n})$.

Example 4.2.8. *For Legendre polynomials on $I = [-1, 1]$ that $w(x) = 1$ and $\langle p_n, p_n\rangle = \frac{2}{2n+1}$, the polynomials $|p_n|$ are bounded by one, then*

$$P_n(z) \sim \frac{2^{2n+1/2}|\Gamma(n+1)|^2}{i\pi\Gamma(2n+2)(z+(z^2-1)^{1/2})^{n+1/2}}(z^2-1)^{-1/4}, \quad z \notin [-1, 1]. \tag{4.31}$$

Then in the Berstein ellipse D_ρ,

$$|a_n| \sim \frac{\sup_{\mathcal{C}_\rho}|f|}{\pi\rho^{n+1/2}}\frac{2^{2n-1/2}(n!)^2}{(2n)!}\int_{\mathcal{C}_\rho}\left|(z^2-1)^{-1/4}\right|d|z| = \mathcal{O}\left(\frac{\sqrt{n}\sup_{\mathcal{C}_\rho|f|}}{\rho^n}\right). \tag{4.32}$$

Therefore, if $n > |\log\rho|^{-1}$ is sufficiently large, the approximation error satisfies

$$E_{n,\infty}(f) = \mathcal{O}\left(\sup_{\mathcal{C}_\rho}|f|\sum_{k>n}\frac{\sqrt{k}}{\rho^k}\right) = \mathcal{O}(\sqrt{n}\rho^{-n}). \tag{4.33}$$

Remark 4.2.9. *It can be shown for the general Jacobi polynomials p_n with weight $(1-x)^\alpha(1+x)^\beta$, the coefficient $|a_n| = \mathcal{O}(\sqrt{n}\rho^{-n})$ as the previous example. However, if $q = \max(\alpha, \beta)$, then*

$$E_{n,\infty}(f) = \sum_{k>n}\mathcal{O}\left(\sqrt{k}\rho^{-k}\sup_{[-1,1]}|p_k|\right) = \begin{cases}\mathcal{O}\left(n^{q+\frac{1}{2}}\rho^{-n}\right), & q \ge -\frac{1}{2},\\ \mathcal{O}\left(\rho^{-n}\right), & q < -\frac{1}{2}.\end{cases} \tag{4.34}$$

In other words, the Chebyshev polynomials have attained the best bound. The approximation error by Legendre polynomials only differs by a factor of $\sqrt{n}$.

4.2.3 Polynomial Approximation for Differentiable Functions

If $f \in C^{m-1}(I)$ with $f^{(m)} \in L^\infty(I)$ instead of being analytic, it is intuitive to conjecture that the coefficients $|a_n| = \mathcal{O}(n^{-m})$ are an analog to the Fourier approximation theory. Unlike the analytic case, extending the definition of f to the Berstein ellipse is not applicable.

Let us focus on the classical orthogonal polynomials which obey Rodrigues' formula (see Theorem 3.4.11).

$$p_n(x)w(x) = c_n \frac{d^n}{dx^n}\left[w(x)|g(x)|^n\right], \tag{4.35}$$

where $\frac{w'}{w} = \frac{h}{g}$ for $h \in \Pi_1$ and $g \in \Pi_2$.

1. Jacobi polynomials: $w(x) = (1-x)^\alpha(1+x)^\beta$ where $\alpha, \beta > -1$, and $\frac{w'}{w} = \frac{-\alpha(1+x)+\beta(1-x)}{1-x^2}$.
2. Laguerre polynomials: $w(x) = x^\alpha e^{-x}$ where $\alpha > -1$ and $\frac{w'}{w} = \frac{\alpha - x}{x}$.
3. Hermite polynomials: $w(x) = e^{-x^2}$ and $\frac{w'}{w} = -2x$.

Denote $Q(x) = c_n w(x)|g(x)|^n$. Since $w(x)g(x)|_{\partial I} = 0$, $\frac{d^k}{dx^k}(w(x)|g(x)|^n)|_{\partial I} = 0$ for $k < n$. We apply integration by parts to (4.23) m times ($m < n$).

$$\begin{aligned} a_n &= \frac{1}{\langle p_n, p_n\rangle_w}\int_I f(x)p_n(x)w(x)dx \\ &= \frac{1}{\langle p_n, p_n\rangle_w}\left(f'(x)Q^{(n-1)}(x)\Big|_{\partial I} - \int_I f'(x)Q^{(n-1)}(x)dx\right) \\ &= \cdots \\ &= \frac{(-1)^m}{\langle p_n, p_n\rangle_w}\int_I f^{(m)}(x)Q^{(n-m)}(x)dx. \end{aligned} \tag{4.36}$$

Therefore, we only have to estimate $\|Q^{(n-m)}\|_\infty$. Using Rodrigues' formula for Jacobi polynomials, it is straightforward to derive the following identity.

Lemma 4.2.10. *For Jacobi polynomials $p_n = P_n^{\alpha,\beta}$, the constant $c_n = \frac{(-1)^n}{2^n n!}$ and*

$$Q^{(n-m)}(x) = \frac{c_n}{c_{n-m}}(1-x)^{\alpha+m}(1+x)^{\beta+m}P_{n-m}^{a+m,\beta+m}(x) \tag{4.37}$$

Proof. Notice that

$$\begin{aligned} c_n\frac{d^{n-m}}{dx^{n-m}}\left[(1-x)^\alpha(1+x)^\beta|(1-x^2)|^n\right] &= c_n\frac{d^{n-m}}{dx^{n-m}}\left[(1-x)^{m+\alpha}(1+x)^{m+\beta}|(1-x^2)|^{n-m}\right] \\ &= \frac{c_n}{c_{n-m}}p_{n-m}^{\alpha+m,\beta+m}(x)(1-x)^{\alpha+m}(1+x)^{\beta+m}. \end{aligned}$$

□

The following estimate is from Nevai et al. (1994) for $\alpha, \beta > -\frac{1}{2}$, which has been improved in a later paper Krasikov (2005).

Theorem 4.2.11. $\max_{[-1,1]}(1-x)^{\alpha+\frac{1}{2}}(1+x)^{\beta+\frac{1}{2}}\left|P_n^{\alpha,\beta}(x)\right|^2 \leq \frac{2e(2+\sqrt{\alpha^2+\beta^2})}{\pi}$.

Note that the above bound is independent of n. Therefore, when $m \geq 1$, there exists a constant $C(\alpha, \beta, m)$ such that

$$|Q^{(n-m)}(x)| \leq C(\alpha, \beta, m)\left|\frac{c_n}{\gamma_n c_{n-m}}\right|. \tag{4.38}$$

Since $Q^{(n-m)}$ is bounded by the product of the following terms.

$$\sup_{[-1,1]}(1-x)^{\frac{\alpha+m}{2}-\frac{1}{4}}(1+x)^{\frac{\beta+m}{2}-\frac{1}{4}} \leq 2^{\frac{|\beta-\alpha|}{2}}$$

$$\sup_{[-1,1]}\left|p_{n-m}^{\alpha+m,\beta+m}(x)(1-x)^{\frac{\alpha+m}{2}+\frac{1}{4}}(1+x)^{\frac{\beta+m}{2}+\frac{1}{4}}\right| \leq \left|\frac{2e(2+\sqrt{(\alpha+m)^2+(\beta+m)^2})}{\pi}\right|^{1/2}.$$

Theorem 4.2.12. *For Jacobi polynomials $p_n = P_n^{\alpha,\beta}$, the coefficient a_n satisfies*

$$|a_n| \leq \frac{C'(\alpha,\beta)\|f^{(m)}\|_\infty}{\gamma_n}\frac{1}{n(n-1)\cdots(n-m)}\frac{1}{\sqrt{n-m}} \tag{4.39}$$

for n sufficiently large.

Proof. By replace $x = \cos\theta$, then

$$P_{n-m}^{m+\alpha,m+\beta}(x)(1-x)^{m+\alpha}(1+x)^{m+\beta} = P_{n-m}^{m+\alpha,m+\beta}(\cos\theta)\left(2\sin^2\frac{\theta}{2}\right)^{m+\alpha}\left(2\cos^2\frac{\theta}{2}\right)^{m+\beta}.$$

Using the Darboux formula for large n and $\theta \in [c(n-m)^{-1}, \pi - c(n-m)^{-1}]$, see Theorem 8.21.13 in Szegö (1939), we have the following asymptotic expression.

$$\begin{aligned}
&P_{n-m}^{m+\alpha,m+\beta}(\cos\theta)(2\sin^2\frac{\theta}{2})^{m+\alpha}(2\cos^2\frac{\theta}{2})^{m+\beta} \\
&= 2^{2m+\alpha+\beta}|n-m|^{-\frac{1}{2}}\left(\pi^{-\frac{1}{2}}\sin^{m+\alpha-\frac{1}{2}}\frac{\theta}{2}\cos^{m+\beta-\frac{1}{2}}\frac{\theta}{2}\cos(N\theta+\gamma) + \mathcal{O}((|n-m|\sin\theta)^{-1})\right),
\end{aligned}$$

where $N = n + \frac{\alpha+\beta+1}{2}$ and $\gamma = -\frac{\pi}{2}(m+\alpha+\frac{1}{2})$. Because the coefficient

$$\begin{aligned}
|a_n| &\leq \frac{1}{\gamma_n}\int_I |f^{(m)}(x)Q^{(n-m)}(x)|dx \\
&\leq \frac{1}{\gamma_n}\int_{I_c}|f^{(m)}(x)Q^{(n-m)}(x)|dx + \frac{1}{\gamma_n}\int_{I_c'}|f^{(m)}(x)Q^{(n-m)}(x)|dx
\end{aligned} \tag{4.40}$$

where $I_c = \{\theta \mid c(n-m)^{-1} \le \theta \le \pi - c(n-m)^{-1}\} = \{x \in I \mid -1 + \frac{c'}{|n-m|^2} \le x \le 1 - \frac{c'}{|n-m|^2}\}$ and $I_c' = I - I_c$. We also have

$$2^{m+\frac{\alpha}{2}+\frac{\beta}{2}-\frac{1}{2}} \sin^{m+\alpha-\frac{1}{2}} \frac{\theta}{2} \cos^{m+\beta-\frac{1}{2}} \frac{\theta}{2} = (1-x)^{\frac{1}{2}(m+\alpha-\frac{1}{2})}(1+x)^{\frac{1}{2}(m+\beta-\frac{1}{2})} \le 2^{|\beta-\alpha|/2}. \tag{4.41}$$

Therefore, applying Theorem 4.2.11 and neglecting the smaller term, there exists a constant C' such that

$$\begin{aligned} |a_n| &\le \frac{1}{\gamma_n}\left(|I_c|\|f^{(m)}\|_{L^\infty(I_c)} \sup_{I_c} |Q^{(n-m)}| + |I_c'|\|f^{(m)}\|_{L^\infty(I_c')} \sup_{I_c'} |Q^{(n-m)}(x)|\right) \\ &\le \left|\frac{c_n}{\gamma_n c_{n-m}}\right| \|f^{(m)}\|_\infty \left[2^{m+q+\frac{1}{2}}\mathcal{O}(|n-m|^{-1/2}) + \cancel{C(\alpha,\beta,m)\mathcal{O}(|n-m|^{-2})}\right] \\ &\le \frac{C'(\alpha,\beta)}{\gamma_n} \frac{1}{n(n-1)\cdots(n-m)} |n-m|^{-\frac{1}{2}} \|f^{(m)}\|_\infty \end{aligned} \tag{4.42}$$

for sufficiently large n. □

Remark 4.2.13. *Now it recovers the optimal rate $\mathcal{O}(n^{-m})$ (resp. $\mathcal{O}(n^{-m+\frac{1}{2}})$) for Chebyshev polynomials (resp. Legendre) when n is sufficiently large. The constant in $\mathcal{O}$ depends only on $\|f^{(m)}\|_\infty$ and α, β.*

4.3 Padé Approximation

Instead of linearly combining the basis functions for approximation, it is also possible to introduce nonlinear dependencies of the parameters for approximation. The most common one is the rational approximation. It uses the rational functions

$$R(x) = \frac{a_0 + a_1 x + \cdots + a_n x^n}{1 + b_1 x + \cdots + b_m x^m} \tag{4.43}$$

to approximate the objective function f, which permits an expansion $f(x) = \sum_{k=0}^{\infty} c_k x^k$. The Padé approximation solves the equations

$$R^{(q)}(0) = f^{(q)}(0), \quad q = 0, 1, \cdots, m+n. \tag{4.44}$$

Lemma 4.3.1 (existence and uniqueness)**.**

4.4 Neural Network

The neural network (NN) is a popular model for constructing nonlinear function approximations. Next, we introduce the fundamentals and leave the rest to later topics. The basic structure of NN consists of several layers of neurons that are connected through linear maps and activation functions.

Example 4.4.1 (Feed-forward). *Let* $\mathbf{f}_0(\mathbf{x}) = \mathbf{x} \in \mathbb{R}^{n_0}$ *and* $W_k \in \mathbb{R}^{n_{k-1} \times n_k}$, $\mathbf{b}_k \in \mathbb{R}^{n_k}$,

$$\mathbf{f}_k(\mathbf{x}) = \sigma(W_k \mathbf{f}_{k-1}(\mathbf{x}) + \mathbf{b}_k), \quad k = 1, 2, \cdots, K. \tag{4.45}$$

and $\mathbf{f}_{K+1} = W_{k+1}\mathbf{f}_k(\mathbf{x})$. *The activation function* σ *is a nonlinear function.*

4.4.1 Universal Approximation Theorem

The universal approximation theorem validates the feasibility of applying neural networks for approximation when the activation function is not polynomial.

4.4.2 Shallow Neural Network

The shallow neural networks (with $K = 1$ hidden layer) can be simplified to

$$\mathbf{f}(\mathbf{x}) = W_2 \sigma(W_1 \mathbf{x} + \mathbf{b}_1). \tag{4.46}$$

For simplicity, we assume that the function $\mathbf{f} : I \mapsto \mathbb{R}$ for some compact set $I \subset \mathbb{R}$. The neural network can be formulated in an explicit form.

$$\mathbf{f}(\mathbf{x}) = \sum_{l=1}^{n} a_l \sigma(w_l \mathbf{x} - b_l), \quad a_l, w_l, b_l \in \mathbb{R}. \tag{4.47}$$

Example 4.4.2. *If the parameters* $w_l \equiv 1$, *then the function* $\mathbf{f}$ *is a linear combination of shifted spline basis* $\sigma(\mathbf{x} - \cdot)$,

$$\mathbf{f}(\mathbf{x}) = \sum_{l=1}^{n} a_l \sigma(\mathbf{x} - b_l). \tag{4.48}$$

For example, if $\sigma(x - \cdot)$ *is a spline function, then the network is a spline approximation with trainable shifts.*

4.4.3 Gradient-Flow

The training stage involves an optimization process that minimizes an empirical loss function.

4.5 Exercises

The source code and tests for the computational part are hosted on GitHub.

4.5.1 Theoretical Part

Problem 4.5.1. *Show that the vector space $C^0([a,b])$ with $\|\cdot\|_\infty$ is not strictly normed.*

Problem 4.5.2. *Let $f(x) \in C^0([-1,1])$ is even (odd). Show that the minimax approximation $q_n(x) \in \Pi_n$ is also even (odd). Hint: Think about the function $\frac{1}{2}(q_n(x) \pm q_n(-x))$, then use the Chebyshev equioscillation theorem and the uniqueness.*

Problem 4.5.3. *Let $\mathcal{S}_k = \{x^{\lambda_1}, x^{\lambda_2}, \cdots, x^{\lambda_k}\}$ that $0 \le \lambda_1 < \lambda_2 < \cdots < \lambda_k$, show that*

$$\min_{f \in span(\mathcal{S}_k)} \|f - x^n\|_{L^2[0,1]} = \frac{1}{\sqrt{2n+1}} \prod_{j=1}^{k} \left| \frac{n-\lambda_j}{n+\lambda_j+1} \right|.$$

Show that the span of set $\lim_{k\to\infty} \mathcal{S}_k$ is dense in $L^2[0,1]$ if

$$\lim_{k\to\infty} \sum_{\lambda \in \mathcal{S}_k} \frac{1}{\lambda + 1/2} = \infty. \tag{4.49}$$

Problem 4.5.4. *Prove the Jacobi polynomials $P_n^{\alpha,\beta}(x)$ achieve maximum/minimum at $x = \pm 1$ if $\alpha, \beta \ge -\frac{1}{2}$.*

Problem 4.5.5. *TODO: analyze gradient flow.*

4.5.2 Computational Part

Problem 4.5.6. *Implement the Remez algorithm and find the minimax approximation in Π_3 for the function $f(x) = e^x$ on $[0,1]$.*

Problem 4.5.7. *Formalize the theorem/proofs in Section 4.1 with `Lean`.*

Extended Reading

Cheney, E. W. and Light, W. A. (2009). *A course in approximation theory*, volume 101. American Mathematical Soc.

Elliott, D. and Tuan, P. (1974). Asymptotic estimates of fourier coefficients. *SIAM Journal on Mathematical Analysis*, 5(1):1–10.

Krasikov, I. (2005). On the maximum value of jacobi polynomials. *Journal of Approximation Theory*, 136(1):1–20.

Muskhelishvili, N. I. and Radok, J. R. M. (2008). *Singular integral equations: boundary problems of function theory and their application to mathematical physics*. Courier Corporation.

Nevai, P., Erdélyi, T., and Magnus, A. P. (1994). Generalized jacobi weights, christoffel functions, and jacobi polynomials. *SIAM Journal on Mathematical Analysis*, 25(2):602–614.

Szegö, G. (1939). *Orthogonal polynomials*, volume 23. American Mathematical Soc.

Trefethen, L. N. (2019). *Approximation Theory and Approximation Practice, Extended Edition*. SIAM.

CHAPTER 5

ORDINARY DIFFERENTIAL EQUATIONS

The ordinary differential equations in the form

$$y' = f(y,t), \quad y(a) = y_0$$

are commonly used in modeling for various fields of physical sciences and biological studies. In this chapter we will derive and analyze the numerical methods for solving the above problems for ordinary differential equations.

5.1 Initial Value Problem

Let $f(y,t) : \mathbb{R}^n \times [a,b] \to \mathbb{R}^N$ be a continuous function, the initial value problem is to determine a differentiable solution y satisfying that

$$y' = f(y,t), \quad y(a) = y_0 \tag{5.1}$$

the derivative $y' \in \mathbb{R}$ denotes taking derivatives on all components of y. The general existence and uniqueness of solution to ODE (5.1) depends on the Lipschitz condition of f

$$\|f(y_1,t) - f(y_2,t)\| \le L\|y_1 - y_2\|, \quad t \in [a,b].$$

Theorem 5.1.1. *If f is L-Lipschitz continuous, then*

1. *The ODE (5.1) has a unique continuously differentiable solution $y : [t_0, t_1] \mapsto \mathbb{R}^N$.*
2. *The stability with respect to initial condition is Lipschitz,*

$$\|y(t) - \tilde{y}(t)\| \le e^{L(t-t_0)}\|y(t_0) - \tilde{y}(t_0)\|$$

where $y(t_0), \tilde{y}(t_0)$ are the initial conditions for y and $\tilde{y}$.

Remark 5.1.2. *Consider a perturbative ODE:*

$$y' = f(y,t) + \delta(t), \quad y(a) = y_0 + \delta_0 \tag{5.2}$$

such that $\|\delta(t)\|, \|\delta_0\| \le \varepsilon$ *for all* $t \in [a,b]$ *and the resulting perturbative equation's solution* $\tilde{y}$ *satisfies*

$$\|y(t) - \tilde{y}(t)\| \le C\varepsilon$$

then the problem is Liapunov stable. The Lipschitz continuity of f *guarantees the Liapunov stability.*

5.1.1 One-step Methods

The simplest numerical solution for IVP is the one-step method. We consider the discretization of solution

$$y_n \simeq y(t_n), \quad n = 0, 1, \ldots, M$$

on the grid

$$\Delta = \{a = t_0 < t_1 < \cdots < t_M = b\}, \quad h_n := t_{n+1} - t_n.$$

Definition 5.1.3 (scheme)**.** *The explicit (forward) one-step method for approximation of the solution to the IVP* (5.1) *is in the from*

$$y_{n+1} = y_n + h_n \phi(t_n, y_n; h_n), \quad y_0 := y(t_0). \tag{5.3}$$

where ϕ *is the iteration function.*

Definition 5.1.4 (convergence)**.** *The* <u>*convergence order*</u> *is* p *if the global error*

$$\max_{n\ge 0} \|y_n - y(t_n)\| \le C\bar{h}^p, \quad \bar{h} := \max_{n\ge 0} h_n \tag{5.4}$$

where C *is independent of the grid* Δ*.*

Definition 5.1.5 (local error)**.** *The local error* $E(t,h)$ *(the quantity* $E(t,h)/h$ *is the local truncation error)*

$$E(t,h) := y(t) + h\phi(t, y(t); h) - y(t+h), \quad 0 \le h \le b - t \tag{5.5}$$

denotes the induced error by one-step method at time $t + h$*.*

Definition 5.1.6 (consistency)**.** *The* <u>*consistency order*</u> *is* p *if*

$$\|E(t,h)\| \le Ch^{p+1}, \quad t \in [a,b], \quad 0 \le h \le b - t, \tag{5.6}$$

where C *is independent of* t, h*.*

Example 5.1.7 (Forward Euler). *If we let $\phi(t, y; h) = f(t, y) \in C^1([a, b] \times \mathbb{R}^N; \mathbb{R}^N)$, then we will have the local error in the form*

$$\begin{aligned} E(t,h) &= y(t) + hf(t, y(t)) - y(t+h) \\ &= y(t) + hf(t, y(t)) - (y(t) + hy'(t) + \frac{h^2}{2}y''(\xi)) \\ &= -\frac{h^2}{2}y''(\xi). \end{aligned} \tag{5.7}$$

which means consistency order is $p = 1$.

Example 5.1.8 (modified Euler). *Let the iteration function*

$$\phi(t, y; h) = a_1 f(t, y) + a_2 f(t + b_1 h, y + b_2 h f(t, y))$$

then local error

$$\begin{aligned} E(t,h) = y(t) &+ a_1 h f(t, y(t)) + a_2 h \left[f(t, y(t)) + \partial_t f|_{t,y(t)} b_1 h + \partial_y f|_{t,y(t)} b_2 h + \mathcal{O}(h^2) \right] \\ &- (y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}y'''(\xi)) \\ = (a_1 + a_2 - 1) &f(t, y(t)) + h^2 \left(a_2 b_1 \partial_t f|_{t,y(t)} + a_2 b_2 \partial_y f|_{t,y(t)} - \frac{1}{2}y''(t) \right) + \mathcal{O}(h^3) \end{aligned}$$

Therefore if the constants

$$a_1 + a_2 = 1, \quad a_2 b_1 = \frac{1}{2}, \quad a_2 b_2 = \frac{1}{2},$$

the consistency order is $p = 2$. This formulation cannot obtain a formula for $p = 3$.

1. *The modified Euler is $a_1 = 0$, $a_2 = 1$, $b_1 = b_2 = \frac{1}{2}$. The algorithm can be divided into two updating sub-steps:*

$$\begin{aligned} y_{n+1/2} &= y_n + \frac{1}{2}h_n f(t_n, y_n), \quad t_{n+1/2} = t_n + \frac{1}{2}h_n, \\ y_{n+1} &= y_{n+1/2} + \frac{1}{2}h_n f(t_{n+1/2}, y_{n+1/2}) \end{aligned} \tag{5.8}$$

2. *The Heun's method is $a_1 = \frac{1}{2}$, $a_2 = \frac{1}{2}$, $b_1 = b_2 = 1$. This method can be written into similar sub-steps as well.*

Remark 5.1.9. *For implicit methods, the iteration function involves unknown values. Two famous examples are implicit Euler:*

$$y_{n+1} = y_n + h_n f(t_{n+1}, y_{n+1})$$

and Crank-Nicolson:

$$y_{n+1} = y_n + \frac{h_n}{2}(f(t_n, y_n) + f(t_{n+1}, y_{n+1}).$$

Theorem 5.1.10. *If the one-step method has consistency order $p \geq 1$ and $\phi(t, y; h)$ is Lipschitz continuous in y variable, then convergence order is p.*

Proof. Let $e_n = y_n - y(t_n)$, then using the local error relation that

$$y(t_{n+1}) = y(t_n) + h_n \phi(t_n, y(t_n); h_n) - E(t_n, h_n) \tag{5.9}$$

we have

$$e_{n+1} = e_n + h_n(\phi(t_n, y_n; h_n) - \phi(t_n, y(t_n); h_n)) + E(t_n, h_n) \tag{5.10}$$

which implies that

$$\begin{aligned}
\|e_{n+1}\| &\leq \|e_n\| + h_n L \|y_n - y(t_n)\| + C h_n^{p+1} \\
&\leq (1 + h_n L)\|e_n\| + C h_n^{p+1} \\
&\leq e^{h_n L}\|e_n\| + C h_n^{p+1} \\
&\leq e^{h_n L} e^{h_{n-1} L}\|e_{n-1}\| + e^{h_{n-1} L} C h_n^{p+1} + C h_{n-1}^{p+1} \\
&\cdots \\
&\leq e^{(b-a)L}\|e_0\| + C e^{(b-a)L} \sum_{n \geq 0} h_n^{p+1} \\
&\leq C(b-a) e^{L(b-a)} \bar{h}^p.
\end{aligned} \tag{5.11}$$

□

Example 5.1.11 (explicit Runge Kutta)**.** *The Runge Kutta method RK4 has convergence order $p = 4$ uses ϕ as*

$$\phi(t, y; h) = \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k4) \tag{5.12}$$

where

$$\begin{aligned}
k_1 &= f(t, y), \\
k_2 &= f(t + \frac{h}{2}, y + \frac{h}{2} k_1), \\
k_3 &= f(t + \frac{h}{2}, y + \frac{h}{2} k_2), \\
k_4 &= f(t + h, y + h k_3).
\end{aligned} \tag{5.13}$$

The proof of consistency order uses Taylor's expansion, see exercises.

5.1.2 Absolute Stability

In the previous section, we have discussed the so-called zero-stability, which shows that small perturbation (comparable to step size) during each step will not cause too much trouble. This stability is with respect to perturbation. The absolute stability is to keep the numerical solution bounded (actually decaying) as $n \to \infty$, which is an asymptotic behavior. The usual toy problem is

$$y'(t) = f(t, y) := \lambda y(t), \quad t > 0 \tag{5.14}$$

where $\lambda \in \mathbb{C}$ and initial condition is $y(0) = 1$. The solution is simple $y(t) = e^{\lambda t}$, of course if $\Re\lambda > 0$, the solution would blow and the solution will decay to zero if $\Re\lambda < 0$.

Definition 5.1.12. *A numerical scheme is absolutely stable if*

$$y_n \to 0, \quad n \to \infty,$$

where the time step is fixed and y_n is the numerical solution to (5.14) under the given scheme. The region of absolute stability refers to the value of $z = h\lambda$ such that

$$\mathcal{A} = \{z = h\lambda \in \mathbb{C} \mid y_n \to 0,\, n \to \infty\} \tag{5.15}$$

Example 5.1.13 (forward Euler)**.** *With the scheme that*

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)y_n$$

we can easily find $y_n = (1 + h\lambda)^n = (1 + z)^n$. This value decays to zero as long as

$$|1 + z| < 1.$$

This region is a unit disk centered at $z = -1$, which means

$$h\lambda \in \mathbb{C}^-, \quad h \in (0, -2\frac{\Re\lambda}{|\lambda|^2}).$$

This shows that only when the step size is small enough, we can make sure the solution reflects the correct behavior of the solution's exponential decay.

Example 5.1.14. *Similarly, consider the implicit Euler, we will have*

$$y_n = (1 - h\lambda)^{-n}$$

then $|1 - z| < 1$ derives $\mathcal{A} \supset \mathbb{C}^-$. This means when $\lambda < 0$, the numerical solution's behavior will be decaying as well.

The numerical scheme is called A-stable if $\mathcal{A} \supset \mathbb{C}^-$. It can be shown that no explicit linear schemes can be A-stable Widlund (1967).

5.1.3 Rounding Error

In this section, we briefly discuss the impact of rounding error. Following the previous theorem about the local error, if each step involves rounding error of size $|\delta_n| \le \delta$

$$y_{n+1} = y_n + h_n\phi(t_n, y_n; h_n) + \delta_n \tag{5.16}$$

then e_{n+1} satisfies

$$\begin{aligned}\|e_{n+1}\| &= \|e_n + h_n(\phi(t_n, y_n; h_n) - \phi(t_n, y(t_n); h_n)) + \delta_n + E(t_n, h_n)\| \\ &\le e^{(b-a)L}\|e_0\| + Ce^{(b-a)L}\sum_{n\ge 0}(h_n^{p+1} + \delta) \\ &\le e^{(b-a)L}\|\delta_0\| + Ce^{(b-a)L}(b-a)\left(\bar{h}^p + \frac{\delta}{\underline{h}}\right)\end{aligned} \tag{5.17}$$

where $\underline{h} = \min_{n\ge 0} h_n$. When the grid is uniform that $h_n = h$, then the optimal grid size is minimizing $h^p + \frac{\delta}{h}$.

5.1.4 Extrapolation Methods

When the grid Δ is uniform, it is known that

$$y(t_n) - y_n = \mathcal{O}(h^p),$$

where p is the consistency order. Actually, one can show that $y(t_n) - y_n$ can be written in an asymptotic expansion. For a proof of the following theorem, we refer to Gragg (1965).

Theorem 5.1.15. *Let f and ϕ be $(p+r)$-times continuously partial differentiable in each variable. Then there exists coefficients $c_{p+j} \in C^{r+1-j}([a,b], \mathbb{R}^N)$ that $c_{p+j}(a) = 0$ for $j = 0, 1, \ldots, r-1$ and*

$$y(t_n) - y_n = \sum_{j=0}^{r-1} c_{p+j}(t_n)h^{p+j} + \mathcal{O}(h^{p+r}). \tag{5.18}$$

In order to find solution's value $y(t_n)$ with higher order of error, one can simply apply Richardson extrapolation.

5.1.5 Adaptive One-step Methods

The idea of adaptive one-step method is similar to the adaptive quadrature. The basic algorithm is briefly outlined in the following:

1. Using an initial uniform grid Δ with grid size of h, one can compute the numerical solution y_n with error of $e_h = \mathcal{O}(h^p)$.

2. For each time step from t_{n-1} to t_n, estimate the error of $e_h = y(t_n) - y_n$ from extrapolation, which is approximated by

   $$y_n(h) - z_n(h) \simeq (1 - 2^{-p})e_h$$

   where $z_n(h)$ is computed through a refined one-step method locally,

   $$\begin{aligned} w_n(h) &= y_n(h) + \frac{h}{2}\phi(t_n, y_n; h/2), \\ z_n(h) &= w_n(h) + \frac{h}{2}\phi(t_n + \frac{h}{2}, w(h); \frac{h}{2}). \end{aligned} \tag{5.19}$$

3. If the error is within certain tolerance threshold, then continue to next time step, otherwise sub-divide $[t_{n-1}, t_n]$ and perform the adaptive one-step method.

5.1.6 Multistep Methods

An m-step method for the ODE (5.1) on uniform grid is formulated by

$$\sum_{j=0}^{m} c_j y_{l+j} = h\phi(t_l, y_l, \ldots, y_{l+m}; h) \tag{5.20}$$

The coefficients $c_j \in \mathbb{R}$ and the leading coefficient $c_m \neq 0$. Clearly the one-step method is a special case that $c_0 = -1$ and $c_1 = 1$. The time stamps are equally spaced $t_l = a + lh$ for $l = 0, \ldots, n$, where $h = \frac{b-a}{n}$. The initial values $y_0, \ldots, y_{m-1}$ are assumed known.

Remark 5.1.16. *Usually, the initial value y_0 is given, while the other initial values are not. The preparation of the remaining values $y_1, \ldots, y_{m-1}$ can be done through the aforementioned one-step methods. The m-step method can also be implicit if unknown values are involved.*

Example 5.1.17. *The central difference explicit scheme*

$$y_{n+1} = y_{n-1} + 2hf(t_n, y_n)$$

Intuitively,

$$y_{n+1} = y_{n-k} + \int_{t_{n-k}}^{t_{n+1}} f(t, y(t))dt \tag{5.21}$$

and one can replace the integral with any quadrature rule involving nodes t_j in between (end nodes included, but could be implicit).

We can define the convergence and consistency for m-step methods similarly by extending one-step methods.

Definition 5.1.18 (convergence)**.** *The convergence order is p if the scheme satisfies*

$$\|y(t_n) - y_n\| \leq Ch^p$$

for an independent C for all n (including initial values).

Definition 5.1.19 (local error)**.** *The local error is*

$$E(t, h) = \left(\sum_{j=0}^{m} c_j y(t + jh)\right) - h\phi(t, y(t), y(t + h), \ldots, y(t + mh); h) \tag{5.22}$$

$E(t, h)/h$ is local truncation error. Here we normalize the formula by setting $c_m = 1$.

Definition 5.1.20 (consistency)**.** *The consistency order is p if local error*

$$\|E(t,h)\| \leq Ch^{p+1}.$$

For the following context, we assume that ϕ *is Lipschitz for all variables except the first one*, clearly the linear multi-step methods satisfies this condition.

Definition 5.1.21 (root condition)**.** *The generating polynomial with respect to multi-step method is*

$$p(\xi) := \sum_{j=0}^{m} c_j \xi^m \tag{5.23}$$

The Dahlquist's root condition is that the roots of $p(\xi)$ satisfies

$$p(\xi) = 0 \Rightarrow |\xi| \leq 1 \tag{5.24}$$

if a root $|\xi| = 1$, then it is simple.

Theorem 5.1.22. *If root condition is satisfied, then*

$$\|y_l - y(t_l)\| \leq C \left(\max_{0 \leq j \leq m-1} \|y_j - y(t_j)\| + \max_{t \in [a.b-mh]} \|E(t,h)\|/h \right) \tag{5.25}$$

Proof. Similar to one-step method, we have

$$\begin{aligned} \sum_{j=0}^{m} c_j y(t_{l+j}) &= h\phi(t_l, y(t_l), \ldots, y_{t_{l+m}}; h) + E(t_l, h) \\ \sum_{j=0}^{m} c_j y_{l+j} &= h\phi(t_l, y_l, \ldots, y_{l+m}; h) \end{aligned} \tag{5.26}$$

Take the difference, then

$$\sum_{j=0}^{m} c_j e_{l+j} = \underbrace{h\left[\phi(t_l, y_l, \ldots, y_{l+m}; h) - \phi(t_l, y(t_l), \ldots, y_{t_{l+m}}; h)\right]}_{\delta_l} - E(t_l, h) \tag{5.27}$$

The right-hand-side can be reviewed as certain perturbation and left-hand-side is a linear recursive scheme. This can be made into matrix form

$$\underbrace{\begin{pmatrix} e_{l+1} \\ e_{l+2} \\ \vdots \\ e_{l+m} \end{pmatrix}}_{E_l} = \underbrace{\begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ -c_0 & \cdots & \cdots & \cdots & -c_{m-1} \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} e_l \\ e_{l+1} \\ \vdots \\ e_{l+m-1} \end{pmatrix}}_{E_l} + \underbrace{\begin{pmatrix} 0 \\ 0 \\ \vdots \\ \delta_l - E(t_l, h) \end{pmatrix}}_{\Delta_l} \tag{5.28}$$

The iteration then can be written as

$$E_l = A^l E_0 + \sum_{j=0}^{l-1} A^{l-1-j} \Delta_j \tag{5.29}$$

If E_l does not below, we need all eigenvalues λ_k of A satisfy $|\lambda_k| \leq 1$, which are the roots of the polynomial (use elementary operations)

$$\det(\lambda I - A) = 0$$

That is exactly the generating polynomial:

$$p(\lambda) = \sum_{j=0}^{m} c_j \lambda^m \tag{5.30}$$

When the root condition is satisfied, for the $|\lambda_k| < 1$, those components will decay exponentially. If $|\lambda_k| = 1$, its geometric multiplicity has to be equal to algebraic multiplicity (then the Jordan block is diagonal), otherwise one Jordan block will be with size greater than one,

$$J_k = \begin{pmatrix} \lambda_k & 1 & & \\ & \lambda_k & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_k \end{pmatrix} \tag{5.31}$$

and

$$\begin{pmatrix} \lambda_k & 1 & & \\ & \lambda_k & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_k \end{pmatrix}^s \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \begin{pmatrix} s\lambda_k^{s-1} \\ \lambda_k^s \\ 0 \\ \vdots \\ 0 \end{pmatrix} \tag{5.32}$$

which will lead to unbounded behavior of A^l. Of course the root condition eliminates this case. Therefore

$$\|E_l\|_\infty \leq C \left(\|E_0\|_\infty + \sum_{j=0}^{l-1} \|\Delta_j\|_\infty \right) \tag{5.33}$$

where Δ_s is bounded by the Lipschitz condition

$$\begin{aligned} \|\Delta_s\|_\infty &\leq \|E(t_s, h)\| + hL \sum_{j=0}^{m} |e_{s+j}| \\ &\leq \|E(t_s, h)\| + hLm\|E_s\|_\infty + hL\|E_{s+1}\|_\infty \end{aligned} \tag{5.34}$$

Hence

$$\sum_{s=0}^{l-1} \|\Delta_s\|_\infty \leq l \max_s \|E(t_s, h)\| + hL(m+1) \sum_{s=0}^{l-1} \|E_s\|_\infty + hL\|E_l\|_\infty \tag{5.35}$$

It is then easy to see when h is small, the error E_l is bounded by E_0 and $E(t_s, h)$ (note $l \leq n \simeq \mathcal{O}(h^{-1})$). □

Corollary 5.1.23. *If root condition is satisfied, then consistency order p implies convergence order p for suitable step size h.*

For consistency order, the main tool is the Taylor expansion, we carry out the following lemma without providing a proof (see exercise).

Lemma 5.1.24. *The linear m-step method*

$$\sum_{j=0}^{m} c_j y_{l+j} = h \sum_{j=0}^{m} \beta_j f(t_{l+j}, y_{l+j}) \tag{5.36}$$

has consistency order p if

$$\sum_{j=0}^{m} (j^s c_j - s j^{s-1} \beta_j) = 0, \quad s = 0, 1, \ldots, p. \tag{5.37}$$

Proof. The proof is simple. Here we need f to be p-times continuously differentiable. □

5.1.7 Adams Method

Let us recall the idea of quadrature for ODE solving.

$$y(t_{l+m}) - y(t_{l+m-1}) = \int_{t_{l+m-1}}^{t_{l+m}} f(t, y(t)) dt \tag{5.38}$$

The m-step method now needs to represent the integral by a discrete quadrature scheme over t_{l+j}, $j = 0, 1, \ldots, m$ or $j = 0, 1, \ldots, m-1$ depending the scheme is implicit or explicit. Following the intuition when we derive quadrature rules, it is first to find a interpolating polynomial. Let $p(x)$ be a degree $(m-1)$ polynomial interpolating on the nodes $(t_j, f(t_j, y_j))$, $j = l, l+1, \ldots, l+m-1$, then approximately

$$y(t_{l+m}) - y(t_{l+m-1}) = \int_{t_{l+m-1}}^{t_{l+m}} p(t) dt \tag{5.39}$$

Note this is extrapolation. In the next, we try to calculate $p(x)$ with the Newton form backward.

$$\begin{aligned} p(t_{l+m-1} + sh) &= \sum_{k=0}^{m-1} p[t_{l+m-1}, \ldots, t_{l+m-1-k}] \prod_{j=0}^{k-1} (t_{l+m-1} + sh - (t_{l+m-1} - jh)) \\ &= \sum_{k=0}^{m-1} p[t_{l+m-1}, \ldots, t_{l+m-1-k}] h^k \prod_{j=0}^{k-1} (s + j) \end{aligned} \tag{5.40}$$

Therefore

$$y(t_{l+m}) - y(t_{l+m-1}) \approx h \int_0^1 \sum_{k=0}^{m-1} p[t_{l+m-1}, \ldots, t_{l+m-1-k}] h^k \prod_{j=0}^{k-1} (s+j) ds \tag{5.41}$$

This method is called Adam-Bashfort method. The above formula can be written into linear form uniquely. Here are a few examples of Adam-Bashfort m-step methods.

1. $m = 1$. $y_{l+1} - y_l = hf(t_l, y_l)$. This is explicit Euler.

2. $m = 2$. $y_{l+2} - y_{l+1} = \frac{h}{2}(3f(t_{l+1}, y_{l+1}) - f(t_l, y_l))$.

3. $m = 3$. $y_{l+3} - y_{l+2} = \frac{h}{12}(23f(t_{l+2}, y_{l+2}) - 16f(t_{l+1}, y_{l+1}) + 5f(t_l, y_l))$.

4. $m = 4$. $y_{l+4} - y_{l+3} = \frac{h}{24}(55f(t_{l+3}, y_{l+3}) - 59f(t_{l+2}, y_{l+2}) + 37f(t_{l+1}, y_{l+1}) - 9f(t_l, y_l))$.

Now we can study the consistency order of the above m-step method. First the generating polynomial is

$$q(\xi) = \xi^{m-1}(\xi - 1)$$

which satisfies root condition clearly. The local error is to compare the interpolating polynomial and the right-hand-side of (5.41), which means

$$E(t, h) = h\mathcal{O}(h^m) = \mathcal{O}(h^{m+1}) \quad \text{since } \deg(p) = m - 1. \tag{5.42}$$

That means consistency order is $p = m$.
When the Adams method considers implicit scheme (called Adam-Moulton method), that is, the interpolation is including the end point.

$$p(t_j) = f(t_j, y_j), \quad j = l, \ldots, l + m, \tag{5.43}$$

the polynomial is of degree m. The similar approach applies. The consistency order is clearly $p = m + 1$ using the error estimate of polynomial interpolation. Here we also list a few cases for this case.

1. $m = 1$. $y_{l+1} - y_l = \frac{h}{2}(f(t_l, y_l) + f(t_{l+1}, y_{l+1}))$. This is Crank-Nicolson method.

2. $m = 2$. $y_{l+2} - y_{l+1} = \frac{h}{12}(5f(t_{l+2}, y_{l+2}) + 8f(t_{l+1}, y_{l+1}) - f(t_l, y_l))$.

3. $m = 3$. $y_{l+3} - y_{l+2} = \frac{h}{24}(9f(t_{l+3}, y_{l+3}) + 19f(t_{l+2}, y_{l+2}) - 5f(t_{l+1}, y_{l+1}) + f(t_l, y_l))$.

5.2 Boundary Value Problem

The boundary value problem (BVP) considers the problem

$$\sum_{j=0}^{m} c_j D^j y = f(x, y, Dy, \ldots, D^{m-1}y) \tag{5.44}$$

with boundary conditions

$$\sum_{j=0}^{m-1} c_{kj} D^j y(a) + d_{kj} D^j y(b) = \alpha_k, \quad k = 0, \ldots, m-1.$$

All BVP can be formulated into vector form of first order ODE. Let $Y_k = D^k y$, then we can obtain an ODE for the unknown vector $\mathcal{Y} = (Y_0, \ldots, Y_m)$ such that

$$DY_k = Y_{k+1}, \quad k = 0, \ldots, m-1 \tag{5.45}$$

and

$$c_m DY_m + \sum_{j=0}^{m-1} c_j Y_j = f(x, Y_0, \ldots, Y_{m-1}). \tag{5.46}$$

Among all kinds of ODE, the second-order ODE is mostly considered (e.g. Sturm-Liouville). We state some properties for these ODEs.

Theorem 5.2.1. *The Sturm-Liouville problem*

$$\begin{aligned} -y''(x) + r(x)y(x) &= f(x), \\ y(a) = y(b) &= 0, \end{aligned} \tag{5.47}$$

where f is a continuous function. This problem permits a unique solution $y \in C^2([a,b])$ when r is a non-negative continuous function.

Remark 5.2.2. *One can show there exists a unique weak solution $y \in H^1([a,b])$ which is Hölder continuous by embedding, therefore it is a strong solution due to $y'' = ry - f \in C^0([a,b])$.*

5.2.1 Finite Difference Method

The finite difference method is to discretize the derivatives by finite difference. Let us recall the central scheme for the first derivative and second derivative of y by

$$\begin{aligned} \frac{y(x+h) - y(x-h)}{2h} &= y'(x) + y'''(\xi)\frac{h^2}{6} \\ \frac{y(x+h) - 2y(x) + y(x-h)}{h^2} &= y''(x) - y^{(4)}(\xi)\frac{h^2}{12} \end{aligned} \tag{5.48}$$

The central scheme usually has better local error than forward or backward difference. In the following, we try to formulate the finite difference method for the Sturm-Liouville problem with $r \geq 0$. Take the grid $\Delta \subset [a, b]$ as

$$\Delta = \{x_k = a + kh,\ k = 0, \ldots, N\} \tag{5.49}$$

such that $h = \frac{b-a}{N}$ as step size which stands for the solution resolution. Then we use central scheme for the second derivative at each x_k,

$$\frac{-y_{k+1} + 2y_k - y_{k-1}}{h^2} + r_k y_k = f_k \tag{5.50}$$

where $r_k = r(x_k)$, $f_k = f(x_k)$, $k = 1, \ldots, N-1$, the two end points are excluded since they are prescribed as $y_0 = y_N = 0$. This linear system about $\mathcal{Y} = (y_1, \ldots, y_{N-1})$ is

$$\frac{1}{h^2}\begin{pmatrix} 2 + r_1 h^2 & -1 & & & & \\ -1 & 2 + r_2 h^2 & -1 & & & \\ & \ddots & \ddots & \ddots & & \\ & & \ddots & \ddots & \ddots & \\ & & & \ddots & \ddots & -1 \\ & & & & -1 & 2 + r_{N-1} h^2 \end{pmatrix} \mathcal{Y} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{pmatrix} \tag{5.51}$$

if we denote the right-hand-side as F, then $\mathcal{Y} = A^{-1}F$. There error can be derived from the approximation error of finite difference, which is

$$\frac{1}{h^2}\begin{pmatrix} 2 + r_1 h^2 & -1 & & & & \\ -1 & 2 + r_2 h^2 & -1 & & & \\ & \ddots & \ddots & \ddots & & \\ & & \ddots & \ddots & \ddots & \\ & & & \ddots & \ddots & -1 \\ & & & & -1 & 2 + r_{N-1} h^2 \end{pmatrix} \widetilde{\mathcal{Y}} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{pmatrix} + \mathcal{O}(h^2)\|y^{(4)}\|_\infty \tag{5.52}$$

where $\widetilde{\mathcal{Y}}$ is the exact solution evaluated at nodes x_k. Therefore we can find the numerical error by

$$\mathcal{Y} - \widetilde{\mathcal{Y}} = A^{-1}\left(\mathcal{O}(h^2)\|y^{(4)}\|_\infty\right) \tag{5.53}$$

For reliable numerical solution, we will require $\|\mathcal{Y} - \widetilde{\mathcal{Y}}\|_\infty \to 0$ as $h \to 0$. This will need the estimate of $\|A\|_\infty$.

Lemma 5.2.3. *Let $0 \leq S \leq T$, that is, $0 \leq s_{ij} \leq t_{ij}$ for all elements. Then*

$$\|S\|_\infty \leq \|T\|_\infty.$$

5.2.2 Galerkin Method

5.3 Sturm Livouille Theory

5.3.1 Orthogonal Polynomials

5.3.2 Perturbation of Orthogonal Basis

When an orthonormal basis is perturbed, the orthogonality will be lost, however, the completeness might still be reserved when the perturbation can be controlled.

Definition 5.3.1 (Bessel system).

5.4 Exercises

5.4.1 Theoretical Part

Problem 5.4.1. *Show the Runge Kutta RK4 has an order of convergence* $p = 4$.

Problem 5.4.2. *Prove Lemma 5.1.24.*

Problem 5.4.3. *Determine the consistency order of the multi-step method*

$$y_{l+2} - y_l = \frac{h}{3}\left(f(t_{l+2}, y_{l+2}) + 4f(t_{l+1}, y_{l+1}) + f(t_l, y_l)\right) \tag{5.54}$$

Problem 5.4.4. *For what values of* γ*, the following method*

$$y_{l+3} + \gamma(y_{l+2} - y_{l+1}) - y_l = \frac{(3+\gamma)h}{2}\left(f(t_{l+2}, y_{l+2}) + f(t_{l+1}, y_{l+1})\right) \tag{5.55}$$

provides a convergent method, what is the order.

5.4.2 Computational Part

Problem 5.4.5. *Implement Runge-Kutta method RK4 and apply the method for the ODE*

$$y' = -y, \quad y(0) = 1. \tag{5.56}$$

Validate the convergence order on $[0, 1]$.

Problem 5.4.6. *Implement the Adam-Bashfort method for* $m = 2$*. The initial two steps can be computed first by RK4.*

Extended Reading

Gragg, W. B. (1965). On extrapolation algorithms for ordinary initial value problems. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(3):384–403.

Widlund, O. B. (1967). A note on unconditionally stable linear multistep methods. *BIT Numerical Mathematics*, 7(1):65–70.

CHAPTER 6

PARTIAL DIFFERENTIAL EQUATIONS

6.1 Finite Difference Method

6.2 Finite Volume Method

6.2.1 Godunov's Scheme

6.3 Pseudo Spectral Method

6.4 IMEX Method

6.5 Sector Operators

6.6 PINN

6.7 Exercises

6.7.1 Theoretical Part

6.7.2 Computational Part

CHAPTER 7

INTEGRAL EQUATIONS

In this chapter, we introduce the common numerical methods for integral equations.

7.1 Nyström Method

7.1.1 Weakly Singular Kernel

7.1.2 Singular Kernel

7.2 Galerkin Method

7.3 Boundary Integrals

CHAPTER 8

MATRIX COMPUTATION

8.1 Decompositions

8.1.1 LU Decomposition

8.1.2 QR Decomposition

8.1.3 Schur Decomposition

8.1.4 Singular Value Decomposition

8.2 Special Matrices

8.2.1 Sparse Matrices

8.2.2 Positive Matrices

8.2.3 Totally Positive Matrices

There are a handful of examples of totally positive kernels (TP). The TP matrices can be regarded as the discrete versions of the kernels Karlin (1968).

Example 8.2.1. *Spline Collocation Polynomial interpolation*

Example 8.2.2. *Green matrices*

Example 8.2.3. *Green's function of elliptic ODEs.*

8.2.4 M-Matrices

8.2.5 Circulant and Toeplitz Matrices

8.3 Iterative Schemes

8.3.1 Jacobi/Gauss-Seidel Iterations

8.3.2 Relaxation Schemes

8.3.3 Chebyshev iterations

8.4 Krylov Subspace Methods

8.4.1 Conjugate Gradient

8.4.2 GMRES

8.5 Preconditioning

8.6 Fast Matrix-Vector Multiplication

8.6.1 Hierarchical Semiseparable Matrices

Example 8.6.1. *Second order elliptic PDE*

8.7 Sketch Algorithms

Extended Reading

Karlin, S. (1968). *Total Positivity*, volume 1. Stanford University Press.

CHAPTER 9

TENSOR COMPUTATION

9.1 Tensor Decomposition

Part II

Foundations

CHAPTER 10

GRADIENT DESCENT

10.1 Fixed Step Length

10.2 Step Length Scheduling

Crossref Chebyshev iteration.

10.3 Momentum Methods

Part III

Topics

CHAPTER 11

EXTENDED TOPICS

11.1 Radial Basis Interpolation

11.2 Quadrature in multi-dimension

11.3 Stiff ODE

11.4 Signal Processing

11.5 Physics Models

11.5.1 Ising Models

11.6 Randomized Algorithm

11.7 Variational Methods

11.8 Sampling Algorithm

11.9 Parallel Computing

11.10 Optimal Transport

11.11 Point Cloud

11.12 Low Rank Approximation

11.13 Computation on Graph

CHAPTER 12

MATRIX INEQUALITIES

12.1 Golden-Thompson inequality

12.2 Klein's inequality

12.3 Araki–Lieb–Thirring inequality

12.4 Exercises

Problem 12.4.1 (Peierls–Bogoliubov). *Prove the following inequality*

$$Tr(e^A e^B) \geq Tr(e^{A+B}) \geq e^f$$

if A, B *are Hermitian matrices and* $Tr(e^B) = 1$, $f = Tr(Ae^B)$.

CHAPTER 13

COMPUTATIONAL INVERSE PROBLEMS

13.1 Regularization

13.2 Unique Continuation

Part IV

Miscellany

CHAPTER 14

PROOF ASSISTANT PROGRAMMING

Part V

Solutions to Exercises

CHAPTER 15

SOLUTIONS TO CHAPTER 1